

AGFZE Scrap Metal Trade Automation Platform

An AI Command Centre that turns unstructured purchase, sales, and shipping email & documents into validated, trackable trade transactions — with a human approval gate before anything reaches SAP, DMS, or the Excel/SharePoint trackers.

Document type: Product Requirement Document (PRD) and Technical Blueprint — Reconciled Edition v1.0

Derived from: AGFZE_BRD_v1.2 (10 Aug 2026), AGFZE_TDD_v1, current-state process documentation, and live tracker/contract samples

Prepared for: AGFZE — Scrap & FA Trading Business

Date: 23 August 2026

Document Control & How To Read This PRD

Reconciliation notice – please read first

This PRD was requested using a general-purpose master specification prompt that assumes the uploaded ZIP contains **source code** for an existing application, and that mandates a specific reference technology stack (Next.js/FastAPI/PostgreSQL/Keycloak/Gemini/GCP). On inspection, **AGFZE.zip contains no source code**. It contains the real Business Requirement Document (BRD v1.2) and Technical Design Document (TDD v1) for AGFZE's Scrap Trading AI Command Centre, three current-state process write-ups, three workflow diagrams, a project poster, and live samples of the loading sheet, sales contract, cost sheet and invoices this business actually runs on.

Rather than invent a fictitious codebase to satisfy the prompt literally, this document treats the BRD + TDD as the authoritative specification of *what* must be built, and applies the master prompt's own **Technology Reconciliation Rules** to decide *how* it is built: the mandated reference stack is used wherever it fits the real requirement, and is extended — never silently — wherever the real integration surface (Microsoft 365, SAP, DMS, a graph data layer) requires something the template doesn't mention. Every such decision is labelled inline using the tags below, exactly as the master prompt's Rule 4 requests.

SPECIFIED	Required verbatim by the master prompt's mandatory technology stack, and consistent with the real requirement.
SOURCE	Not present in the master prompt at all — discovered as a hard requirement in the BRD/TDD/process files and added per Reconciliation Rule 3.
ADAPTED	The master prompt specifies something, but the real integration constraints require a variant or a specific configuration of it.
N/A	Required by the master prompt's template but has no equivalent in this domain (e.g. the Indian legal-dataset clause), and is intentionally not carried forward.

Naming

The source documents refer to the client throughout only as “**AGFZE**” without expanding the acronym, and this document follows that same convention. Two real counterparties appear in the raw sample trackers used to derive the schema in this document (a supplier and a buyer); their names, exact contract numbers and transaction values are **not reproduced** here — every worked example in this PRD uses invented placeholder entities (e.g. “Meridian Metals Trading FZE”, contract **AGFZE/CU/EXA/041/2026-27**) so the document can be shared beyond the immediate deal team without exposing counterparty-confidential detail. Field names, business rules, thresholds and process steps are reproduced faithfully.

Scope of this edition

This edition covers Phase 1–3 of the programme (email intake through Purchase, Sales, FA processing, shipment tracking, tracker/SAP/DMS integration, and the web Command Centre) as defined in BRD §4 and TDD §1–2. Multi-tenant productisation for other trading houses (referenced in Section 15) is a forward-looking option, not part of the committed AGFZE scope.

Table of Contents

Section 1 — Problem Deep Analysis	5
Section 2 — Product Vision	10
Section 3 — Target Users	12
Section 4 — Core Platform Features	18
Section 5 — User Journey	45
Section 6 — Website Structure (Sitemap)	48
Section 7A — Frontend Pages Catalog	51
Section 7B — Backend API Endpoints Catalog	70
Section 8 — Page-by-Page UI Breakdown	85
Section 9 — Database Structure	88
Section 10 — Backend Structure (FastAPI)	110
Section 11 — Frontend Structure (Next.js)	115
Section 12 — API Contract	118
Section 13 — Feature Logic	120
Section 14 — UX Principles	129
Section 15 — Commercial Model (Monetization)	131
Section 16 — Security and Privacy	133
Section 17 — Scalability and Infrastructure	137
Section 18 — Developer Handoff Notes	140

SECTION 1

01 Problem Deep Analysis

SOURCE Every claim in this section is drawn from BRD §1–2 (Purpose & Background, Business Objectives), the three current-state process documents, and the three workflow diagrams — not inferred.

1.1 — What The Problem Is

AGFZE runs a document-heavy, multi-counterparty trading business — buying scrap metal (and a second commodity stream the business refers to internally as **FA**) from suppliers and brokers, and selling it on to industrial buyers, largely priced against the London Metal Exchange (LME) with a percentage discount or premium. Every deal generates a chain of six to twelve documents — broker email, purchase contract, provisional or final supplier invoice, loading sheet entry, cost sheet entry, a combined PDF document pack, an SAP entry, a DMS upload, an Original Bill of Lading (OBL), a sales contract, a sales invoice, a bank document set — and every one of those documents has to agree with every other one on quantity, price, container number and counterparty before the deal can close.

Today, none of that reconciliation is software-assisted. It happens because one person — the HOD — reads every incoming email, prints or manually cross-checks every invoice against the deal terms, types the same nine or ten data points into three or four different places (a Loading Sheet workbook, a Cost Sheet workbook, SAP, a document naming convention, a Shipment Status tracker), and personally signs the contract, the cost sheet and the invoice before anything moves downstream. The purchase workflow alone documents thirteen sequential manual steps from “broker informs of the deal” to “purchase file closed,” and the sales workflow documents thirteen more, with a measured ~7 minutes 20 seconds of manual data entry for the SAP step alone, repeated for every single container on every single deal.

This is simultaneously an **information problem** (the facts of a deal live in email text and PDF images, not in structured fields), a **process problem** (the same nine data points are retyped by hand into four to six different systems with no cross-check between them), and an **audit/access problem** (there is no single place to see “where is this container right now, and has its invoice actually been matched to its contract” — that answer currently lives only in the HOD’s head and in whichever Excel tab was last updated). It is not a cost-of-goods problem and not a pricing problem — AGFZE’s commercial terms and LME-hedging discipline are already sound; what is missing is the operating system that turns those terms into a governed, traceable, low-effort back office.

1.2 — Why It Happens Psychologically

- **Single-point cognitive bottleneck.** One senior person (the HOD) is the de facto validation engine for nearly every transaction — self-sourced deals are only ~10%

of volume, and ~90% still route through the HOD for broker-originated deals. This isn't a preference, it's a trust problem: nobody else has full visibility across purchase, sales and payment simultaneously, so the safest thing for the organisation to do is keep one accountable person in the loop for everything, even at the cost of that person becoming a bottleneck. This is a classic single point of failure created by *information asymmetry*, not by any one person wanting control.

- **Loss aversion on “the number.”** A misplaced decimal on an LME percentage or a quantity tolerance directly costs real money on a live commodity position. When the downside of a typo is a five- or six-figure invoice mismatch, people rationally over-invest in manual, repeated cross-checking rather than trusting a system they cannot see inside — this is precisely why the Loading Sheet is described as the “single source of truth” even though it is a spreadsheet: a spreadsheet is at least *inspectable* in a way a black-box system is not.
- **Status-quo bias reinforced by tolerable pain.** Every individual step (retype the invoice into the loading sheet, rename the PDF, run the Power Automate script for SAP) takes minutes, not hours, so no single step feels bad enough to justify the perceived risk of replacing the whole chain. The pain is death by a thousand five-minute cuts rather than one dramatic failure, which is exactly the profile of a problem that persists for years despite everyone agreeing it should be fixed.
- **Fear of losing the audit trail.** Any change to a payment term or a price fixation clause today requires “an OVERRIDE with a recorded reason” — people already know they need to be able to explain, months later, why a number changed. That need is currently met by tribal memory and folder discipline, which is fragile, and which makes people reluctant to change the underlying process even though they dislike it, because at least they know how to defend today's process to an auditor.
- **Ambiguity between “provisional” and “final.”** A large share of deals are priced against an LME percentage that is only fixed later (a “provisional” or “quotation-period” contract) — so the same container can legitimately have two, three or four valid documents (provisional invoice, differences invoice, credit/debit note, final invoice) across its life. Systems and people alike default to conservative, manual handling here because getting the sequencing wrong creates a compounding reconciliation error, not a one-off mistake.

1.3 — Who Experiences This Problem Most

- **Organisation type:** a mid-size, high-transaction-velocity commodity trading business operating out of a UAE free-zone entity, buying and selling internationally (India, China, Japan and other markets are explicitly named as distinct documentation territories), settling in USD, and interfacing with SAP as its ERP of record.
- **Professional roles most affected:** the HOD / business approver (the deepest bottleneck), Purchase-desk staff, Sales-desk staff, Logistics/Operations staff who own container and shipment tracking, and Finance staff who own invoice-to-payment matching. FA-stream staff run a structurally identical but separately owned process.

- **Skill level and technical comfort:** high domain expertise in trading, contracts and LME pricing mechanics; day-to-day tools are Outlook, Excel and a SAP front end — not developer tools. Any new system has to feel like “a better version of the Excel tracker,” not like an engineering platform, or adoption will fail regardless of how correct the automation is.
- **Regulatory/compliance context:** cross-border trade documentation (certificates of origin, chemical analysis certificates for China-bound cargo, Form 6/Form 9 for India-bound cargo), UAE free-zone company obligations, bank-driven trade-settlement documentation (CAD via DHL, or TT against telex release), and an internal audit expectation that every workflow action, approval, exception and downstream update is logged and traceable.
- **Geographic/jurisdictional constraints:** counterparties and destination-country paperwork differ by territory (India, China, Japan each require a different document set and, in places, different wording on an otherwise identical contract template), so the same commodity and quantity can require different mandatory-document logic purely because of where the container is going.

1.4 — Real-Life Scenarios

Scenario 1 — The provisional-invoice trap. Priya, a purchase-desk officer, receives a provisional supplier invoice for a copper cargo priced at “LME minus 6%” with the LME rate still unfixed. She enters it into the Loading Sheet and the Cost Sheet by hand. Three weeks later the final invoice arrives with the LME now fixed and a different total. Nothing in the current process automatically tells her that the earlier provisional entry needs to be superseded rather than duplicated — she has to remember which of six open rows belongs to this container, and if she is on leave that week, whoever covers for her may not.

Scenario 2 — The quantity-mismatch that isn’t caught until the bank stage. A sales contract for five containers is raised. Six B/Ls arrive because a partial shipment was split. Today, nobody automatically compares “invoices raised” against “containers on the contract” — the mismatch (BR-08’s “duplicate/repeated references”) surfaces only if a human happens to notice it before the bank-document package goes out, which is the single most expensive place to catch this kind of error, because customer-facing paperwork has already been signed and dispatched via DHL.

Scenario 3 — The HOD as single approver during a compressed window. DHL dispatch of the bank documents must happen within a tight window once the OBL and invoice are ready. Every one of those documents currently requires the HOD’s physical or e-signature on the invoice, the cost sheet and the contract. If the HOD is unavailable at the exact moment three deals need signing, every one of those deals stalls — not because the commercial decision is unclear, but because the governance step has no fallback path.

Scenario 4 — The container that goes quiet. A container’s shipment status stops updating in the Shipment Status tracker because the shipping line’s website changed its container-search flow, or because nobody has checked it in a few days. The

customer, expecting an ETA, calls Operations. Operations has to manually check the shipping line's tracking page and cross-reference it against a tab in an Excel workbook shared over SharePoint before they can answer — and if the price is still unfixed (a quotation-period deal), a delayed ETA has real financial consequences for when the price gets locked in.

Scenario 5 — The commodity-description mismatch that only a human would catch. A purchase deal is logged as “copper scrap – cobra.” The matching sales contract, generated a few days later for the same underlying containers but with China-specific wording, needs to describe the goods as “recycled raw material for copper” for customs purposes on the China leg while remaining the *same commodity code* on both sides. If a junior staff member preparing the sales paperwork doesn't know this territory-specific nuance, the two contracts can end up describing different goods for the same containers — exactly the kind of “mismatch/error” the current process depends on one experienced person catching by eye.

1.5 — Why This Problem Is Globally Relevant

Nothing about “trade documents live in email and get manually reconciled into spreadsheets before an ERP entry is made” is specific to scrap metal, to the UAE, or to AGFZE. Any physical-commodity trading desk — metals, agri-bulk, energy, recycled materials — runs the same shape of process: broker-originated deal by email, provisional pricing against a public index, physical-document chain (contract, invoice, bill of lading) that must match before payment, and an ERP/DMS pair that only wants clean, final data. The specific documents differ (a certificate of origin instead of a chemical-analysis certificate, a different price index instead of LME) but the underlying automation problem — classify, extract, match, validate, gate on human approval, then write once downstream — is identical. This is precisely why the platform's architecture is built as a configurable rule engine and a document-type/field-mapping layer rather than as scrap-metal-specific code: the same Command Centre pattern is directly reusable for any other commodity trading house facing the same broker-email-to-ERP gap, which is exactly the basis for the forward-looking commercial option described in Section 15.

1.6 — Emotional and Behavioural Aspects

People inside this process describe it, implicitly, in the language of *vigilance* rather than *productivity* — the workflow diagrams themselves flag “if errors flagged → rectify and resend,” “if issues raised → rectify and resend,” “resolve bank queries promptly to avoid payment delays” as first-class steps, not edge cases. That framing — constant readiness for something to be wrong — produces low-grade, chronic vigilance fatigue rather than a single dramatic stressor, and it concentrates disproportionately on the HOD, who is the named approver on nearly every governance gate in both the purchase and sales workflows. The emotional cost of getting a step wrong is not embarrassment, it is a real financial exposure on a live commodity position, which is why every meaningful gate in the current process already has a human signature attached — the organisation has already, correctly, decided that machine judgement alone is not enough here. Any product for this

problem has to *keep that feeling of control* for the human approver while removing the repetitive labour around it, or it will be experienced as a loss of safety rather than a productivity gain.

1.7 — Why Existing Solutions Fail

Existing approach	Why it falls short
Excel trackers (Loading Sheet, Cost Sheet, Shipment Status) over SharePoint	Correct as a shared ledger, but purely passive — nothing validates an entry against the contract, checks a tolerance, or flags a duplicate. “Single source of truth” only for what has already been typed in correctly.
Manual email triage in Outlook	Scales linearly with headcount and with deal volume; there is no persistent, queryable record of “what came in, what was it, was it actioned” beyond the mailbox itself.
Power Automate scripts for SAP entry	Automates <i>keystrokes</i> (TC/DPR sheet population) but not <i>judgement</i> — the script still runs on manually transcribed, unvalidated data, so it automates the risk of a typo just as fast as it automates the typing.
Point-in-time manual OCR/reading of PDFs	Works, but is not reusable, generates no structured record, and depends entirely on the reader’s domain knowledge (e.g. recognising an India-vs-China wording nuance) being current and undocumented.
Generic RPA / BPM tooling	Can move files and trigger steps, but has no native concept of a trade document, a commodity code, an LME-linked price clause, or a matching rule between a contract and its invoices — every one of those would have to be hand-built anyway, at which point a purpose-built agent architecture (Section 13) is the more direct investment.
Hiring more back-office staff	Adds throughput but not governance — more people transcribing the same unvalidated data into the same disconnected systems increases, rather than reduces, the chance of an undetected mismatch, and does nothing for the single-approver bottleneck.

SECTION 2

02 Product Vision

2.1 — What This Platform Aims To Solve

The AGFZE AI Command Centre is a governed, human-approved automation layer that sits between AGFZE's existing mailbox and its existing systems of record (the Loading Sheet / Cost Sheet / Shipment Status trackers, SAP, and the DMS) and does three things no current tool does together: (1) reads incoming trade email and attachments and turns them into structured, classified data automatically; (2) matches and validates that data against the open transaction, the contract, and configured business rules (quantity tolerance, mandatory documents, duplicate detection) before anything is treated as fact; and (3) writes downstream — to the trackers, to SAP, to the DMS — **only after** a named human has approved it, with a complete, queryable audit trail from the original email through to the downstream posting.

2.2 — How It Helps Users Make Decisions or Progress

Today a purchase or sales officer's job is 80% transcription and cross-checking and 20% judgement. The platform inverts that ratio: it performs the transcription and the first-pass cross-checking automatically (extraction, matching, tolerance and completeness checks), and it surfaces to the human exactly the cases that need judgement — an exception queue ranked by age, owner and reason, not a firehose of every transaction. Before the platform, a user has to *discover* that something is wrong (a mismatched container, a missing certificate, a sixth invoice against a five-container contract) by manually re-deriving the comparison; after the platform, the system has already run that comparison and the user's job is to *decide* what to do about the specific, named discrepancy it found. The user goes from "is anything wrong here?" to "here is what's wrong, and here is the evidence" — which is a fundamentally faster and less error-prone decision to make.

2.3 — The Core Philosophy of the Platform

- **Read and reconcile everything; commit nothing without a human.** The system is allowed to ingest, classify, extract, match and pre-validate autonomously. It is never allowed to post to SAP, upload a final document to the DMS, or overwrite an approved tracker row without a captured human approval — this is the "golden governance rule" carried over unchanged from the source design, and it is non-negotiable in this PRD.
- **The tracker/portal relationship is additive, not adversarial.** The platform does not ask AGFZE to abandon Excel on day one. It reads the trackers as a reference source during transition and writes back to them under configuration, while the portal becomes the primary transaction record — Excel is phased out at AGFZE's pace, not the platform's.

- **Configuration over hard-coding.** Tolerances, mandatory-document checklists, commodity codes, territory-specific document sets and approval thresholds are configuration, not code, because they will change as AGFZE's business changes and because Section 1.5 depends on this same engine being reusable outside scrap metal.
- **Traceability is a feature, not a compliance afterthought.** Every number the platform ever shows in a dashboard or a report must be able to answer, on demand, "which email, which document, and whose approval produced this."

2.4 — The Transformation the User Experiences

Before: A purchase or sales officer opens Outlook, manually reads a supplier invoice, prints or eyeballs it against the contract, retypes eight to ten fields into the Loading Sheet, retypes them again into the Cost Sheet, waits for the HOD to be free to sign, manually renames a merged PDF, and runs a Power Automate script by hand for the SAP entry — all before they can even start on the next deal. Confidence that nothing has been missed comes entirely from personal diligence and experience, not from the system.

After: The same officer opens the AI Command Centre to a queue that already shows the transaction pre-populated from the incoming email and invoice, a confidence-scored extraction with the source document one click away, and a clear flag if something doesn't match (wrong container, missing certificate, quantity outside tolerance). Their job is to review the flagged fields, correct anything the AI got wrong, and route it for approval — the retyping, the cross-referencing and the SAP keystrokes are gone. Confidence now comes from a system that actively checked the numbers, not from having personally re-derived them.

2.5 — The AI-Assisted Safety Philosophy

AI in this platform performs classification, extraction and first-pass matching — it does not make the trading, pricing or approval decision, and it never triggers an irreversible downstream action by itself. The platform will surface, wherever AI-derived data is shown to a user, language equivalent to:

"AI-extracted information may contain errors and must be verified against the source document before approval. This platform does not replace the designated approver's review. For transactions above configured value or risk thresholds, or where source documents conflict, escalate to the HOD before proceeding."

This disclaimer, and a visible confidence score plus a one-click link back to the source document, appears on every extraction result, every matching/validation outcome, and every generated draft document (draft sales contract, draft invoice) throughout the product — never only on a single terms-of-use page — because BR-11 requires confidence score, source-document reference and manual override history to be retained for every AI output, and a disclaimer that only exists in a footer would not meet that bar.

SECTION 3

03 Target Users

Five personas are defined, mapped directly to the stakeholder roles named in BRD §3 (Business Approver/HOD, Purchase User, Sales User, Logistics User, Finance User). The FA-stream user is intentionally not modelled as a sixth, separate persona: BRD §6.3 and the poster both describe FA processing as structurally identical to Purchase/Sales processing with a distinct document/field configuration, so an FA officer is, in product terms, a Purchase or Sales Desk Officer working a different commodity-code configuration — covered by Personas 2–3 with an FA-scoped view.

Rashid Al-Marri

HOD / BUSINESS APPROVER

Age & location	44, Dubai, UAE (free-zone HQ)
Education	Bachelor's in commerce/trade finance; two decades in metals trading
Occupation & authority	Head of Department — final signing authority on contracts, invoices and cost sheets across both Scrap and FA; the single named approver on almost every governance gate today
Primary problem	Is the accountable approver for a volume of deals that already exceeds what one person can safely eyeball in detail; cannot delegate approval without losing the audit trail that currently only exists in his own review
Current behaviour	Reviews and signs invoices, cost sheets and contracts individually, largely from forwarded emails and printed packs; personally reconciles the daily payments list
Pain points	Becomes the bottleneck the moment three deals need signing at once; has no single dashboard of exception ageing across Purchase, Sales, FA and Logistics; audit defence today means reconstructing history from email and spreadsheet versions
Motivations	Wants approval load reduced to genuinely judgement-requiring cases; wants one screen that shows every open exception, its age and its owner; wants to be able to defend any number to an auditor in minutes, not days
Fears	Losing visibility/control if approvals are automated away; a system silently posting to SAP or releasing a document without his sign-off; being blamed for an error the system introduced but he didn't catch
Technical comfort	Moderate — comfortable with Outlook, Excel and SAP's transaction screens; not a developer and will not tolerate a tool that feels like one
Language	English primary; reads India/China/Japan contract variants but does not need the portal itself localised
Needs from the platform	A single approval queue ranked by age/value/risk, one-click access to the source email and document for anything he's asked to approve, and an irreversible-action gate that is provably enforced, not just documented
How he finds it	Commissioned it — he is the internal sponsor identified in BRD §3 and Section 9 (UAT/Go-Live) as final sign-off authority

Fatima Hussain

PURCHASE DESK OFFICER

Age & location	29, Dubai, UAE
Education	Bachelor's in business/logistics; three years on the purchase desk
Occupation & income	Mid-level purchase officer; no approval authority — prepares transactions for HOD sign-off
Primary problem	Spends most of her day transcribing supplier invoices into the Loading Sheet and Cost Sheet and manually checking them against the purchase contract and LME hedge rate
Current behaviour	Prints or screen-checks every invoice line by line against the contract terms; retypes the same fields into two spreadsheets; renames merged PDFs by hand; runs the SAP Power Automate script herself
Pain points	Re-entering identical data multiple times; discovering a provisional-vs-final invoice conflict only when she happens to notice two rows for the same batch; no visibility into whether a document pack is actually complete before she submits it
Motivations	Wants the transcription done for her so she can focus on judgement calls; wants a clear “what’s missing” list instead of having to derive it herself
Fears	An automated tool quietly getting a number wrong and her not catching it before the HOD signs; being replaced rather than supported
Technical comfort	High comfort with Excel formulas and SAP transaction codes; low tolerance for anything that requires IT support to use day-to-day
Language	English and Arabic; works India/China purchase paperwork variants regularly
Needs from the platform	Pre-filled transaction records from extracted invoice/contract data with confidence scores and inline correction, automatic matching against the Loading Sheet/batch number, and a completeness checklist per transaction type
How she finds it	Rolled out to her desk as part of the phased AGFZE go-live; onboarded during UAT

Wei Chen Tan

SALES DESK OFFICER

Age & location	33, Dubai, UAE (handles China/APAC accounts)
Education	Bachelor's in international trade
Occupation	Sales officer preparing sales contracts and invoices against purchase-side deal terms; no final approval authority
Primary problem	Must keep a sales contract's commodity description, quantity and pricing clause perfectly consistent with the matching purchase contract, across territory-specific document variants, while the price may still be unfixed
Current behaviour	Copies an existing contract template and manually edits the highlighted fields; waits on a scanned OBL before generating the SAP invoice number; manually tracks whether each customer has fixed their price yet
Pain points	A commodity-description or quantity mismatch between the purchase and sales sides is only caught by careful manual reading; six invoices can get raised against a five-container contract before anyone notices; provisional-price contracts need a manual clause rewrite when the customer later fixes price
Motivations	Wants the platform to flag a purchase/sales mismatch automatically and to track price-fixation status per customer without a side spreadsheet
Fears	Sending a customer-facing contract or invoice with an error that damages a customer relationship; a system generating and sending a document without her review
Technical comfort	Comfortable with Word/Excel templates; moderate comfort with new software given clear guidance
Language	English and Mandarin; regularly handles China-specific contract wording
Needs from the platform	Auto-linked purchase ↔ sales transaction view, a contract/invoice-vs-container-count validator, a price-fixation tracker per customer, and drafts (never auto-sent documents) generated from approved templates
How she finds it	Rolled out to the sales desk in the same phased release as the purchase desk

Arjun Menon

LOGISTICS & SHIPMENT COORDINATOR

Age & location	31, Dubai, UAE
Education	Diploma in supply chain / shipping operations
Occupation	Owns container and vessel tracking, KYC coordination, and the Shipment Status tracker
Primary problem	Manually checks shipping-line websites container by container and updates a shared Excel tracker; has no single dashboard tying shipment status back to invoice and payment status
Current behaviour	Starts tracking a container only once its invoice has been raised; manually reconciles OBL originals against the file; fields customer ETA queries by checking the tracker and the carrier site separately
Pain points	Status goes stale when a carrier's site changes or nobody has time to check; delayed/missing shipment status has no automatic ageing or ownership; post-delivery issues (damage, detention, quality claims) are tracked ad hoc
Motivations	Wants scheduled, automatic status pulls where a carrier API/website supports it, and a dashboard that shows every open shipment's status, ETA and any exception in one place
Fears	Telling a customer a wrong ETA because the tracker was stale; being blamed for a carrier-side delay that was never his to control
Technical comfort	Comfortable with Excel and shared trackers; not a developer
Language	English, working knowledge of shipping terminology across territories
Needs from the platform	A shipment dashboard unifying container, B/L, carrier status and linked transaction; automatic ageing and owner assignment for stalled shipments; one-click access to the linked purchase/sales record
How he finds it	Introduced through the Logistics/Operations rollout track

Meera Nair

FINANCE / RECONCILIATION OFFICER

Age & location	36, Dubai, UAE
Education	Bachelor's in accounting/finance
Occupation	Reconciles invoice values against contract terms and bank payment instructions (CAD/TT); resolves bank queries
Primary problem	Invoice-value and payment-term checks are manual, and any override to a payment term (e.g. shifting a no-advance counterparty to 100% CAD) is not consistently recorded with a reason
Current behaviour	Cross-checks invoice value/rate/quantity against the contract by eye; chases bank queries by phone/email; reviews the daily payments list assembled for HOD approval
Pain points	Out-of-tolerance invoice values are found late, sometimes only when the bank raises a query; no structured log of payment-term overrides and the reason for each
Motivations	Wants invoice-value/tolerance checks run automatically before a document reaches the bank-dispatch stage, and every payment-term override captured with a mandatory reason
Fears	A payment released against a mismatched or duplicate invoice; being unable to explain a historical override during an audit
Technical comfort	High comfort with SAP and Excel; moderate comfort with new systems
Language	English
Needs from the platform	Automated invoice-value/tolerance validation prior to bank dispatch, a mandatory-reason override log, and a finance-scoped exception queue
How she finds it	Introduced through the Finance rollout track alongside Purchase/Sales

SECTION 4

04 Core Platform Features

Eleven features are specified, one per functional-requirement group in BRD §6 (FR-EI, FR-PU, FR-FA, FR-SA, FR-LG, FR-IN, FR-DB, FR-RP), split where BRD groups two concerns under one heading. Every feature names the exact FR/BR IDs it implements so traceability back to the BRD is direct.

1. Email Intake & Classification

INGESTION · FR-EI-01...06

What it does

Polls (or receives, via Microsoft Graph change notifications) the approved AGFZE group/shared mailbox, captures every inbound message and its attachments, and classifies the message into one of: `purchase`, `sales`, `fa`, `logistics`, `approval`, `follow_up`, `informational`, or `exception`. A unique Request ID and transaction shell are created for every message that is not purely informational.

Why it exists

Every downstream workflow starts from an email today (Scenario 1–5, Section 1.4); FR-EI-01 through FR-EI-05 exist specifically to remove the manual step of a person reading, triaging and hand-labelling every inbound message before work can begin.

User workflow

1. System-initiated: no user action required for capture. A new row appears in the **Inbox / Request Queue** workbench screen within the configured polling interval (default 2 minutes) or immediately on a Graph push notification.
2. The assigned business user (routed by classification: Purchase/Sales/FA/Logistics) opens the request from the queue.
3. The user sees the AI-assigned category, a confidence score, the original email body, and every attachment thumbnail.
4. If classification confidence is below the configured threshold (default 0.75), the row is flagged *Needs Review* and the category field is user-editable with a required reason for the change.
5. User confirms or corrects the category; the request proceeds automatically into document classification (Feature 2).

Data required

- From Microsoft Graph: message id, internetMessageId, from/to, receivedDateTime, subject, body (text), attachment list (filename, contentType, size, contentBytes reference)
- From configuration: approved-mailbox list, category taxonomy, confidence threshold, keyword/sender routing hints
- Pydantic `EmailIntakeIn`: `graph_message_id: str`, `mailbox: EmailStr`, `received_at: datetime`, `subject: str`, `body_text: str`, `attachments: list[AttachmentRef]`

Processing logic

1. Email Intake & Classification Agent subscribes to the mailbox via a Microsoft Graph delta query / webhook (Feature 8) and normalises the payload.

2. Email body + attachment metadata (not attachment bytes yet) is sent to `gemini_service.classify_email()` with a structured-JSON system prompt enumerating the eight allowed categories and requiring a confidence score and a one-sentence justification.
3. A Request row and an EmailMessage row are created; a Request ID of the form `REQ-{yyyyMMdd}-{seq}` is issued (BR-01: every transaction must have a unique Request ID from email/portal intake).
4. Attachments are queued to Feature 2 for document-type classification and extraction.

Output generated

A Request record with status `Classified`; a queue entry visible in the Inbox workbench; a domain event `RequestClassified` published for the audit log and any downstream subscribers.

AI integration details

Gemini 2.5 Flash, text-only call (no document bytes at this stage), JSON-mode structured output: `{"category": str, "confidence": float, "rationale": str}`. Provider is selected via `AI_PROVIDER` (Section 13.1); a classification failure or timeout routes the request to *Needs Review* rather than blocking intake.

Background job details

Runs as a scheduled Cloud Run Job (default every 2 minutes) for mailbox polling, plus an event-driven path via a Graph webhook subscription (renewed automatically before its ~3-day expiry) for near-real-time capture. Both paths converge on the same `ingest_email()` service function to avoid duplicate processing (de-duplicated on `graph_message_id`).

Technical notes

FR-EI-06 explicitly does not require a new dedicated mailbox — the agent is granted least-privilege, read-only (initially) access to the existing shared mailbox via an Azure AD app registration using Client Credentials Flow (Section 16.1), scoped to `Mail.Read.Shared` only until the rollout plan (Section 18.2) enables any write-back scope.

2. Document Classification & Extraction Engine

AI CORE · FR-EI-03, FR-PU-02,
FR-SA-02, FR-FA-02

What it does

Takes every attachment on a classified request and (a) classifies it as invoice / contract / B-L / shipping document / tracker export / approval evidence / FA document / unknown, then (b) extracts a structured, typed field set specific to that document type, with a per-field confidence score and a bounding-box or page/paragraph reference back into the source file.

Why it exists

This is the single engine that removes the “Dev prints the invoice, cross-checks it manually and enters the details” step (Purchase workflow, step 6) and the equivalent manual reading step on the sales side — it is shared by Features 3, 4 and 5 rather than re-implemented per document type.

User workflow

1. User opens a request's Document Review screen and sees each attachment as a thumbnail with its predicted type and confidence badge.
2. User clicks an attachment to open a split view: the rendered page (PDF/ image) on the left, extracted fields on the right, each field highlighted against the confidence score (green ≥ 0.9 , amber 0.7–0.9, red < 0.7).
3. Low-confidence fields are pre-focused for the user; the user edits the value inline if wrong (Zod-validated per field type) and the correction is stored as an override with the original AI value retained (BR-11).
4. User clicks **Confirm Extraction**; the record moves to **Extracted** and becomes eligible for matching (Feature 3/4/5).

Data required

- Raw document bytes (PDF, image, DOCX, XLSX) fetched from Graph/ attachment store or portal upload
- Document-type field schemas (configuration): invoice fields, contract fields, B/L fields, tracker fields (Section 9.3 table `document_type_schemas`)
- Pydantic `ExtractionResultOut`: `document_id: UUID`, `document_type: DocTypeEnum`, `fields: list[ExtractedField]` where `ExtractedField = {name, value, confidence, source_ref, overridden: bool}`

Processing logic

1. File-type detection (magic bytes, not extension) routes the file: text-layer PDF → PyMuPDF text/layout extraction first; scanned/image PDF or photo → page rasterisation and Gemini 2.5 Flash multimodal document understanding; DOCX → python-docx text extraction; XLSX/XLS/CSV → pandas/openpyxl parse against the known tracker-column layout (Section 9,

- informed by the real Loading Sheet/Sales Contract/Cost Sheet structures reviewed during discovery).
2. Document Classification Agent runs a first Gemini call against the extracted text/first-page image to assign the document type from the configured taxonomy.
 3. Document Extraction Agent runs a second, type-specific Gemini call using a per-type JSON schema (invoice schema: invoice number, contract/PO reference, batch number, commodity code, quantity, rate, currency, amount, container/B-L reference, dates; contract schema: contract number, buyer, seller, commodity, quantity ± tolerance, price base/LME%, Incoterm, ports, payment terms, tolerance).
 4. Each field's confidence score and page/paragraph reference are persisted alongside the value; scores below threshold are marked `needs_review`.
 5. Long/multi-page documents are chunked (per-page for scanned PDFs) with a final consolidation pass that resolves field values across chunks.

Output generated

An ExtractionResult row per document; per-field values written to the associated TradeTransaction/PurchaseLeg/SalesLeg draft; a domain event `ExtractionCompleted`; the original file stored immutably (Section 13.9) and never overwritten.

AI integration details

Gemini 2.5 Flash multimodal for scanned/image documents (this is the one point where Gemini's document-vision capability is used directly for OCR-equivalent reading, rather than a separate OCR product); PyMuPDF used first wherever a genuine text layer exists, both to reduce AI cost/latency and because deterministic text extraction is more reliable than vision-based reading for a clean digital PDF. Vertex AI is the fallback provider path per `AI_PROVIDER` switching if Gemini API quota is exhausted (Section 17.3).

Background job details

Runs as an async FastAPI background task for small/simple documents (single-page invoice) and escalates to a Cloud Run Job for multi-page or multi-document packs, to avoid holding an HTTP request open during extraction. Job status is polled by the frontend via `GET /api/v1/jobs/{job_id}/status` (Section 12.7).

Technical notes

The extraction schema is configuration-driven per document type and per territory variant (India/China/Japan contract wording differences, Section 13.3) so that adding a new mandatory field or a new territory does not require a code change — only a schema-configuration update.

3. Purchase Transaction Processing & Matching

CORE WORKFLOW ·
FR - PU - 01...06

What it does

Registers the purchase-side input pack (supplier invoice, purchase contract, loading details, supporting documents), matches the extracted data to an existing Loading Sheet entry or opens a new one, validates it against configured business rules, creates/updates the governed portal transaction record, and routes it for purchase approval.

Why it exists

Directly replaces purchase workflow steps 5–8 (supplier invoice received → entered into loading sheet → contract prepared → entered into cost sheet) — the highest-frequency, most repetitive part of the current process (Section 1.1, 1.4 Scenario 1).

User workflow

1. Purchase officer opens a classified purchase request from the queue; sees the pre-filled Purchase Transaction Workspace: supplier, batch number, commodity code, quantity, LME%, provisional/final flag, linked documents.
2. Officer reviews the Matching panel: the system shows either “Matched to Loading Sheet batch #{n}” with a similarity score, or “No match found — create new batch” with the proposed next sequential batch number.
3. Officer reviews the Validation panel: a checklist of BR-04 (mandatory documents present), BR-05 (quantity within configured tolerance, default $\pm 5\%$), BR-06 (invoice value/rate/amount within tolerance) — each shown pass/fail with the specific field and threshold.
4. If any check fails, the record is blocked from proceeding and appears in the Exception Queue (Feature 7) instead of the approval queue.
5. If all checks pass, the officer clicks **Submit for Approval**; the record moves to **Approval Pending** (Feature 7).

Data required

- Required fields (BRD §7.3 Purchase domain): Request ID, supplier name, supplier invoice number, purchase contract number, batch number, quantity, rate, amount, currency, Loading Sheet reference, mandatory-document status
- Pydantic `PurchaseLegIn`: `supplier_name: str`, `supplier_invoice_no: str`, `contract_no: str`, `batch_no: str | None`, `commodity_code: CommodityCodeEnum`, `quantity_mt: Decimal`, `lme_percent: Decimal | None`, `price_basis: Literal["fixed", "lme_percent"]`, `currency: Literal["USD"]`, `amount: Decimal`, `invoice_status: Literal["provisional", "final"]`

Processing logic

1. Matching Agent searches open TradeTransaction/PurchaseLeg records by batch number (if present on the document), else by contract number + commodity + supplier + quantity proximity (rapidfuzz `token_sort_ratio`, threshold 90) to catch near-duplicate supplier-name spellings.
2. If a provisional invoice already exists for the same batch and a final invoice now arrives, the Matching Agent links the new document as a *supersession* of the provisional one rather than a new row (directly implements the “provisional invoice re-entered once final invoice received” rule from the purchase workflow diagram).
3. Validation Agent runs BR-04/05/06/08 against the matched record and configured tolerance table.
4. On pass, the System/Workflow service creates or updates the PurchaseLeg + TradeTransaction record and, where the Loading Sheet remains in active use during transition (BR-09), synchronises the corresponding row via the Microsoft Graph Excel API.

Output generated

A governed PurchaseLeg record with status `Validation Pending` → `Approval Pending`; a synced Loading Sheet row where applicable; domain events `TransactionMatched`, `ValidationPassed` / `ValidationBlocked`.

AI integration details

Matching uses rapidfuzz (deterministic, not Gemini) for reference-string similarity, and Gemini only where free-text disambiguation is genuinely needed (e.g. a supplier name spelled two different ways across a contract and an invoice) — this split is deliberate: BR-11's confidence/override trail is most useful where a judgement call was made, and a pure string-similarity match doesn't need one.

Background job details

Matching and validation run synchronously in the request/response cycle for a single document (typically <2s); a full document pack (invoice + contract + supporting docs arriving together) is processed as a Cloud Run Job with per-document sub-steps reported through the job's `progress` field.

Technical notes

Tolerance values ($\pm 5\%$ quantity, invoice-value tolerance) are marked in BRD §7.1 (BR-05) as an indicative draft subject to AGFZE confirmation — the schema stores them as per-commodity, per-tenant configuration rows, never as a hard-coded constant, specifically so this confirmation does not require a deployment.

4. Sales Transaction Processing & Document Generation

CORE WORKFLOW ·
FR-SA-01...06

What it does

Registers the sales-side trigger (OBL, shipping confirmation, or an AGFZE-approved draft B/L), extracts and matches sales-side data against the linked purchase/cargo record, validates invoice readiness and mandatory supporting documents, and — where AGFZE's rules permit — generates a draft sales contract or invoice from an approved template for human review before it is ever sent.

Why it exists

Replaces sales workflow steps 2–9 (verify & enter deal → prepare sales contract → generate SAP invoice number → prepare sales documents → verify, sign & send) and directly targets Scenario 2 and Scenario 5 in Section 1.4 — the invoice-count-vs-contract-quantity mismatch and the purchase/sales commodity-description mismatch.

User workflow

1. Sales officer opens a classified sales request; the workspace shows the linked purchase transaction (if matched), customer, contract type (fixed price or LME%-provisional), and territory (India/China/Japan document-set variant).
2. Officer reviews the Cross-Contract Consistency panel: commodity code compared field-by-field against the linked purchase leg, with any mismatch (e.g. differing commodity description for the same code) highlighted in red per the "MISMATCH / error" rule from the contract-process diagram.
3. Officer reviews the Invoice-vs-Contract-Quantity panel: a running count of invoices/containers raised against the contract's total quantity — fewer than contracted shows an amber "awaiting remaining shipments" state; more than contracted is hard-blocked as a red flag requiring HOD review (implements the diagram's explicit validation rule).
4. If the contract is LME%-provisional and the customer has since fixed price, the officer is prompted to record the fixation date/rate; the system marks clause 3 for revision and clause 4 for removal in the generated draft rather than requiring a manual full rewrite.
5. Officer clicks **Generate Draft**; a draft DOCX/PDF is produced from the approved template with the highlighted "changes every deal" fields populated; the officer reviews and edits before routing for HOD signature (Feature 7) — the platform never sends a customer-facing document automatically.

Data required

- Required fields (BRD §7.3 Sales domain): Request ID, customer name, sales invoice number, sales contract/transaction reference, batch/transaction reference, quantity, rate, amount, currency, OBL/draft-B-L reference, mandatory-document status
- Pydantic `SalesLegIn`: `customer_name: str`, `territory: Literal["india", "china", "japan", "other"]`, `linked_purchase_leg_id: UUID | None`, `contract_no: str`, `price_basis: Literal["fixed", "lme_percent"]`, `customer_fixation_status: Literal["unfixed", "fixed"]`, `bl_reference: str`, `containers: list[str]`, `payment_condition: Literal["CAD", "TT"]`

Processing logic

1. Matching Agent links the sales request to the purchase-side TradeTransaction by shared batch/contract/container reference.
2. Validation Agent checks: mandatory-document completeness for the territory (e.g. certificate of origin + Form 6/Form 9 for India, packing list + certificate of origin for China), commodity-code consistency against the linked purchase leg, and invoice-count vs. contract-quantity.
3. Where a draft is permitted (AGFZE-configured; final committed posting always waits for final shipping-document validation per BR-07), the `docx_service` populates the approved sales-contract or invoice template — territory-specific document set and wording variant selected by the `territory` field, not a separate template per country maintained by hand.
4. Payment-condition logic branches: CAD → DHL dispatch reference required before closure; TT → telex-release confirmation required before closure.

Output generated

A governed SalesLeg record; a generated draft document (DOCX and PDF) linked to its source evidence and awaiting signature; domain events `TransactionMatched`, `ValidationPassed` / `ValidationBlocked`, `DocumentPackGenerated`.

AI integration details

Gemini generates the structured *content* for a draft (which clause to mark for revision, which fields to populate) as JSON; the binary DOCX/PDF itself is always produced by python-docx/PyMuPDF from an approved template — Gemini is never used to generate the binary file directly (consistent with the reference stack's document-processing rule).

Background job details

Draft generation runs as an async background task (typically a few seconds); the frontend polls job status and displays the rendered draft inline once `completed`, with a download link and an inline "request changes" action that re-queues generation with edited fields.

Technical notes

The commodity-code table (AL, CU, CUZNS, MIX, HMS, ...) and the customer short-code convention are stored as reference data (Section 9), not embedded in template text, so the same generation logic serves every territory variant.

5. FA Transaction Processing

CORE WORKFLOW · FR-FA-01...06

What it does

Applies the same intake → classify → extract → match → validate → approve pipeline as Purchase/Sales processing (Features 3–4) to AGFZE's second business stream, referred to throughout the source documents only as **FA**, against a separately configured document/field checklist and approval matrix.

Why it exists

BRD §6.3 requires FA to receive “the same controlled framework” as Scrap, while explicitly leaving FA-specific documents, fields, tolerances and approval rules as “business inputs to be finalised with AGFZE during discovery” (BRD §6.4 note) — this feature exists to guarantee that discovery work only ever means *configuration*, never a second parallel codebase.

User workflow

1. FA officer opens a classified FA request from an FA-scoped queue view (same Inbox UI, filtered by business stream).
2. Extraction, matching and validation panels are functionally identical to Feature 3/4's, driven by the FA document/field/tolerance configuration set instead of the Scrap one.
3. FA approver/HOD reviews and approves exactly as in Feature 7, using the FA approval matrix.

Data required

Per BRD §7.3 FA domain: Request ID, counterparty, transaction/contract/batch reference, quantity, rate, amount, currency, document type, approval status, plus additional FA-specific fields to be finalised with AGFZE. Modelled as **FaLeg** with a JSONB **extra_fields** column (Section 9) specifically so undetermined fields do not block the base schema from shipping in Phase 2.

Processing logic

Identical pipeline to Features 3/4, parameterised by **business_stream = “FA”**: the Classification, Extraction, Matching and Validation Agents all read their field schema and tolerance table by stream, so no stream-specific code path exists beyond configuration and the FA-specific document checklist.

Output generated

A governed FaLeg record following the same state machine (Section 13.8) as Scrap transactions.

AI integration details

Same as Feature 2 (Gemini 2.5 Flash, shared extraction engine), with the FA document-type schema substituted.

Background job details

Same background-task/Cloud Run Job split as Feature 3.

Technical notes

This PRD deliberately does not invent FA-specific document types, fields or tolerances that were not in the source material (per the Absolute Rule against speculative content) — Section 18.2's Phase 1 plan schedules an explicit FA discovery/configuration workshop with AGFZE before FA go-live, exactly as BRD §6.4 anticipates.

6. Validation & Business Rule Engine GOVERNANCE CORE · BR-01...13

What it does

A single, shared, configuration-driven rule engine that every transaction (Purchase, Sales, FA) passes through before it is allowed to reach Approval Pending — checking document completeness, quantity/value tolerance, container/reference matching, duplicate detection, and payment-term override reasons.

Why it exists

BR-01 through BR-13 are the codification of exactly what the HOD currently checks by eye (Section 1.2); centralising them in one engine (rather than duplicating checks inside the Purchase/Sales/FA features) is what makes BR-12's audit-logging requirement and Section 1.5's cross-domain reusability possible.

User workflow

1. Runs automatically as part of every transaction's lifecycle — no direct user-initiated workflow — but every result is user-visible as a pass/fail checklist inside the transaction workspace (Features 3–5), each row showing the rule ID, the specific field(s) checked, the threshold applied and the actual value found.
2. Where a rule fails, the user sees a **Resolve** action that routes to the Exception Queue (Feature 7) with the failing rule pre-filled as the reason.
3. Configuration screens (Admin area, Section 8) let an authorised admin edit tolerance values, mandatory-document checklists per transaction type/territory, and duplicate-detection thresholds without a deployment.

Data required

- Per-rule configuration: `rule_id`, `scope (commodity_code/territory/transaction_type)`, `threshold_value`, `active`
- Pydantic `RuleEvaluationOut`: `rule_id: str`, `passed: bool`, `field: str | None`, `expected: str | None`, `actual: str | None`, `message: str`

Processing logic

1. BR-01 (unique Request ID) and BR-02 (a business reference must exist before matching) are enforced at intake (Feature 1) and matching (Feature 2), respectively — fail-fast rather than deferred.
2. BR-03 (container match), BR-04 (mandatory documents present), BR-05 (quantity tolerance, default configurable $\pm 5\%$), BR-06 (invoice value/rate/amount tolerance) run against the matched record once extraction is confirmed.
3. BR-07 (sales-document preparation may start from an OBL or an AGFZE-approved draft B/L, but final posting waits for final shipping-document

validation) is enforced as a status gate, not a UI restriction — the state machine (Section 13.8) has no transition from **Extracted** straight to **Committed** that skips this check.

4. BR-08 routes any failure — missing document, mismatched identifier, out-of-tolerance value, duplicate reference — to **Exception Open**.
5. BR-13 (repeated emails/documents must not create duplicate downstream postings) is enforced via a content hash + reference-number uniqueness check at the Matching Agent, linking a repeat submission to the existing transaction instead of creating a new one.

Output generated

A per-transaction **rule_evaluations** array persisted alongside the transaction (queryable for audit); status transitions to **Validation Pending** → pass/fail.

AI integration details

Deterministic rule evaluation (no AI) for numeric tolerance/duplicate/reference checks; Gemini is used only for the narrow case of free-text mandatory-document detection when a required document's presence must be inferred from an email body rather than a structured field (e.g. “certificate to follow separately”).

Background job details

Runs synchronously as part of the transaction pipeline (typically <500ms per rule set); no separate background job.

Technical notes

Every threshold in this engine (tolerance percentages, mandatory-document lists, duplicate-detection sensitivity) is explicitly called out in BRD §7.1 as “the current draft... subject to AGFZE confirmation” — the schema design in Section 9 stores these as versioned configuration rows precisely so a threshold change is an admin-screen edit, never a code deployment.

7. Exception & Approval Workflow

GOVERNANCE CORE · FR-IN-01, EXCEPTION HANDLING MATRIX

What it does

Routes every failed validation to a category-specific exception queue with an owner, priority and ageing clock (per BRD §7.2's ten exception categories), and routes every passed validation to an approval queue gated on a named approver's decision before any downstream posting is allowed.

Why it exists

This is the literal implementation of the platform's "golden governance rule" (Section 2.3) and directly targets Scenario 3 (HOD-as-single-point-of-approval) by making every pending approval visible, ranked and delegable, rather than living only in the HOD's inbox.

User workflow

1. A blocked transaction appears in the Exception Queue, grouped by exception type (missing document, mismatched container, invoice-value tolerance, quantity tolerance, unmatched reference, low AI confidence, duplicate, shipment-status stale, approval overdue, integration failure) with its configured owner role pre-assigned per the Exception Handling Matrix.
2. Owner opens the exception, sees the specific rule that failed and the exact field/value, takes the matrix-defined action (e.g. "notify sender with missing-document list," "route to matching-review queue," "hold and route to Finance") and records a resolution note.
3. Resolved exceptions re-enter the normal pipeline at the step they left it, not at the beginning.
4. A passed transaction appears in the Approval Queue, ranked by configured priority/value/age; the approver (HOD or delegated approver) opens the transaction, reviews the summary and the linked source documents, and clicks **Approve**, **Reject**, or **Request Changes** — every decision requires a captured timestamp and, for reject/request-changes, a reason.
5. Only an **Approved** transaction becomes eligible for Feature 8's downstream posting.

Data required

- Pydantic `ExceptionCaseOut`: `exception_type: ExceptionTypeEnum`, `owner_role: str`, `opened_at: datetime`, `age_hours: int`, `priority: Literal["low", "medium", "high"]`, `resolution_note: str | None`
- Pydantic `ApprovalTaskOut`: `approver_role: str`, `decision: Literal["pending", "approved", "rejected", "changes_requested"]`, `decided_by: UUID | None`, `decided_at: datetime | None`, `reason: str | None`

Processing logic

1. Approval & Exception Agent evaluates the Exception Handling Matrix row-by-row against the Validation Engine's output and assigns owner role + priority automatically.
2. Ageing is computed continuously (not only on page load) so dashboard counts (Feature 10) are always current.
3. "Approval not received" (an exception category in its own right) escalates automatically per a configured ageing rule — e.g. unresolved after N business hours re-notifies the approver and, after a second threshold, flags the HOD.

Output generated

An ExceptionCase or ApprovalTask row; domain events `ExceptionCreated` / `ExceptionResolved` , `ApprovalRequested` / `ApprovalGranted` / `ApprovalRejected` ; on approval, a domain event that unlocks Feature 8.

AI integration details

No generative AI in the decision path itself — approval is exclusively a human action, by design (Section 2.5); Gemini is used only to draft a plain-language summary of "what changed and why this needs your approval" to speed up the approver's review, never to make or suggest the approve/reject decision.

Background job details

Ageing/escalation recalculation runs as a lightweight scheduled Cloud Scheduler job every 15 minutes; exception/approval creation itself is synchronous with the triggering validation event.

Technical notes

The state machine (Section 13.8) makes `Approved` a required predecessor state to `Integration Pending` with no bypass path in the API layer (enforced server-side, not just hidden in the UI) — this is the concrete mechanism behind BR-10 ("SAP or DMS updates must not be committed until required business approval is captured").

8. Tracker / SAP / DMS Integration Hub

DOWNSTREAM INTEGRATION ·
FR-IN-02...05

What it does

Once a transaction is **Approved**, synchronises the governed record to the Excel/SharePoint trackers where they remain in use, posts the transaction to SAP through the confirmed API or approved integration mechanism, uploads approved documents to the DMS, and routes any integration failure to a dedicated technical exception queue with retry.

Why it exists

This is the feature that finally removes the manual Power Automate script execution and manual DMS upload/rename steps from both workflow diagrams (purchase steps 10–13, sales steps 7 and 11) while preserving every downstream system AGFZE already depends on — the platform is explicitly an automation layer in front of SAP/DMS, not a replacement for either.

User workflow

1. No end-user action is required for a successful integration; the transaction detail view shows a live Integration panel: tracker sync status, SAP reference number (once posted) or failure reason, DMS document ID/link (once uploaded).
2. On failure, the transaction appears in a technical Integration Monitor queue (IT/Support-owned) with the failure reason, a **Retry** action, and — where configured — an approved manual fallback path (e.g. the existing Power Automate script) that the support user can trigger explicitly.
3. Once all configured integrations succeed, the transaction moves to **Committed** and, when its full document/shipment lifecycle is done, to **Closed**.

Data required

- Pydantic `IntegrationJobOut`: `target_system: Literal["tracker", "sap", "dms"]`, `status: Literal["queued", "processing", "succeeded", "failed"]`, `external_reference: str | None`, `failure_reason: str | None`, `attempt_count: int`
- SAP: TC (Trading Contract)/PO and DPR object identifiers per BRD/TDD terminology; DMS: target folder/naming convention (e.g. `ADV-{contract}-{qty}-{status}` for purchase, `S0-{batch}-{qty}-{Final/Prov}` for sales, matching the naming already in use)

Processing logic

1. Tracker Update Agent writes the approved record to the relevant Excel workbook (Loading Sheet, Cost Sheet, Shipment Status) via the Microsoft

Graph Excel/SharePoint API — never by opening/saving the file directly — so concurrent human edits in the same workbook are not clobbered.

2. SAP posting is attempted first through a confirmed SAP OData/BAPI/API endpoint where AGFZE's SAP landscape exposes one; where it does not (to be confirmed during integration design, TDD §11), the job instead prepares the TC/DPR entry payload and hands it to an assisted Power Automate flow, preserving a human-supervised fallback rather than forcing a brittle screen-scrape integration.
3. DMS upload posts the finalised document pack (merged PDF, named per AGFZE's existing convention) via the DMS's REST API, storing the returned document ID/link on the transaction.
4. Any step failure logs a structured failure reason, increments `attempt_count`, and retries with exponential backoff up to a configured maximum before routing to the technical exception queue (FR-IN-05).

Output generated

Updated tracker rows (where still in use); an SAP reference number or a logged failure reason; a DMS document ID/link; domain events

`IntegrationJobQueued` / `Succeeded` / `Failed`, `DmsUploadCompleted`.

AI integration details

None — this feature is deliberately deterministic integration code, not an AI feature, so that a failure is always attributable to a specific API call rather than to model non-determinism.

Background job details

Each target system (tracker, SAP, DMS) is queued as an independent Cloud Run Job keyed to the transaction's Approved event, so a DMS outage does not block a tracker sync or vice versa; job status is exposed per-target on the Integration Monitor screen.

Technical notes

TDD §11 explicitly flags that exact SAP object names/API availability and the DMS's endpoint contract are to be finalised during detailed integration design with AGFZE's IT/SAP/DMS teams — this PRD's schema (Section 9) stores `target_system`, `external_reference` and `failure_reason` generically so the connector implementation can be finalised without a schema migration.

9. Shipment & Cargo Tracking

LOGISTICS · FR-LG-01...05

What it does

Captures and unifies container, B/L, shipping-line, vessel, port and milestone information from documents and portal records, attempts scheduled automatic status updates from shipping-line sources where available, and links shipment status back to the associated purchase/sales/FA transaction record.

Why it exists

Removes the manual carrier-website lookups described in Scenario 4 (Section 1.4) and the operations workflow's "begin tracking via the shipping-status Excel sheet, containers move to the tracking sheet only once an invoice has been raised" rule, replacing point-in-time manual checks with a scheduled pull plus a single unified shipment view.

User workflow

1. Logistics coordinator opens the Shipment Dashboard: every open shipment as a card/row showing container number, B/L number, carrier, POL/POD, ETA/ETD, current milestone, and linked transaction.
2. A shipment with no update beyond a configured staleness threshold (default 48 hours) is flagged and auto-assigned to the coordinator with an ageing badge (FR-LG-04).
3. Coordinator can trigger a manual refresh for a single container, or open the linked purchase/sales transaction directly from the shipment card.
4. On delivery/close-out, the coordinator records post-delivery issues (quality claim, damage, detention) against the shipment, visible to Sales/Finance on the same record.

Data required

- Per BRD §7.3 Shipment/Logistics domain: Request ID, container number, B/L number, shipping line, vessel, port of loading, port of discharge, ETA, ETD, milestone, shipment status
- Pydantic `ShipmentOut`: `container_no: str`, `bl_no: str`, `carrier: str`, `pol: str`, `pod: str`, `eta: date | None`, `etd: date | None`, `milestone: str`, `status: Literal["on_schedule", "delayed", "arrived", "exception"]`, `last_checked_at: datetime`

Processing logic

1. Shipment Tracking Agent extracts container/B-L/vessel data from purchase and sales documents (Feature 2) as soon as an invoice raises the container into scope, matching the operations rule that a container only enters tracking once its invoice exists.
2. A scheduled job attempts a status pull per container against configured, AGFZE-approved shipping-line APIs or endpoints where technically

available; where no API access exists, the container is left for manual status entry with the same data model, so the UI is identical either way.

3. Any status change is compared to the previous value and, if the delta looks materially wrong (e.g. ETA moved backwards by an implausible margin), is flagged for review rather than silently accepted.
4. Shipment status is linked to the owning TradeTransaction so a stalled shipment surfaces automatically wherever that transaction is viewed (Purchase/Sales workspace, Dashboard).

Output generated

A Shipment record per container; a domain event `ShipmentStatusUpdated`; a synced row in the Shipment Status tracker where it remains in use (BR-09).

AI integration details

Gemini is used only to parse unstructured carrier-status text (e.g. a tracking page's free-text milestone description) into the fixed `milestone / status` enum; the scheduling and API-calling logic itself is deterministic.

Background job details

Cloud Scheduler triggers a Cloud Run Job on a configurable interval (default every 6 hours) per active shipment; failures (carrier site unavailable, API error) are logged per-container without blocking the batch, and a container that fails repeatedly is escalated to FR-LG-04's exception path.

Technical notes

BRD §6.5 (FR-LG-02) frames carrier access as “attempt... using available APIs or other approved mechanisms,” not a guarantee — the design must degrade gracefully to manual status entry per carrier without changing the data model or the dashboard.

10. Dashboard & Workbench

VISIBILITY · FR-DB-01...07

What it does

The single web operational surface (the “AI Command Centre”) giving role-based, real-time visibility into pending/processed/failed/exception transactions, exception ageing and ownership, document/validation/approval/shipment status, automation KPIs, and one-click access back to the source email, document, transaction and relevant tracker/system reference.

Why it exists

FR-DB-07 designates this as “the primary operational interface for Scrap and FA users” — it is the feature that actually replaces the Excel-tracker-as-status-board habit described in Section 1.7, by being the one place status can always be found instead of “whichever tab was last updated.”

User workflow

1. User signs in and lands on a role-scoped dashboard (Purchase/Sales/Logistics/Finance/HOD view differ only in which widgets and queues are shown, per Section 8’s page-by-page breakdown).
2. Widgets show: open transaction counts by status, exception counts by category/ageing, approval-queue depth, automation KPIs (extraction accuracy, average processing turnaround, automation percentage, integration failure count — FR-DB-04), and a shipment-status summary.
3. Every widget is clickable through to its underlying queue (e.g. clicking “12 exceptions > 24h old” opens the Exception Queue pre-filtered).
4. From any transaction, the user can open the originating email, the source document, the portal transaction record, and — subject to role-based access — the linked tracker row or SAP/DMS reference, in one click (FR-DB-06).

Data required

Aggregates read from the same TradeTransaction/ExceptionCase/ApprovalTask/Shipment/IntegrationJob tables used elsewhere (Section 9); no separate data entry on this feature.

Processing logic

1. Dashboard aggregate endpoints (Section 12) compute counts/percentages server-side (never client-side over a full dataset) using indexed status/ageing columns, with a short server-side cache (30–60s) to keep the dashboard responsive under concurrent use without hammering the database on every page view.
2. Role-based filtering is applied at the query layer (Section 10.5’s `get_current_user` dependency), not only hidden in the UI, so a Sales user’s dashboard queries physically cannot return Purchase-only exception detail beyond what their role permits.

Output generated

Recharts-rendered widgets (bar/line/donut per KPI), a filterable transaction/exception/approval table, and deep links into every other feature's detail view.

AI integration details

None directly — this is a read/aggregate surface over data other features have already produced.

Background job details

None beyond the standard aggregate-cache refresh described above.

Technical notes

Every number shown here must be traceable to source records (Section 2.5) — dashboard aggregate queries are built exclusively from governed portal tables, never from a denormalised reporting snapshot that could drift from the transaction of record.

11. Automated Reporting Engine

REPORTING · FR-RP-01...05

What it does

Generates a daily operational report for the HOD and designated stakeholders, a monthly management summary (transaction status, approvals, exceptions, ageing, shipment visibility, agreed KPIs) for Scrap and FA, and supports authorised on-demand/ad-hoc reports filtered by date range, business stream, transaction status and other approved filters — every figure traceable back to its source transaction and audit trail.

Why it exists

Replaces the operations workflow's manual "prepare reports — built from container numbers as required by HOD... daily standard reports plus, on HOD request, payment, sales & tracking reports" step with a scheduled, governed, self-service equivalent.

User workflow

1. Daily/monthly reports are generated automatically on schedule and appear in the Reports screen, with configured distribution (FR-RP-05) to approved recipients — exact templates and distribution rules to be confirmed with AGFZE, per BRD §6.8.
2. An authorised user opens the Ad-hoc Report Builder, selects date range, business stream (Scrap/FA), transaction status and any other configured filter, and generates a report on demand.
3. Every figure in a generated report is a clickable drill-through back to the underlying transaction list and, from there, to the individual transaction's audit trail (FR-RP-04).
4. Reports are downloadable as PDF/Excel and, where configured, distributed automatically to approved recipients/channels.

Data required

```
Pydantic ReportRequestIn: report_type:
Literal["daily","monthly","adhoc"], date_from: date, date_to: date,
business_stream: Literal["scrap","fa","both"], status_filter:
list[str] | None, format: Literal["pdf","xlsx"] .
```

Processing logic

1. Scheduled reports run as Cloud Scheduler-triggered Cloud Run Jobs (daily at a configured time; monthly on the 1st) that query the governed portal tables directly (never a copy), assemble the report data, and render via pandas (aggregation) + python-docx/PyMuPDF (PDF) or openpyxl (Excel export).
2. Ad-hoc report requests run as an async job with the same rendering path, returned via the standard job-status polling convention (Section 12.7).

3. Every rendered report embeds a report-generation reference ID (BRD §7.3 Control/Workflow domain) that resolves back to the exact query parameters and the audit log entry for that generation.

Output generated

A stored PDF/XLSX report file (versioned, not overwritten), a report-generation reference, and — where distribution is configured — an outbound notification/email with the report attached or linked.

AI integration details

Gemini may generate a short natural-language executive summary paragraph to accompany the monthly management report (e.g. highlighting the largest exception category or the biggest ageing change), clearly labelled as AI-generated with the standard disclaimer (Section 2.5); every figure in the report body itself is computed deterministically, never by the model.

Background job details

See Processing logic — Cloud Scheduler + Cloud Run Jobs for scheduled reports, async FastAPI background task + job polling for ad-hoc reports.

Technical notes

BRD §6.8 explicitly leaves exact report templates and distribution rules to be confirmed with AGFZE — the report engine is therefore built against a template-configuration table (Section 9), not hard-coded report layouts, so finalising a template is a configuration change.

Scope added after initial drafting

This feature was added following an explicit follow-up instruction and was not part of BRD/TDD discovery. It is labelled **SPECIFIED** throughout this PRD to distinguish it from the BRD/TDD-sourced Features 1–11. It is designed to reinforce, not loosen, the governance model established in Sections 2.3 and 13.8 — see the explicit offline-scope boundary below.

12. Progressive Web App, Offline Resilience & Notifications

PLATFORM CAPABILITY ·
ADDED SCOPE

What it does

Makes the Command Centre installable as a standalone app on Android, iOS/iPadOS (via Safari's home-screen install), Windows, and macOS/desktop browsers; keeps read-only visibility of recently viewed data available when the network drops, with a clear offline page for anything not cached; sends transactional email (approval requests, exception assignment, report-ready, missing-document follow-ups) as professionally designed HTML email; and delivers real-time browser push notifications for time-sensitive events (an approval awaiting the HOD, a shipment exception) even when the Command Centre tab is closed.

Why it exists

Directly extends Section 5 Stage 9 (the HOD approving from a phone) and the operations reality described in Section 1 — trade desk staff are not always at a desk with a stable connection, and the current process's biggest single failure mode (Scenario 3: the HOD unreachable at the exact moment several deals need signing) is exactly what push notifications and installability are meant to reduce.

User workflow

1. An **Install** button appears in the global header (Section 8) whenever the browser fires a `beforeinstallprompt` event (Chrome/Edge/Android) or, on iOS Safari where that event doesn't exist, the button opens a short instructional dialog ("Tap Share, then Add to Home Screen") since iOS requires that manual step.
2. On first sign-in after install, or on first visit generally, the user is asked, at a contextually relevant moment (after their first approval or exception assignment — never on page load) to grant notification permission, with a plain-language explanation of what they'll be notified about.
3. If the network drops while browsing a previously visited screen (Dashboard, Transactions list, Shipment Dashboard), the page continues to render its last-cached data with a persistent "You're viewing cached data from {time} — reconnect to update" banner rather than an error.
4. If the network drops on a screen/action that was never cached, or the user attempts a mutating action (submit, approve, generate draft) while offline, they are routed to a dedicated **Offline** page explaining that this specific action requires a connection, with an automatic retry once connectivity returns.
5. A relevant event (approval requested, exception assigned, report ready) triggers both an HTML email and, for a user who has granted permission, a

browser push notification; clicking either deep-links straight into the relevant screen.

Data required

- `PushSubscriptionIn{endpoint:str, keys:{p256dh:str, auth:str}}` (from the browser's `PushManager`)
- Email/notification preference (already modelled on `User.notification_channel`, Section 9.1)
- Cacheable read endpoints whitelist (dashboard summary, transaction/shipment/exception/approval lists, KPI data) — deliberately excludes anything that must always reflect the current server state before a decision is made

Processing logic

1. A Workbox-generated service worker precaches the Next.js static build output and app shell at install time, and applies *stale-while-revalidate* to the whitelisted GET read endpoints (serve cached instantly, refresh in the background) and *network-only* to every mutating (POST/PATCH/PUT/DELETE) request and every AI/extraction/approval endpoint — mutating calls are never queued for background sync, by design (see the governance note below).
2. On `fetch` failure for a non-whitelisted or non-cached navigation, the service worker's navigation fallback serves the precached `/offline` route instead of the browser's default offline error page.
3. Push: the frontend requests notification permission → subscribes via `registration.pushManager.subscribe({userVisibleOnly:true, applicationServerKey: VAPID_PUBLIC_KEY})` → posts the resulting subscription to the backend. When a domain event that maps to a notification rule fires (Section 13.8's event list), `notification_service.py` renders the message and calls `push_service.py` (using `pywebpush` with the VAPID private key) to deliver it to every active subscription for the relevant user(s); a `410 Gone` response from a push endpoint removes the stale subscription.
4. Email: the same domain event triggers `email_service.py`, which renders an HTML template (Section 13.12) and sends it over SMTP.

Output generated

An installed, launchable app icon on the user's device; a cached, read-only offline experience for recently viewed screens; delivered HTML emails; delivered browser push notifications; a `notifications` row (Section 9.1) for every delivered notification regardless of channel, so the in-app Notification Centre (Section 6/7) remains the single durable record even if a push notification was missed or an email was filtered.

AI integration details

None — this is deterministic platform infrastructure.

Background job details

Push and email delivery are fire-and-forget async tasks triggered directly off the domain-event bus (Section 13.8), not polled jobs — a failure to deliver a push notification or email is logged and retried up to three times with backoff, but never blocks or rolls back the underlying transaction event itself.

Technical notes — the offline/governance boundary***Deliberate design constraint***

Offline support in this platform is read-only by design. No approval decision, submission, draft generation, or field correction is ever queued client-side for later sync — Section 2.3's golden governance rule ("read and reconcile everything; commit nothing without a human, in the moment, against current data") would be meaningfully weakened by an offline-queued approval that might be decided against stale data and silently replayed later. The Offline page is explicit about this rather than pretending the action succeeded.

05 SECTION 5 User Journey

Adapting this section to an internal enterprise tool

The AI Command Centre is a single-tenant internal platform for AGFZE staff, not a public product with an acquisition funnel — so “Discovery and Landing” below is adapted to mean the rollout communication and first sign-in experience described in BRD §9 (Implementation Plan), not marketing.

Stage 1 — Discovery and Landing

A user first encounters the platform through an internal rollout communication tied to their team’s go-live wave (Purchase/Sales first, then Logistics/Finance, then FA — Section 18.2), sent by the HOD/programme sponsor, containing the portal URL and their go-live date. On first visit, the sign-in screen shows the AGFZE Command Centre logo, a one-line description of what changed (“your Purchase/Sales/Logistics work now starts here instead of Outlook + Excel”), and a single **Sign in with Microsoft** button — no public marketing content, no unauthenticated content beyond the sign-in screen itself and the legal/disclaimer pages (Section 6).

Stage 2 — Understanding the Platform

Because every user already knows the underlying business process (Section 1.3), the platform teaches only the *mapping* from old habit to new screen — a short, dismissible first-login walkthrough (3–4 steps, Framer-Motion-animated tooltip callouts) pointing at: “this is where your inbox used to be” (Inbox/Request Queue), “this is where you used to retype the Loading Sheet” (Transaction Workspace), “this is where the HOD used to sign by hand” (Approval Queue, if the user has approval rights). Time-to-value is designed to be under five minutes because the mental model transfers almost entirely from the existing process.

Stage 3 — Authentication via Keycloak (brokering Microsoft Entra ID)

User clicks **Sign in with Microsoft** → NextAuth.js initiates an OIDC Authorization Code Flow (with PKCE) against the Keycloak realm → Keycloak, configured with Microsoft Entra ID as an upstream identity provider (identity brokering, not a second credential set), redirects the browser to AGFZE’s actual Microsoft 365 login → user authenticates with their existing corporate credentials (and MFA, if AGFZE’s Entra ID policy requires it) → Microsoft redirects back to Keycloak with an authorization code → Keycloak exchanges it, applies AGFZE’s Keycloak-side role mapping (Purchase/Sales/Logistics/Finance/FA/Approver/Admin), and issues its own authorization code to the Next.js app → NextAuth.js exchanges that code for a signed JWT → the JWT is stored in a secure, HTTP-only, SameSite cookie → the user lands on their role-scoped dashboard. Every subsequent API call carries the JWT as a Bearer token, verified on the backend against Keycloak’s JWKS endpoint via python-jose. Error

states: invalid/expired Microsoft credentials surface Microsoft's own error inline before redirecting back; an account with no AGFZE role mapping in Keycloak sees a clear “your account isn’t yet provisioned for the Command Centre — contact IT” screen rather than a raw 403; an expired session triggers a silent refresh via the stored refresh token, falling back to a full re-login prompt only if the refresh token itself has expired.

Stage 4 — Onboarding

First sign-in triggers a one-time profile confirmation (display name and email pulled from Entra ID, read-only; preferred notification channel, editable) and, for Purchase/Sales/FA users, a one-screen preference for default queue filters (e.g. “show me Scrap only” vs “Scrap + FA”). No organisation-setup step is needed — AGFZE is the only tenant in this deployment (Section 15 discusses multi-tenant as a future option, not current scope) — so onboarding is deliberately short: profile confirmation, the Stage 2 walkthrough, then straight to the dashboard.

Stage 5 — First Use of a Core Feature

For a Purchase officer, first real use is opening a newly classified purchase request from the Inbox queue (Feature 1): click the row → Transaction Workspace opens → extracted fields render with confidence badges (Feature 2) while a “Matching...” skeleton shows briefly → Matching panel populates (“Matched to Loading Sheet batch #77, 94% confidence” or “No match — new batch proposed”) → Validation panel populates (BR-04/05/06 checklist) → if everything passes, a single **Submit for Approval** button is enabled; if something fails, the same panel shows a red row with a **Resolve** link into the Exception Queue instead. Every AI-derived value is visibly labelled and links back to the source document page it was read from.

Stage 6 — Receiving and Understanding Results

Once HOD approval completes (Feature 7) and downstream integration succeeds (Feature 8), the transaction detail view shows a clear timeline: Received → Classified → Extracted → Matched → Validated → Approved (by whom, when) → Committed (tracker/SAP/DMS references, or a failure reason with retry status). The standard AI disclaimer banner (Section 2.5) is present on every screen showing AI-extracted or AI-drafted content, and every number on the screen is one click from its source email/document.

Stage 7 — Exporting or Downloading

From a transaction, a generated document (draft sales contract/invoice, DOCX or PDF) or a report (Feature 11) is downloaded via a signed, time-limited, authenticated API endpoint (never a direct storage URL) — user clicks **Download** → a short loading spinner (server assembles/streams the file) → the browser receives the file with a content-disposition filename following AGFZE’s existing naming convention (e.g. `S0-Batch-Qty-Final.pdf`) → a toast confirms success; a failed generation shows an inline error with a **Retry** action rather than a broken download.

Stage 8 — Returning User Experience

A returning user lands directly on their role-scoped dashboard (Feature 10), not a generic home page — open exceptions, pending approvals and in-flight transactions are exactly where they left them, because state lives server-side in the governed transaction record, not in browser session state. Starting new work means opening the Inbox queue (new items appear automatically as email arrives) or, for FA/Sales, using the manual “register a transaction” entry point for cases with no inbound email trigger.

Stage 9 — Mobile and Responsive Experience

The primary workbench (Transaction Workspace, Exception Queue, Approval Queue) is used predominantly on desktop given the density of extraction/validation data, but the Dashboard, Approval Queue and notifications are fully responsive: the sidebar collapses into a Radix UI Sheet-based hamburger drawer below the `md` breakpoint, data tables convert to stacked card layouts, and the Approval Queue’s approve/reject actions remain single-thumb reachable — this specifically supports the HOD approving a time-sensitive transaction from a phone rather than waiting to be at a desk, which is one of the concrete improvements this platform is meant to deliver over the current email-and-signature process.

SECTION 6

06 Website Structure — Complete Sitemap

34 routes across 10 categories. “Auth” = requires a signed-in session (NextAuth.js middleware); every authenticated route additionally enforces role-based access server-side (Section 16.2), so the table below marks the *minimum* role where narrower than “any authenticated user.”

Public Pages

Route	Auth	AI	Purpose
/	No	No	Sign-in landing; redirects authenticated users straight to / dashboard
/ disclaimer	No	No	Standing AI-disclaimer and platform-scope statement (Section 2.5)
/privacy	No	No	Data-handling and retention statement
/terms	No	No	Internal platform terms of use

Authentication Pages

Route	Auth	AI	Purpose
/signin	No	No	Triggers NextAuth.js OIDC flow to Keycloak (brokering Entra ID)
/signout	Yes	No	Clears session, redirects to Keycloak/Entra ID logout, then to /

Dashboard & Navigation

Route	Auth	AI	Purpose
/ dashboard	Yes	No	Role-scoped home — KPI widgets, queue depths, shortcuts (Feature 10)

Core Tool Pages

Route	Auth	AI	Job	Purpose
/inbox	Yes	Yes	Yes	Request queue — classified inbound emails (Feature 1)
/inbox/upload	Yes (Purchase/Sales/FA)	Yes	Yes	Manual document/portal intake route (FR-PU-01 portal path)
/transactions	Yes	No	No	Cross-stream transaction list, filterable, paginated
/transactions/new	Yes	No	No	Manual transaction registration (no email trigger case)
/transactions/purchase/[id]	Yes (Purchase)	Yes	Yes	Purchase Transaction Workspace (Feature 3)

/transactions/ sales/[id]	Yes (Sales)	Yes	Yes	Sales Transaction Workspace + draft generation (Feature 4)
/transactions/fa/ [id]	Yes (FA)	Yes	Yes	FA Transaction Workspace (Feature 5)

Document Management Pages

Route	Auth	AI	Purpose
/documents	Yes	No	Searchable document list across all transactions
/documents/[id]	Yes	Yes	Document Review — extraction split-view (Feature 2)

Results & Report Pages

Route	Auth	AI	Job	Purpose
/exceptions	Yes	No	No	Exception Queue by category/owner/age (Feature 7)
/exceptions/ [id]	Yes	No	No	Exception detail + resolution action
/approvals	Yes (Approver)	Yes	No	Approval Queue, ranked (Feature 7)
/approvals/[id]	Yes (Approver)	Yes	No	Approval decision screen with AI summary
/reports	Yes	Yes	Yes	Daily/monthly report list + download (Feature 11)
/reports/ builder	Yes	No	Yes	Ad-hoc report builder (Feature 11)
/reports/[id]	Yes	No	No	Generated report viewer with drill-through

Analytics / Visualisation Pages

Route	Auth	AI	Job	Purpose
/shipments	Yes	No	Yes	Shipment Dashboard, unified container/B-L view (Feature 9)
/shipments/ [id]	Yes	No	No	Single shipment detail + post-delivery issue log
/analytics	Yes	No	No	Automation KPI charts — Recharts (part of Feature 10)

Administration Pages

Route	Auth	AI	Purpose
/admin	Yes (Admin)	No	Admin landing — system health, config summary
/admin/users	Yes (Admin)	No	

User/role management (Keycloak Admin API-backed)

/admin/rules	Yes (Admin)	No	Tolerance / mandatory-document / duplicate-threshold config (Feature 6)
/admin/document-types	Yes (Admin)	No	Document-type field-schema & territory-variant config (Feature 2)
/admin/integrations	Yes (Admin/IT)	No	Integration Monitor — tracker/SAP/DMS job detail & retry (Feature 8)
/admin/audit	Yes (Admin/Auditor)	No	Audit Explorer — full event log, exportable

Profile & Settings Pages

Route	Auth	AI	Purpose
/settings	Yes	No	Profile (read-only from Entra ID), notification preferences
/notifications	Yes	No	Notification centre (approvals due, exceptions assigned, reports ready)

Error & System Pages

Route	Auth	Purpose
not-found.tsx	No	404 — unmatched route
error.tsx	No	Route-segment error boundary with retry
/offline	No	Service-worker navigation fallback — shown when a request fails offline and no cached version exists (Feature 12)

Not a route, but part of the site's global chrome per Feature 12: an **Install** button in the header (Section 8), rendered whenever the browser supports PWA installation.

07 Frontend Pages — Complete Catalog

The four public/legal pages, the two auth-trigger pages and the two error pages are static/near-static and are catalogued compactly at the end of this Part; the 24 substantive application pages below each receive the full template.

Dashboard	/APP/DASHBOARD/PAGE.TSX → /DASHBOARD
Auth required	Yes — any authenticated role
Purpose	Role-scoped operational home: open-transaction counts, exception/approval queue depth, automation KPIs, shipment summary (Feature 10)
UI components	shadcn Card KPI tiles, Recharts donut (status mix) + bar (exceptions by category) + line (turnaround trend), shadcn Tabs for stream (Scrap/FA), Skeleton loaders per widget
API endpoints consumed	GET /api/v1/dashboards/summary , GET /api/v1/dashboards/kpis
Inputs required	Optional stream filter (query param), date-range picker (defaults to last 30 days)
Outputs displayed	Counts, percentages, charts; every tile links through to its filtered queue
Buttons/actions	Stream toggle, date-range picker, “View all” per widget (navigates to /transactions, /exceptions, /approvals, /shipments filtered)
Navigation links	Sidebar to every other authenticated route the user’s role permits
Background job	No (reads pre-aggregated, cached summary)
AI integration	No — pure aggregate display

Inbox / Request Queue

/APP/(PROTECTED)/INBOX/PAGE.TSX → /INBOX

Auth required	Yes
Purpose	Classified inbound-email queue; entry point into Feature 1
UI components	shadcn Table with sort/filter/pagination, category Badge with confidence tint, Select for stream/category filter, empty-state illustration when queue is clear
API endpoints consumed	GET /api/v1/requests , PATCH /api/v1/requests/{id}/category
Inputs required	Filter: category (Zod enum), date range, confidence-below-threshold toggle
Outputs displayed	Sender, subject, received time, AI category + confidence, attachment count, current status
Buttons/actions	Row click → opens linked transaction workspace or Document Review; “Correct category” inline edit with required reason
Navigation links	To /transactions/{stream}/{id} or /documents/{id} depending on row type
Background job	No (reads results of Feature 1’s polling job)
AI integration	Yes — displays classification category + confidence from Feature 1

Manual Intake / Upload

/APP/(PROTECTED)/INBOX/UPLOAD/PAGE.TSX → /INBOX/UPLOAD

Auth required	Yes — Purchase/Sales/FA roles
Purpose	Portal-based document intake for the FR-PU-01 “portal intake route” (deals without an email trigger, or bulk backfill)
UI components	Drag-and-drop upload zone (FormData API), file-type/size validation banner, per-file progress bar, shadcn Select for stream/document-type hint
API endpoints consumed	POST /api/v1/documents/upload
Inputs required	File(s): PDF/DOCX/XLSX/JPEG/PNG, max 25MB each; stream (required); document-type hint (optional, AI overrides if confident)
Outputs displayed	Per-file upload/queued status; link to the created Request once classification completes
Buttons/actions	“Upload & classify” (primary), “Remove” per queued file before submit
Navigation links	To /inbox on completion
Background job	Yes — triggers the same classification/extraction pipeline as email intake
AI integration	Yes — document classification + extraction (Feature 2)

Transactions List

/APP/(PROTECTED)/TRANSACTIONS/PAGE.TSX → /TRANSACTIONS

Auth required	Yes
Purpose	Cross-stream, filterable, paginated list of every TradeTransaction the user's role can see
UI components	shadcn <code>Table</code> , <code>DropDownMenu</code> column filters, status <code>Badge</code> (colour-coded to the state machine, Section 13.8), pagination controls
API endpoints consumed	<code>GET /api/v1/transactions</code>
Inputs required	<code>page</code> , <code>page_size</code> , <code>sort_by</code> , <code>search</code> , stream/status/date filters (Section 12.5 pagination convention)
Outputs displayed	Batch/contract number, counterparty, stream, status, value, age, linked shipment status
Buttons/actions	Row click → stream-specific workspace; "New transaction" → <code>/transactions/new</code>
Navigation links	To workspace detail pages, to <code>/transactions/new</code>
Background job	No
AI integration	No

**New Transaction
(Manual)**

/APP/(PROTECTED)/TRANSACTIONS/NEW/PAGE.TSX → /TRANSACTIONS/NEW

Auth required	Yes
Purpose	Registers a transaction with no inbound email trigger (e.g. an FA deal originated by phone)
UI components	Multi-step shadcn form (stream → counterparty/contract basics → document upload), Progress stepper
API endpoints consumed	POST /api/v1/transactions
Inputs required	Stream (required), counterparty name (required), contract/batch reference (optional — system proposes next sequential batch if blank), initial documents (optional at this step)
Outputs displayed	Created Request ID and a redirect into the relevant workspace
Buttons/actions	“Create draft transaction” (primary), “Cancel”
Navigation links	To the created transaction's workspace
Background job	No (job starts once documents are attached)
AI integration	No at creation; Yes once documents are attached

Purchase
Transaction
Workspace

/APP/(PROTECTED)/TRANSACTIONS/PURCHASE/[ID]/
PAGE.TSX → /TRANSACTIONS/PURCHASE/{ID}

Auth required	Yes — Purchase role (read for others per RBAC, Section 16.2)
Purpose	Primary workspace for Feature 3 — review extraction, matching, validation, submit for approval
UI components	Split layout: document viewer (left, PyMuPDF-rendered page images) + field panel (right); shadcn <code>Accordion</code> for Extraction / Matching / Validation / History sections; confidence-tinted field <code>Input</code> s; <code>Alert</code> for blocked validations
API endpoints consumed	<code>GET /api/v1/transactions/{id}</code> , <code>PATCH /api/v1/transactions/{id}/fields</code> , <code>POST /api/v1/transactions/{id}/submit</code>
Inputs required	Corrected field values (typed per Pydantic/Zod schema, e.g. <code>quantity_mt: positive decimal</code>), correction reason (required when overriding an AI value $\geq$ configured confidence)
Outputs displayed	Matched batch/contract, validation checklist with pass/fail per BR-04/05/06, linked documents, full status history
Buttons/actions	“Save changes,” “Submit for Approval” (disabled until validation passes), “Send to Exception Queue”
Navigation links	To <code>/documents/{id}</code> for each linked document, to <code>/exceptions/{id}</code> if blocked
Background job	Yes — re-validation after a field correction
AI integration	Yes — extraction display, matching suggestion

Sales Transaction
Workspace

/APP/(PROTECTED)/TRANSACTIONS/SALES/{ID}/
PAGE.TSX → /TRANSACTIONS/SALES/{ID}

Auth required	Yes — Sales role
Purpose	Feature 4 workspace: cross-contract consistency, invoice-vs-quantity check, price-fixation handling, draft generation
UI components	Same split layout as Purchase workspace, plus a “Linked Purchase Leg” comparison card highlighting any mismatched field in red (per the contract-process diagram’s mismatch rule), a container-count progress bar (invoiced vs. contracted), a “Generate Draft” panel with template preview
API endpoints consumed	GET /api/v1/transactions/{id} , PATCH /api/v1/transactions/{id}/fields , POST /api/v1/transactions/{id}/generate-draft , POST /api/v1/transactions/{id}/submit
Inputs required	Price-fixation date/rate (when applicable), corrected fields, draft-generation template choice (territory-driven default)
Outputs displayed	Cross-contract diff, invoice-count-vs-contract-quantity meter, generated draft preview (inline PDF viewer)
Buttons/actions	“Record price fixation,” “Generate Draft,” “Request Changes” (re-generates with edits), “Submit for Approval”
Navigation links	To linked Purchase workspace, to /documents/{id}
Background job	Yes — draft generation (Feature 4)
AI integration	Yes — extraction, mismatch detection, draft content generation

FA Transaction Workspace

/APP/(PROTECTED)/TRANSACTIONS/FA/{ID}/PAGE.TSX
→ /TRANSACTIONS/FA/{ID}

Auth required	Yes — FA role
Purpose	Feature 5 workspace — identical shape to Purchase/Sales, parameterised by the FA field/tolerance configuration
UI components	Same as Purchase Workspace, plus a generic “Additional FA Fields” panel driven by the <code>extra_fields</code> JSONB schema (renders whatever fields discovery has configured, without a frontend code change)
API endpoints consumed	<code>GET /api/v1/transactions/{id}</code> , <code>PATCH /api/v1/transactions/{id}/fields</code> , <code>POST /api/v1/transactions/{id}/submit</code>
Inputs required	Same base fields as Purchase, plus dynamically rendered FA-specific fields
Outputs displayed	Same as Purchase Workspace
Buttons/actions	Same as Purchase Workspace
Navigation links	Same pattern as Purchase Workspace
Background job	Yes
AI integration	Yes

Documents List

/APP/(PROTECTED)/DOCUMENTS/PAGE.TSX → /DOCUMENTS

Auth required	Yes
Purpose	Searchable index of every document across every transaction, for lookup independent of the owning transaction
UI components	shadcn <code>Table</code> + search <code>Input</code> , document-type <code>Badge</code> , thumbnail preview on hover
API endpoints consumed	<code>GET /api/v1/documents</code>
Inputs required	Search text (filename/reference), document-type filter, date range
Outputs displayed	Filename, type, linked transaction, upload/received date, extraction status
Buttons/actions	Row click → <code>/documents/{id}</code>
Navigation links	To <code>/documents/{id}</code> , to the linked transaction
Background job	No
AI integration	No (shows results only)

Document Review	/APP/(PROTECTED)/DOCUMENTS/[ID]/PAGE.TSX → /DOCUMENTS/{ID}
Auth required	Yes
Purpose	Feature 2's core screen — side-by-side source rendering and structured extraction with confidence and correction
UI components	PDF/image page viewer with zoom, field list grouped by extraction schema section, confidence-coloured field chips (green/amber/red), Dialog for full-size page view, override-reason Textarea
API endpoints consumed	GET /api/v1/documents/{id} , PATCH /api/v1/documents/{id}/fields , POST /api/v1/documents/{id}/confirm
Inputs required	Corrected field values (typed per schema), override reason (required below-threshold fields)
Outputs displayed	Every extracted field, its confidence, its source page/paragraph reference, override history
Buttons/actions	“Confirm Extraction” (unlocks matching), “Reclassify document type”
Navigation links	To the owning transaction
Background job	Yes — re-extraction after a document-type correction
AI integration	Yes — this is the primary extraction-review surface

Exception Queue	<code>/APP/(PROTECTED)/EXCEPTIONS/PAGE.TSX → /EXCEPTIONS</code>
Auth required	Yes
Purpose	Feature 7's exception list — grouped by category/owner/age per the Exception Handling Matrix
UI components	shadcn Tabs per exception category, ageing Badge (colour ramps green → amber → red past configured thresholds), owner-role filter
API endpoints consumed	<code>GET /api/v1/exceptions</code>
Inputs required	Category, owner role, age-range filters
Outputs displayed	Exception type, linked transaction, age, priority, current owner
Buttons/actions	Row click → <code>/exceptions/{id}</code> ; bulk “Reassign owner” action for admins
Navigation links	To exception detail, to the linked transaction
Background job	No (ageing recalculated by a scheduled job, Feature 7)
AI integration	No

Exception Detail	<code>/APP/(PROTECTED)/EXCEPTIONS/[ID]/PAGE.TSX → /EXCEPTIONS/{ID}</code>
Auth required	Yes
Purpose	Shows the specific failing rule/field/value and the matrix-defined action; captures resolution
UI components	Rule-failure card (rule ID, field, expected vs actual), resolution Textarea , action buttons matching the matrix's defined action for that category
API endpoints consumed	<code>GET /api/v1/exceptions/{id}</code> , <code>POST /api/v1/exceptions/{id}/resolve</code>
Inputs required	Resolution note (required), corrected field where the resolution requires one
Outputs displayed	Full rule-evaluation context, source document links
Buttons/actions	“Resolve & re-run validation,” “Escalate to HOD”
Navigation links	To the linked transaction/document
Background job	Yes — re-validation on resolve
AI integration	No

Approval Queue /APP/(PROTECTED)/APPROVALS/PAGE.TSX → /APPROVALS	
Auth required	Yes — Approver/HOD role
Purpose	Feature 7's ranked approval list — the direct replacement for “the HOD signs everything individually”
UI components	shadcn Table ranked by configurable priority (value/age/risk), value/age Badge s, bulk-select for same-type low-risk approvals
API endpoints consumed	GET /api/v1/approvals
Inputs required	Rank-by selector (age/value/risk), stream filter
Outputs displayed	Transaction summary, value, age, AI-generated plain-language summary of what changed
Buttons/actions	Row click → /approvals/{id}; bulk “Approve selected” (only for a configured, genuinely low-risk subset)
Navigation links	To approval detail, to the linked transaction
Background job	No
AI integration	Yes — plain-language approval summary (never the decision itself)

Approval Decision

/APP/{PROTECTED}/APPROVALS/{ID}/PAGE.TSX → /APPROVALS/{ID}

Auth required	Yes — Approver/HOD role
Purpose	Single-transaction approval screen — the platform’s most safety-critical page
UI components	Full transaction summary, validation-pass confirmation, source-document links, mandatory reason Textarea for Reject/Request Changes, prominent AI disclaimer banner
API endpoints consumed	GET /api/v1/approvals/{id} , POST /api/v1/approvals/{id}/decide
Inputs required	Decision (approve/reject/changes_requested — Zod enum), reason (required unless approve)
Outputs displayed	Everything needed to decide without leaving the page: values, validation results, linked documents, exception history if any
Buttons/actions	“Approve” (primary, confirmation dialog above a configured value threshold), “Reject,” “Request Changes”
Navigation links	To every linked document and the transaction workspace
Background job	Yes — unlocks Feature 8 integration on approval
AI integration	Yes — summary only

Integration Monitor

/APP/(PROTECTED)/ADMIN/INTEGRATIONS/PAGE.TSX → /ADMIN/INTEGRATIONS

Auth required	Yes — IT/Admin role
Purpose	Feature 8's technical monitor — tracker/SAP/DMS job status, retries, manual fallback trigger
UI components	Per-target-system Tabs (Tracker/SAP/DMS), failure-reason Alert , retry-count Badge , “Trigger manual fallback” confirmation Dialog
API endpoints consumed	GET /api/v1/integrations/jobs , POST /api/v1/integrations/jobs/{id}/retry
Inputs required	Target-system filter, status filter
Outputs displayed	Job status, attempt count, failure reason, external reference once succeeded
Buttons/actions	“Retry,” “Trigger approved manual fallback” (logged, reason required)
Navigation links	To the linked transaction
Background job	Yes — retry re-queues the integration job
AI integration	No

Shipment Dashboard

/APP/(PROTECTED)/SHIPMENTS/PAGE.TSX → /SHIPMENTS

Auth required	Yes
Purpose	Feature 9's unified container/vessel/B-L view
UI components	Card grid or table toggle, status Badge (on schedule/delayed/arrived/exception), staleness indicator, manual-refresh icon button per row
API endpoints consumed	GET /api/v1/shipments , POST /api/v1/shipments/{id}/refresh
Inputs required	Status/carrier/POD filters
Outputs displayed	Container, B/L, carrier, POL/POD, ETA/ETD, milestone, linked transaction
Buttons/actions	“Refresh status” per container, row click → /shipments/{id}
Navigation links	To shipment detail, to linked transaction
Background job	Yes — manual refresh triggers an on-demand status pull
AI integration	No directly (parsing happens server-side)

Shipment
Detail

/APP/(PROTECTED)/SHIPMENTS/[ID]/PAGE.TSX → /
SHIPMENTS/{ID}

Auth required	Yes
Purpose	Single-container milestone history and post-delivery issue log
UI components	Milestone timeline, post-delivery issue form (quality/damage/detention), linked-transaction card
API endpoints consumed	GET /api/v1/shipments/{id} , POST /api/v1/shipments/{id}/issues
Inputs required	Issue type (Zod enum), description, supporting document upload (optional)
Outputs displayed	Full milestone history, logged issues
Buttons/actions	“Log issue,” “Refresh status”
Navigation links	To linked transaction
Background job	No
AI integration	No

Analytics

/APP/(PROTECTED)/ANALYTICS/PAGE.TSX → /ANALYTICS

Auth required	Yes
Purpose	FR-DB-04 automation KPIs — extraction accuracy, turnaround, automation %, integration failures
UI components	Recharts line (turnaround trend), bar (accuracy by document type), donut (automation vs. manual share); date-range picker
API endpoints consumed	GET /api/v1/dashboards/kpis?range=...
Inputs required	Date range, stream filter
Outputs displayed	Trend charts with tooltips, exportable as image/CSV
Buttons/actions	“Export chart data”
Navigation links	None beyond sidebar
Background job	No
AI integration	No

Reports

/APP/(PROTECTED)/REPORTS/PAGE.TSX → /REPORTS

Auth required	Yes
Purpose	Feature 11 — list of scheduled (daily/monthly) and prior ad-hoc reports
UI components	shadcn <code>Table</code> , type <code>Badge</code> (daily/monthly/adhoc), download icon button
API endpoints consumed	<code>GET /api/v1/reports</code>
Inputs required	Type/date filters
Outputs displayed	Report name, type, generated date, format, generation reference
Buttons/actions	“Download,” “New ad-hoc report” → /reports/builder
Navigation links	To /reports/builder, to /reports/{id}
Background job	No (scheduled generation is Feature 11’s own job)
AI integration	No

Ad-hoc Report Builder

/APP/(PROTECTED)/REPORTS/BUILDER/PAGE.TSX → /REPORTS/BUILDER

Auth required	Yes — authorised reporting role
Purpose	Self-service report generation per FR-RP-03
UI components	Filter form (date range, stream, status), format toggle (PDF/XLSX), “Generate” button with job-progress bar
API endpoints consumed	<code>POST /api/v1/reports</code> , <code>GET /api/v1/jobs/{job_id}/status</code>
Inputs required	<code>date_from</code> , <code>date_to</code> (required), <code>business_stream</code> , <code>status_filter</code> , <code>format</code>
Outputs displayed	Generation progress, then a preview + download link on completion
Buttons/actions	“Generate Report”
Navigation links	To /reports/{id} on completion
Background job	Yes
AI integration	Optional executive-summary paragraph only

Report Viewer	<code>/APP/(PROTECTED)/REPORTS/{ID}/PAGE.TSX → /REPORTS/{ID}</code>
Auth required	Yes
Purpose	Renders a generated report with drill-through to source transactions (FR-RP-04)
UI components	Inline PDF viewer, per-figure “view transactions” link
API endpoints consumed	<code>GET /api/v1/reports/{id}</code>
Inputs required	None
Outputs displayed	Full report content, generation metadata (query parameters used, generated-by, generated-at)
Buttons/actions	“Download,” “Re-generate with same filters”
Navigation links	To /transactions filtered by the figure clicked
Background job	No
AI integration	No

Audit Explorer	<code>/APP/(PROTECTED)/ADMIN/AUDIT/PAGE.TSX → /ADMIN/AUDIT</code>
Auth required	Yes — Admin/Auditor role
Purpose	Full, exportable event-log view for compliance (BR-12, Section 16.8)
UI components	shadcn <code>Table</code> with event-type filter, actor filter, entity-reference search, “Export CSV” button
API endpoints consumed	<code>GET /api/v1/audit</code>
Inputs required	Date range, event-type filter, actor filter, entity-reference search
Outputs displayed	Timestamp, actor, event type, entity reference, before/after summary (metadata only — never full document or AI-prompt text, Section 16.8)
Buttons/actions	“Export CSV”
Navigation links	To the referenced entity where the user’s role permits
Background job	No
AI integration	No

Admin — Users, Rules, Document Types

/APP/(PROTECTED)/ADMIN/{USERS,RULES,DOCUMENT-TYPES}/PAGE.TSX

Auth required	Yes — Admin role
Purpose	/admin/users: role assignment via Keycloak Admin API. /admin/rules: tolerance/mandatory-document/duplicate-threshold configuration (Feature 6). /admin/document-types: per-type, per-territory field-schema configuration (Feature 2)
UI components	shadcn Table + edit Dialog per config row; JSON-schema-aware field editor for document-type schemas; confirmation dialog on any change affecting live validation
API endpoints consumed	GET/PATCH /api/v1/admin/users , GET/PATCH /api/v1/admin/rules , GET/PATCH /api/v1/admin/document-types
Inputs required	Role assignment (per user), threshold values (numeric, validated against sane ranges), field-schema edits
Outputs displayed	Current configuration with last-changed-by/when metadata
Buttons/actions	“Save,” which requires a change-reason note, itself audit-logged
Navigation links	Cross-links between the three admin sub-areas
Background job	No
AI integration	No

Settings & Notifications

/APP/(PROTECTED)/{SETTINGS,NOTIFICATIONS}/PAGE.TSX

Auth required	Yes
Purpose	/settings: read-only profile (from Entra ID) plus notification-channel preference. /notifications: in-app notification centre (approvals due, exceptions assigned, reports ready)
UI components	Profile Card , preference Switch es, notification list with read/unread state
API endpoints consumed	GET /api/v1/users/me , PATCH /api/v1/users/me/preferences , GET /api/v1/notifications
Inputs required	Notification-channel toggle (in-app/email), default queue-filter preference
Outputs displayed	Profile fields, notification list
Buttons/actions	“Save preferences,” “Mark all as read”
Navigation links	Notification click → the relevant transaction/exception/approval/report
Background job	No
AI integration	No

Compact entries — public, auth-trigger and error pages

Page	Route	Components	Notes
Sign-in landing	/	Logo, one-line value statement, single SSO button	Redirects to /dashboard if already authenticated
Disclaimer / Privacy / Terms	/ disclaimer / privacy / terms	Static Markdown-rendered content in the shared legal layout	Footer-linked from every page
Sign-in trigger	/signin	Immediate redirect, no rendered UI	Invokes NextAuth.js signIn(“keycloak”)
Sign-out	/signout	Confirmation, then redirect	Clears session cookie, redirects through Keycloak/Entra ID logout
404 Not Found	not-found.tsx	Illustration + “Back to dashboard” button	Global catch-all
Error boundary	error.tsx	Friendly error message, “Retry” + “Report issue” buttons, Sentry event ID shown	Route-segment scoped per Next.js App Router convention
Offline	/offline		

Illustration, “You’re offline” message, list of what’s still available from cache, auto-retry indicator	Service-worker navigation fallback (Feature 12)
---	---

07

Backend API Endpoints — Complete Catalog

All paths are prefixed `/api/v1/` (omitted below for brevity). “Auth” = JWT required via `get_current_user`. “Rate-limited” = enforced by slowapi in production only (Section 16.3). Router modules follow the folder structure in Section 10.1.

Router: requests.py (email intake / request queue — Feature 1)

GET /requests		REQUESTS.PY
Auth / Rate-limited	Yes / Yes	
Purpose	Paginated, filterable request queue (Inbox page)	
Request schema	Query: page:int=1, page_size:int=25, category: CategoryEnum None, stream: StreamEnum None, min_confidence: float None, search: str None	
Response schema	RequestListOut{items: list[RequestSummary], total:int, page:int, page_size:int, total_pages:int} where RequestSummary{id:UUID, sender:str, subject:str, received_at:datetime, category:str, confidence:float, attachment_count:int, status:str}	
Processing logic	Validates query params → builds a filtered, indexed SQLAlchemy query on <code>requests</code> joined to <code>email_messages</code> → applies role-based stream filter from the JWT claims → returns page	
Error responses	401 missing/invalid token; 422 invalid query params	
Background job / AI	No / No (reads Feature 1's prior output)	

PATCH /requests/{id}/category

REQUESTS.PY

Auth / Rate-limited	Yes / Yes
Purpose	User-corrected category override (Feature 1 low-confidence flow)
Request schema	<code>CategoryOverrideIn{category: CategoryEnum, reason: str = Field(min_length=10)}</code>
Response schema	<code>RequestSummary</code> (updated)
Processing logic	Validates the request exists and belongs to a stream the user's role can edit → writes the override, retaining the original AI value (BR-11) → emits <code>RequestClassified</code> (manual) audit event
Error responses	<code>403</code> role mismatch; <code>404</code> not found; <code>422</code> reason too short
Background job / AI	No / No

Router: documents.py (classification, extraction, upload — Feature 2)**POST /documents/upload**

DOCUMENTS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Manual/portal intake route (FR-PU-01 portal path); backs / inbox/upload
Request schema	<code>multipart/form-data</code> : <code>files: list[UploadFile], stream: StreamEnum, document_type_hint: DocTypeEnum None</code>
Response schema	<code>UploadAcceptedOut{request_id:UUID, documents: list[{id:UUID, filename:str, status:str}]}</code>
Processing logic	Validates content-type by magic bytes (not extension) and size (≤25MB) → stores via <code>storage_service</code> with a UUID filename → creates a Request + Document row per file → queues classification/extraction job
Error responses	<code>413</code> file too large; <code>415</code> unsupported type; <code>422</code> missing stream
Background job / AI	Yes (classification+extraction job) / Yes (Gemini)

GET /documents · GET /documents/{id}

DOCUMENTS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Document list (search) and single-document extraction detail (Document Review page)
Request schema	List: <code>page</code> , <code>page_size</code> , <code>search</code> , <code>document_type</code> , <code>date_from</code> , <code>date_to</code> . Detail: path <code>id</code> : <code>UUID</code>
Response schema	<code>DocumentOut{id, filename, document_type, transaction_id, received_at, extraction_status, fields: list[ExtractedField], page_image_urls: list[str]}</code>
Processing logic	Detail endpoint resolves signed, short-lived URLs for page images from <code>storage_service</code> rather than exposing the raw file path; ownership/role check against the linked transaction's stream
Error responses	<code>403</code> stream mismatch; <code>404</code> not found
Background job / AI	No / No (reads prior extraction)

PATCH /documents/{id}/fields · POST /documents/{id}/confirm

DOCUMENTS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Inline field correction and extraction confirmation (Feature 2)
Request schema	<code>FieldCorrectionIn{corrections: list[{field:str, value:str, reason:str None}]}</code> ; confirm takes no body
Response schema	<code>DocumentOut</code> (updated)
Processing logic	Each correction is validated against the field's typed schema (Section 9.3) → original AI value retained alongside the override (BR-11) → confirm transitions the document to <code>Extracted</code> and triggers Matching (Feature 3/4/5)
Error responses	<code>422</code> value fails type/tolerance validation; <code>409</code> document already confirmed
Background job / AI	Confirm triggers matching job / No (correction itself is not an AI call)

Router: transactions.py (Features 3, 4, 5)

GET /transactions · GET /transactions/{id}

TRANSACTIONS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Cross-stream list and single-transaction full detail (Transactions List, Purchase/Sales/Fa Workspaces)
Request schema	List: <code>page, page_size, sort_by, sort_order, search, stream, status</code> . Detail: path <code>id: UUID</code>
Response schema	<code>TransactionOut{id, stream, status, counterparty, batch_no, contract_no, commodity_code, quantity_mt, currency, amount, purchase_leg, sales_leg, fa_leg, validation_results: list[RuleEvaluationOut], linked_documents, linked_shipment, status_history}</code>
Processing logic	Detail endpoint assembles the full transaction graph (purchase/sales/fa leg, validations, documents, shipment) in one call to avoid the frontend issuing five requests to render one workspace
Error responses	<code>403</code> stream/role mismatch; <code>404</code>
Background job / AI	No / No

POST /transactions

TRANSACTIONS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Manual transaction registration (New Transaction page — no email trigger)
Request schema	<code>TransactionCreateIn{stream: StreamEnum, counterparty_name:str, contract_no:str None, batch_no:str None}</code>
Response schema	<code>TransactionOut</code> (newly created, status <code>Received</code>)
Processing logic	Issues a Request ID (BR-01) → proposes next sequential batch number if none supplied → creates the stream-appropriate leg row (Purchase/Sales/Fa)
Error responses	<code>422</code> missing counterparty/stream
Background job / AI	No / No

PATCH /transactions/{id}/fields

TRANSACTIONS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Corrects transaction-level fields and re-runs the Validation Engine (Feature 6)
Request schema	<code>TransactionFieldsIn</code> — stream-specific Pydantic model (<code>PurchaseLegIn</code> / <code>SalesLegIn</code> / <code>FaLegIn</code> , Section 4), partial-update semantics
Response schema	<code>TransactionOut</code> (updated, with fresh <code>validation_results</code>)
Processing logic	Applies the update inside a DB transaction → synchronously re-runs BR-03...08 → persists new <code>rule_evaluations</code> → emits <code>ValidationPassed</code> / <code>ValidationBlocked</code>
Error responses	<code>409</code> transaction already approved/committed (immutable past that point); <code>422</code> field validation failure
Background job / AI	No (synchronous) / No

POST /transactions/{id}/generate-draft

TRANSACTIONS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Generates a draft sales contract/invoice from an approved template (Feature 4)
Request schema	<code>DraftGenerateIn{template_id:UUID, price_fixation: {fixed:bool, rate:Decimal} None, fixed_at:date None} None}</code>
Response schema	<code>JobAcceptedOut{job_id:UUID}</code> (poll via <code>/jobs/{job_id}/status</code>)
Processing logic	Calls <code>gemini_service</code> for structured clause content → hands structured content to <code>docx_service</code> / <code>pdf_service</code> to populate the territory-specific template → stores the draft via <code>storage_service</code> , never auto-sent
Error responses	<code>409</code> BR-07 gate not yet satisfied (final shipping validation pending); <code>422</code> missing template
Background job / AI	Yes / Yes — Gemini 2.5 Flash for content, python-docx/ PyMuPDF for binary

POST /transactions/{id}/submit

TRANSACTIONS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Submits a validated transaction into the Approval Queue (Feature 7)
Request schema	No body
Response schema	<code>TransactionOut{status:"Approval Pending", ...}</code>
Processing logic	Hard server-side check that every required <code>rule_evaluations</code> row currently passes — this is the enforcement point, not merely a UI-disabled button — before creating the ApprovalTask
Error responses	<code>409</code> outstanding validation failures (returns the specific failing rules)
Background job / AI	No / No

Router: exceptions.py & approvals.py (Feature 7)

GET /exceptions · GET /exceptions/{id}

EXCEPTIONS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Exception Queue list and detail (Exception Handling Matrix)
Request schema	List: <code>category, owner_role, min_age_hours, page, page_size</code> . Detail: path <code>id</code>
Response schema	<code>ExceptionCaseOut{id, exception_type, transaction_id, owner_role, opened_at, age_hours, priority, rule_id, field, expected, actual, resolution_note}</code>
Processing logic	Ageing computed from <code>opened_at</code> at query time (not stored stale); category/owner mapping resolved from the configured Exception Handling Matrix table
Error responses	<code>404</code>
Background job / AI	No / No

POST /exceptions/{id}/resolve

EXCEPTIONS.PY

Auth / Rate-limited Yes / Yes

Purpose Records resolution and re-triggers validation at the point the transaction left the pipeline

Request schema `ExceptionResolveIn{resolution_note:str=Field(min_length=10), field_correction: dict|None, escalate_to_hod:bool=False}`

Response schema `ExceptionCaseOut{status:"resolved", ...}`

Processing logic Applies any field correction through the same path as `PATCH / transactions/{id}/fields` → re-runs the specific failed rule → on pass, emits `ExceptionResolved` and returns the transaction to its pre-exception pipeline position

Error responses `422` note too short; `409` rule still failing after correction

Background job / AI No / No

GET /approvals · GET /approvals/{id}

APPROVALS.PY

Auth / Rate-limited Yes (Approver role) / Yes

Purpose Approval Queue list and single-decision detail

Request schema List: `rank_by: Literal["age", "value", "risk"], stream, page, page_size`. Detail: path `id`

Response schema `ApprovalTaskOut{id, transaction_id, requested_at, value, age_hours, ai_summary:str, decision, decided_by, decided_at, reason}`

Processing logic Ranking applied server-side per `rank_by`; `ai_summary` generated lazily on first view and cached, not regenerated per request

Error responses `403` non-approver role; `404`

Background job / AI No / Yes — summary generation only

POST /approvals/{id}/decide

APPROVALS.PY

Auth / Rate-limited	Yes (Approver role) / Yes
Purpose	Records the human approval decision — the platform’s single most safety-critical write
Request schema	<code>ApprovalDecisionIn{decision: Literal[“approved”, “rejected”, “changes_requested”], reason:str None}</code> — <code>reason</code> required unless <code>approved</code> (Pydantic validator)
Response schema	<code>ApprovalTaskOut</code> (decided)
Processing logic	Persists <code>decided_by</code> (from JWT <code>sub</code> , never client-supplied), <code>decided_at</code> (server clock) → on <code>approved</code> , transitions the transaction to <code>Approved</code> and enqueues Feature 8’s integration jobs — this transition has no other entry point in the codebase (BR-10 enforcement point)
Error responses	<code>403</code> user is not the assigned/eligible approver; <code>409</code> already decided; <code>422</code> missing required reason
Background job / AI	Approved → triggers Feature 8 jobs / No

Router: integrations.py (Feature 8)

GET /integrations/jobs · POST /integrations/jobs/{id}/retry INTEGRATIONS.PY

Auth / Rate-limited	Yes (IT/Admin role) / Yes
Purpose	Integration Monitor list and manual retry
Request schema	List: <code>target_system: Literal["tracker", "sap", "dms"], status, page, page_size</code> . Retry: no body
Response schema	<code>IntegrationJobOut{id, transaction_id, target_system, status, external_reference, failure_reason, attempt_count, last_attempted_at}</code>
Processing logic	Retry re-queues the specific target-system job only (never the other two) with backoff reset; each target system's connector (tracker via Graph Excel API, SAP via OData/BAPI or Power-Automate-assisted flow, DMS via its REST API) is invoked independently
Error responses	<code>409</code> job already succeeded; <code>502</code> downstream system unreachable (surfaced with the raw failure reason, never swallowed)
Background job / AI	Yes / No

Router: shipments.py (Feature 9)

GET /shipments · GET /shipments/{id} · POST /shipments/{id}/refresh · POST /shipments/{id}/issues SHIPMENTS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Shipment Dashboard, detail, on-demand refresh, post-delivery issue logging
Request schema	List: status, carrier, pod, page, page_size . Issues: ShipmentIssueIn{issue_type: Literal["quality","damage","detention","other"], description:str, document_id:UUID None}
Response schema	ShipmentOut (Section 4 Feature 9 schema) plus milestones: list[MilestoneEvent] and issues: list[ShipmentIssueOut] on detail
Processing logic	Refresh triggers a single-container on-demand pull through the same connector the scheduled job uses, so behaviour is identical whether triggered manually or on schedule
Error responses	404 ; 502 carrier source unavailable (returns last-known status, not an error page)
Background job / AI	Refresh: yes / Milestone-text parsing: yes (Gemini)

Router: dashboards.py (Feature 10) & reports.py (Feature 11)

GET /dashboards/summary · GET /dashboards/kpis DASHBOARDS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Backs the Dashboard and Analytics pages
Request schema	stream, date_from, date_to
Response schema	DashboardSummaryOut{status_counts, exception_counts_by_category, approval_queue_depth, shipment_status_mix}; KpiOut{automation_pct, extraction_accuracy_pct, avg_turnaround_minutes, integration_failure_count, trend: list[{date,value}]}
Processing logic	Server-side aggregation over indexed status/date columns with a 30–60s cache (Feature 10); role-based row filtering applied before aggregation, not after
Error responses	422 invalid date range
Background job / AI	No / No

GET /reports · POST /reports · GET /reports/{id} REPORTS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Report list, ad-hoc generation, single-report retrieval
Request schema	ReportRequestIn (Section 4 Feature 11 schema)
Response schema	JobAcceptedOut{job_id} on POST; ReportOut{id, type, format, generated_at, generation_ref, download_url, figures: list[{label,value,drill_through_filter}]} on GET
Processing logic	Queries governed tables directly (never a denormalised copy) → renders via pandas + python-docx/PyMuPDF/openpyxl → stores the file immutably with a generation_ref resolvable in the Audit Explorer
Error responses	422 invalid filter combination
Background job / AI	Yes / Optional executive-summary paragraph only

Router: admin.py & audit.py

GET/PATCH /admin/users · GET/PATCH /admin/rules · GET/PATCH /admin/document-types ADMIN.PY

Auth / Rate-limited	Yes (Admin role, <code>get_current_admin_user</code>) / Yes
Purpose	Role assignment (proxied to Keycloak Admin API), tolerance/threshold configuration, document-type field-schema configuration
Request schema	<code>UserRoleUpdateIn{user_id:UUID, roles:list[RoleEnum]}; RuleConfigIn{rule_id:str, scope:dict, threshold_value:Decimal, active:bool}; DocTypeSchemaIn{document_type:str, territory:str None, fields:list[FieldSchema]}</code>
Response schema	Echoes the updated configuration row plus <code>updated_by</code> / <code>updated_at</code>
Processing logic	Every PATCH requires a <code>change_reason</code> field, persisted and audit-logged (BR-12) before the configuration takes effect; user-role PATCH calls the Keycloak Admin API synchronously and only commits locally on success
Error responses	<code>403</code> non-admin; <code>502</code> Keycloak Admin API unreachable (local state left unchanged)
Background job / AI	No / No

GET /audit

AUDIT.PY

Auth / Rate-limited	Yes (Admin/Auditor role) / Yes
Purpose	Audit Explorer — the read surface over every domain event (Section 13.8)
Request schema	<code>event_type, actor_id, entity_type, entity_id, date_from, date_to, page, page_size</code>
Response schema	<code>AuditEventOut{id, occurred_at, actor, event_type, entity_type, entity_id, metadata: dict}</code> — <code>metadata</code> excludes full document/AI-prompt text by construction (Section 16.8)
Processing logic	Read-only over the append-only <code>audit_events</code> table; export endpoint streams CSV rather than materialising the full result set in memory
Error responses	<code>403</code> non-admin/auditor
Background job / AI	No / No

Router: jobs.py, users.py, notifications.py, health.py (cross-cutting)

GET /jobs/{job_id}/status

JOBS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Universal polling endpoint for every background job in the platform (Section 12.7)
Request schema	Path <code>job_id: UUID</code>
Response schema	<code>JobStatusOut{job_id, status: Literal["queued", "processing", "completed", "failed"], progress:int, result_url:str None, error_message:str None}</code>
Processing logic	Reads the job's row from the shared <code>background_jobs</code> table; never re-derives status from the target system directly, so polling cost stays O(1) regardless of job type
Error responses	<code>404</code> unknown job; <code>403</code> job belongs to a transaction outside the caller's role scope
Background job / AI	N/A (this endpoint reads job state) / No

GET /users/me · PATCH /users/me/ preferences · GET /notifications

USERS.PY,
NOTIFICATIONS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Profile read, preference update, notification centre
Request schema	<code>UserPreferencesIn{notification_channel: Literal["in_app","email","both"], default_stream_filter: StreamEnum None}</code>
Response schema	<code>UserOut{id, display_name, email, roles, preferences}; NotificationOut{id, type, message, read, created_at, deep_link}</code>
Processing logic	Profile fields are read-only mirrors of Entra ID claims from the JWT — never independently editable, to avoid the portal's profile drifting from the corporate directory
Error responses	401
Background job / AI	No / No

GET /health · GET /health/ready

HEALTH.PY

Auth / Rate-limited	No / No (excluded from slowapi and from JWT middleware)
Purpose	Cloud Run liveness/readiness probes (Section 17.1)
Request schema	None
Response schema	<code>{"status":"ok"}</code> ; readiness additionally checks DB connectivity and returns 503 if the pool cannot acquire a connection
Processing logic	Liveness never touches the database; readiness does a lightweight <code>SELECT 1</code>
Error responses	503 not ready
Background job / AI	No / No

Router: notifications.py — additions for Feature 12 (Web Push)

GET /notifications/vapid-public-key · POST / notifications/push-subscribe · DELETE / notifications/push-subscribe NOTIFICATIONS.PY

Auth / Rate-limited	Yes / Yes
Purpose	Supplies the VAPID public key to the frontend, registers a browser's push subscription, and removes it on opt-out or on a 410 Gone from the push service
Request schema	PushSubscriptionIn{endpoint:str, keys:{p256dh:str, auth:str}}
Response schema	{"vapid_public_key": str}; {"id": UUID, "status": "subscribed"}
Processing logic	Subscription is upserted keyed on (user_id, endpoint) so re-subscribing the same browser doesn't create duplicate rows; the VAPID private key never leaves Google Secret Manager / the backend process
Error responses	422 malformed subscription payload
Background job / AI	No / No

SECTION 8

08 Page-by-Page UI Breakdown

How this section is organised

Section 7 already specifies every page's components, endpoints and actions. This section describes the shared chrome (header, sidebar, footer) once, then the layout/visual treatment for each distinct page *archetype* — workspace, queue/list, dashboard, form, admin/config — since every page in Section 7 is an instance of one of these five archetypes and repeating identical layout language 24 times would add length without adding information.

Global Header / Navigation (every authenticated page)

Fixed top bar, `h-16`, `bg-white dark:bg-slate-900`, bottom border `border-b border-slate-200`. Left: AGFZE Command Centre wordmark + current section breadcrumb. Centre: global search (`Cmd/Ctrl+K` command palette via shadcn `Command`) across transactions/documents/shipments. Right: notification bell with unread-count badge (opens a `Popover` preview of /notifications), role badge, user avatar `DropdownMenu` (Profile, Settings, Sign out). Below `md` breakpoint, the left sidebar collapses into a Radix `Sheet` triggered by a hamburger icon in the header; the search and notification icons remain visible, the breadcrumb hides.

Left sidebar (desktop, `w-60`, collapsible to icon-only `w-16`): grouped nav — Dashboard; Inbox; Transactions; Documents; Exceptions; Approvals (role-gated); Shipments; Analytics; Reports; Admin (role-gated) — each item an icon (lucide-react) + label, active state `bg-slate-100` with a 3px left accent border in copper (`#A75D35`), badge counts for Exceptions/Approvals pulled from the same aggregate the Dashboard uses.

Archetype 1 — Workspace pages (Purchase/Sales/FA Transaction Workspace, Document Review)

Two-column layout at `lg+` (`grid grid-cols-[1fr_1fr] gap-6`): left column is the source-document viewer (sticky, scrollable independently, page-navigation controls, zoom controls, `bg-slate-50` letterboxing); right column is a vertically stacked set of shadcn `Accordion` sections (Extraction, Matching, Validation, History) that default to Extraction and Validation open, Matching/History collapsed, so the highest-signal information is visible without scrolling first — directly implementing the “most important information first, progressive disclosure for detail” principle (Section 14.1). Below `lg`, columns stack vertically with the document viewer first and a “Jump to fields” sticky mini-nav.

Forms within workspaces: every editable field is a shadcn `Input / Select` pre-filled with the AI value, left-bordered in the confidence colour (green/amber/red per Section 4 Feature 2); editing a field below the confidence threshold opens an inline reason `Textarea` (Zod: required, min 10 characters) before the field can be saved; Save is a sticky bottom action bar with “Save changes” (secondary) and the stage-

appropriate primary action (“Submit for Approval,” disabled with a tooltip explaining exactly which rule is still failing, until validation passes). Success shows a `react-hot-toast` confirmation (auto-dismiss 5s); validation failure keeps the user on-page with the specific failing field scrolled into view and highlighted, never a generic error toast alone.

Archetype 2 — Queue / list pages (Inbox, Transactions, Documents, Exceptions, Approvals, Shipments, Reports, Audit Explorer)

Full-width shadcn `Table` below a filter bar (`flex flex-wrap gap-2` of `Select / DatePicker / Input` controls); table header sticky on scroll; row hover `bg-slate-50`; status/category shown as a coloured `Badge` using one consistent palette platform-wide (see Section 14.9). Pagination is a shadcn `Pagination` control bottom-right showing “{page_size} of {total}”; sort is column-header click with a small chevron indicator. On mobile (below `md`), the table becomes a stacked card list — each row’s key fields render as a compact card (title line + two metadata lines + status badge) rather than a horizontally scrolling table, per Section 14.6.

Archetype 3 — Dashboard-style pages (Dashboard, Analytics, Shipment Dashboard)

Responsive KPI-tile grid (`grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4`) above a two-column chart row (`grid grid-cols-1 xl:grid-cols-2 gap-6`) containing Recharts components — donut for status mix, bar for exception-by-category, line for turnaround trend — each in a shadcn `Card` with a consistent header (title + date-range control) and a `ResponsiveContainer` so charts reflow rather than overflow at narrow widths. Every tile and chart segment is a button-like element (`cursor-pointer`, subtle hover elevation) that deep-links to the filtered queue behind it.

Archetype 4 — Focused form pages (New Transaction, Ad-hoc Report Builder, Manual Intake/Upload)

Centred single-column form, max width `max-w-2xl`, a shadcn `Progress` /stepper for multi-step forms (New Transaction), grouped field sections with a section label and helper microcopy above each group rather than a wall of unlabelled inputs. The Manual Intake page’s drop zone is a large dashed-border target (`border-2 border-dashed`) with a copper highlight on drag-over, per-file rows appearing below with a progress bar and a remove (×) affordance before submission.

Archetype 5 — Admin/config pages (Admin Users/Rules/Document Types, Integration Monitor)

Table-plus-edit-dialog pattern: the main view is Archetype 2’s list pattern; editing opens a shadcn `Dialog` (not a full page navigation) containing the specific configuration form, always ending in a required “reason for this change” field before Save is enabled — this is a deliberate, repeated UI constraint (not just a backend

validation) so that every admin edit visibly costs the same small amount of friction as a database migration would, discouraging casual, undocumented threshold changes.

Loading States

Workspace and queue pages show shadow `Skeleton` blocks shaped like the eventual content (table-row skeletons for queues, field-shaped skeletons for workspaces) rather than a generic spinner, so the layout doesn't jump on load. Background-job-backed actions (extraction, matching, draft generation, report generation) show a determinate `Progress` bar bound to the polled `progress` field with status microcopy ("Extracting document... usually 10–30 seconds," "Generating draft contract...") — the user can navigate away and return; a toast notifies on completion if they've left the page.

Empty States

Every list/queue has a dedicated empty state: a simple line illustration, a one-line explanation ("No exceptions right now — nice work" for Exceptions; "Nothing pending your approval" for Approvals), and, where relevant, a single action (e.g. Documents' empty state offers "Upload a document").

Error States

Inline field errors render directly beneath the field in red, small text, paired with the exact fix needed ("Quantity must be within 5% of the contracted 125 MT — you entered 132 MT"); API-level failures show a `react-hot-toast` error plus, for a failed background job, an inline `Alert` with a "Retry" button that re-submits the same job rather than forcing the user to re-enter data. Route-level failures render Next.js's `error.tsx` boundary: a calm, non-technical message, a "Try again" button, a "Report this issue" link (opens a pre-filled support request with the Sentry event ID attached), and a link back to the Dashboard.

Footer

Slim footer (not present inside workspace pages, to avoid competing with the document viewer for vertical space; present on Dashboard, queue pages, and all public pages): AGFZE Command Centre © current year, links to Disclaimer/Privacy/Terms, and — persistently, not just on a legal page — a one-line reminder: "AI-extracted data must be verified before approval. See Disclaimer."

SECTION 9

09 Database Structure

SPECIFIED PostgreSQL 15 (Cloud SQL in production), SQLAlchemy 2.0 async + asyncpg, Alembic migrations — exactly as mandated.

SOURCE A second, non-relational **graph data layer** (Section 9.6) is added — TDD §10.1 names it explicitly as a first-class store (“operational database, graph database, object store and audit log”) with no equivalent anywhere in the reference stack.

24 tables across six groups: identity & intake, the transaction graph (parent + three legs), logistics, governance (rules/exceptions/approvals), integration & documents, and configuration/reference data. Every table uses a UUID primary key (`uuid4`, generated application-side) rather than a serial integer, so that IDs are safe to reference across the relational store, the graph store and the immutable object store without collision.

```
from sqlalchemy import String, Integer, Numeric, Boolean, Text,
ForeignKey, DateTime, Date, UniqueConstraint, Index
from sqlalchemy.dialects.postgresql import UUID, JSONB, ARRAY, ENUM
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column,
relationship
from datetime import datetime, date
from decimal import Decimal
import uuid

class Base(DeclarativeBase):
    pass
```


9.1 — Identity & Intake

```

class User(Base):
    __tablename__ = "users"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    keycloak_id: Mapped[str] = mapped_column(String(255), unique=True,
nullable=False, index=True)
    entra_object_id: Mapped[str | None] = mapped_column(String(255),
unique=True, nullable=True)
    email: Mapped[str] = mapped_column(String(255), unique=True,
nullable=False, index=True)
    display_name: Mapped[str] = mapped_column(String(255),
nullable=False)
    roles: Mapped[list[str]] = mapped_column(ARRAY(String(50)),
nullable=False, default=list)
    default_stream_filter: Mapped[str | None] = mapped_column(String(20),
nullable=True)
    notification_channel: Mapped[str] = mapped_column(String(20),
nullable=False, default="in_app")
    is_active: Mapped[bool] = mapped_column(Boolean, nullable=False,
default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()")
    last_login_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)

class EmailMessage(Base):
    __tablename__ = "email_messages"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    graph_message_id: Mapped[str] = mapped_column(String(512),
unique=True, nullable=False, index=True)
    mailbox: Mapped[str] = mapped_column(String(255), nullable=False,
index=True)
    sender: Mapped[str] = mapped_column(String(255), nullable=False)
    subject: Mapped[str] = mapped_column(String(998), nullable=False)
    body_text: Mapped[str] = mapped_column(Text, nullable=False)
    received_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), nullable=False, index=True)
    raw_stored_ref: Mapped[str] = mapped_column(String(512),
nullable=False) # immutable object-store pointer

class Request(Base):
    __tablename__ = "requests"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    request_code: Mapped[str] = mapped_column(String(32), unique=True,
nullable=False, index=True) # REQ-20260823-0001, BR-01
    source: Mapped[str] = mapped_column(ENUM("email", "portal",
name="request_source"), nullable=False)
    email_message_id: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("email_messages.id",
ondelete="SET NULL"), nullable=True)
    category: Mapped[str | None] = mapped_column(String(30),
nullable=True) # purchase/sales/fa/logistics/approval/follow_up/
informational/exception
    category_confidence: Mapped[Decimal | None] =
mapped_column(Numeric(4, 3), nullable=True)
    category_overridden: Mapped[bool] = mapped_column(Boolean,
nullable=False, default=False)
    stream: Mapped[str | None] = mapped_column(String(10), nullable=True,

```

```

index=True) # scrap | fa
    status: Mapped[str] = mapped_column(String(30), nullable=False,
default="Received", index=True) # Section 13.8 state machine
    created_by: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("users.id", ondelete="SET
NULL"), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()", index=True)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()", onupdate=datetime.utcnow)

class Document(Base):
    __tablename__ = "documents"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    request_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("requests.id", ondelete="CASCADE"), nullable=False,
index=True)
    transaction_id: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("trade_transactions.id",
ondelete="SET NULL"), nullable=True, index=True)
    filename: Mapped[str] = mapped_column(String(512), nullable=False)
    document_type: Mapped[str | None] = mapped_column(String(30),
nullable=True, index=True) # invoice/contract/bl/tracker/
approval_evidence/fa_document/unknown
    territory: Mapped[str | None] = mapped_column(String(20),
nullable=True) # india/china/japan/other
    storage_ref: Mapped[str] = mapped_column(String(512),
nullable=False) # storage-agnostic reference, never a raw path
    content_hash: Mapped[str] = mapped_column(String(64), nullable=False,
index=True) # BR-13 duplicate detection
    page_count: Mapped[int] = mapped_column(Integer, nullable=False,
default=1)
    extraction_status: Mapped[str] = mapped_column(String(20),
nullable=False, default="pending", index=True)
    classification_confidence: Mapped[Decimal | None] =
mapped_column(Numeric(4, 3), nullable=True)
    uploaded_by: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("users.id", ondelete="SET
NULL"), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()")
    __table_args__ = (Index("ix_documents_content_hash_type",
"content_hash", "document_type"),)

class ExtractedField(Base):
    __tablename__ = "extracted_fields"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    document_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("documents.id", ondelete="CASCADE"), nullable=False,
index=True)
    field_name: Mapped[str] = mapped_column(String(100), nullable=False)
    field_value: Mapped[str | None] = mapped_column(Text, nullable=True)
    confidence: Mapped[Decimal | None] = mapped_column(Numeric(4, 3),
nullable=True)
    source_page: Mapped[int | None] = mapped_column(Integer,
nullable=True)
    source_ref: Mapped[str | None] = mapped_column(Text, nullable=True)
# paragraph/bbox reference back into the source file

```

```
is_overridden: Mapped[bool] = mapped_column(Boolean, nullable=False,
default=False)
original_ai_value: Mapped[str | None] = mapped_column(Text,
nullable=True) # BR-11: retained even after override
override_reason: Mapped[str | None] = mapped_column(Text,
nullable=True)
overridden_by: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("users.id", ondelete="SET
NULL"), nullable=True)
overridden_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)
__table_args__ = (UniqueConstraint("document_id", "field_name",
name="uq_document_field"),)
```

9.2 — The Transaction Graph (parent + three legs)

```

class TradeTransaction(Base):
    __tablename__ = "trade_transactions"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    transaction_code: Mapped[str] = mapped_column(String(64),
unique=True, nullable=False, index=True)
    stream: Mapped[str] = mapped_column(String(10), nullable=False,
index=True) # scrap | fa
    status: Mapped[str] = mapped_column(String(30), nullable=False,
default="Received", index=True) # Section 13.8
    commodity_code: Mapped[str | None] = mapped_column(String(16),
ForeignKey("commodity_codes.code", ondelete="SET NULL"), nullable=True,
index=True)
    batch_no: Mapped[str | None] = mapped_column(String(32), unique=True,
nullable=True, index=True)
    quantity_mt: Mapped[Decimal | None] = mapped_column(Numeric(12, 3),
nullable=True)
    price_basis: Mapped[str | None] = mapped_column(String(20),
nullable=True) # fixed | lme_percent
    lme_percent: Mapped[Decimal | None] = mapped_column(Numeric(6, 3),
nullable=True)
    currency: Mapped[str] = mapped_column(String(3), nullable=False,
default="USD")
    request_id: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("requests.id", ondelete="SET
NULL"), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()", index=True)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()", onupdate=datetime.utcnow)
    closed_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)
    purchase_leg: Mapped["PurchaseLeg | None"] =
relationship(back_populates="transaction", uselist=False, cascade="all,
delete-orphan")
    sales_leg: Mapped["SalesLeg | None"] =
relationship(back_populates="transaction", uselist=False, cascade="all,
delete-orphan")
    fa_leg: Mapped["FaLeg | None"] =
relationship(back_populates="transaction", uselist=False, cascade="all,
delete-orphan")
    __table_args__ = (Index("ix_txn_stream_status", "stream", "status"),)

class PurchaseLeg(Base):
    __tablename__ = "purchase_legs"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    transaction_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("trade_transactions.id", ondelete="CASCADE"), unique=True,
nullable=False)
    supplier_name: Mapped[str] = mapped_column(String(255),
nullable=False, index=True)
    supplier_invoice_no: Mapped[str | None] = mapped_column(String(100),
nullable=True, index=True)
    contract_no: Mapped[str] = mapped_column(String(100), nullable=False,
index=True)
    invoice_status: Mapped[str] = mapped_column(String(20),
nullable=False, default="provisional") # provisional | final
    amount: Mapped[Decimal | None] = mapped_column(Numeric(14, 2),
nullable=True)

```

```

        rate: Mapped[Decimal | None] = mapped_column(Numeric(12, 4),
        nullable=True)
        advance_payment_pct: Mapped[Decimal | None] =
        mapped_column(Numeric(5, 2), nullable=True)
        hedge_date: Mapped[date | None] = mapped_column(Date, nullable=True)
        pol: Mapped[str | None] = mapped_column(String(100), nullable=True)
        created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
        nullable=False, server_default="now()")
        transaction: Mapped["TradeTransaction"] =
        relationship(back_populates="purchase_leg")

class SalesLeg(Base):
    __tablename__ = "sales_legs"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
    primary_key=True, default=uuid.uuid4)
    transaction_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
    ForeignKey("trade_transactions.id", ondelete="CASCADE"), unique=True,
    nullable=False)
    customer_name: Mapped[str] = mapped_column(String(255),
    nullable=False, index=True)
    territory: Mapped[str] = mapped_column(String(20), nullable=False) #
    india | china | japan | other
    sales_contract_no: Mapped[str | None] = mapped_column(String(100),
    nullable=True, index=True)
    sales_invoice_no: Mapped[str | None] = mapped_column(String(100),
    nullable=True, index=True)
    bl_reference: Mapped[str | None] = mapped_column(String(100),
    nullable=True, index=True)
    payment_condition: Mapped[str | None] = mapped_column(String(10),
    nullable=True) # CAD | TT
    customer_fixation_status: Mapped[str] = mapped_column(String(20),
    nullable=False, default="unfixed")
    fixation_rate: Mapped[Decimal | None] = mapped_column(Numeric(12, 4),
    nullable=True)
    fixation_date: Mapped[date | None] = mapped_column(Date,
    nullable=True)
    pod: Mapped[str | None] = mapped_column(String(100), nullable=True)
    icd: Mapped[str | None] = mapped_column(String(100), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
    nullable=False, server_default="now()")
    transaction: Mapped["TradeTransaction"] =
    relationship(back_populates="sales_leg")

class FaLeg(Base):
    __tablename__ = "fa_legs"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
    primary_key=True, default=uuid.uuid4)
    transaction_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
    ForeignKey("trade_transactions.id", ondelete="CASCADE"), unique=True,
    nullable=False)
    counterparty_name: Mapped[str] = mapped_column(String(255),
    nullable=False, index=True)
    fa_contract_ref: Mapped[str | None] = mapped_column(String(100),
    nullable=True, index=True)
    document_type: Mapped[str | None] = mapped_column(String(50),
    nullable=True)
    extra_fields: Mapped[dict] = mapped_column(JSONB, nullable=False,
    default=dict) # discovery-configured, undetermined at PRD time
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
    nullable=False, server_default="now()")

```

```
transaction: Mapped["TradeTransaction"] =  
relationship(back_populates="fa_leg")
```

Indexes: `ix_txn_stream_status` supports the Dashboard's per-stream status-count queries (Feature 10); `contract_no` / `batch_no` / `bl_reference` are individually indexed because they are the Matching Agent's primary lookup keys (Feature 3/4, BR-02).

9.3 — Logistics

```

class Container(Base):
    __tablename__ = "containers"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    transaction_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("trade_transactions.id", ondelete="CASCADE"), nullable=False,
index=True)
    container_no: Mapped[str] = mapped_column(String(20), nullable=False,
index=True) # BR-03 match key
    seal_no: Mapped[str | None] = mapped_column(String(30),
nullable=True)
    quantity_mt: Mapped[Decimal | None] = mapped_column(Numeric(10, 3),
nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()")

class Shipment(Base):
    __tablename__ = "shipments"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    transaction_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("trade_transactions.id", ondelete="CASCADE"), nullable=False,
index=True)
    container_id: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("containers.id",
ondelete="SET NULL"), nullable=True)
    bl_no: Mapped[str | None] = mapped_column(String(50), nullable=True,
index=True)
    carrier: Mapped[str | None] = mapped_column(String(100),
nullable=True)
    vessel: Mapped[str | None] = mapped_column(String(100),
nullable=True)
    pol: Mapped[str | None] = mapped_column(String(100), nullable=True)
    pod: Mapped[str | None] = mapped_column(String(100), nullable=True)
    etd: Mapped[date | None] = mapped_column(Date, nullable=True)
    eta: Mapped[date | None] = mapped_column(Date, nullable=True)
    milestone: Mapped[str | None] = mapped_column(String(100),
nullable=True)
    status: Mapped[str] = mapped_column(String(20), nullable=False,
default="on_schedule", index=True) # on_schedule|delayed|arrived|
exception
    last_checked_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()")

class BillOfLading(Base):
    __tablename__ = "bill_of_ladings"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    shipment_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("shipments.id", ondelete="CASCADE"), nullable=False,
index=True)
    bl_type: Mapped[str] = mapped_column(String(20), nullable=False,
default="original") # original|seaway|draft
    bl_number: Mapped[str] = mapped_column(String(50), nullable=False,
index=True)
    is_original_received: Mapped[bool] = mapped_column(Boolean,
nullable=False, default=False)
    document_id: Mapped[uuid.UUID | None] =

```

```
mapped_column(UUID(as_uuid=True), ForeignKey("documents.id",
ondelete="SET NULL"), nullable=True)
    received_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)

class ShipmentIssue(Base):
    __tablename__ = "shipment_issues"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    shipment_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("shipments.id", ondelete="CASCADE"), nullable=False,
index=True)
    issue_type: Mapped[str] = mapped_column(String(20), nullable=False)
# quality|damage|detention|other
    description: Mapped[str] = mapped_column(Text, nullable=False)
    document_id: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("documents.id",
ondelete="SET NULL"), nullable=True)
    logged_by: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("users.id", ondelete="SET NULL"), nullable=True)
    logged_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()")
    resolved_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)
```

9.4 — Governance: Rule Evaluations, Exceptions, Approvals

```

class RuleEvaluation(Base):
    __tablename__ = "rule_evaluations"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    transaction_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("trade_transactions.id", ondelete="CASCADE"), nullable=False,
index=True)
    rule_id: Mapped[str] = mapped_column(String(10), nullable=False,
index=True) # BR-01 .. BR-13
    passed: Mapped[bool] = mapped_column(Boolean, nullable=False)
    field_name: Mapped[str | None] = mapped_column(String(100),
nullable=True)
    expected_value: Mapped[str | None] = mapped_column(String(255),
nullable=True)
    actual_value: Mapped[str | None] = mapped_column(String(255),
nullable=True)
    message: Mapped[str] = mapped_column(Text, nullable=False)
    evaluated_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), nullable=False,
server_default="now()")
    __table_args__ = (Index("ix_rule_eval_txn_rule", "transaction_id",
"rule_id"),)

class ExceptionCase(Base):
    __tablename__ = "exception_cases"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    transaction_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("trade_transactions.id", ondelete="CASCADE"), nullable=False,
index=True)
    exception_type: Mapped[str] = mapped_column(String(50),
nullable=False, index=True) # 10 categories, BRD $7.2
    rule_id: Mapped[str | None] = mapped_column(String(10),
nullable=True)
    owner_role: Mapped[str] = mapped_column(String(30), nullable=False,
index=True)
    priority: Mapped[str] = mapped_column(String(10), nullable=False,
default="medium")
    opened_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()", index=True)
    resolved_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)
    resolution_note: Mapped[str | None] = mapped_column(Text,
nullable=True)
    escalated: Mapped[bool] = mapped_column(Boolean, nullable=False,
default=False)
    __table_args__ = (Index("ix_exc_open_owner", "owner_role",
"resolved_at"),)

class ApprovalTask(Base):
    __tablename__ = "approval_tasks"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    transaction_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("trade_transactions.id", ondelete="CASCADE"), nullable=False,
index=True)
    approver_role: Mapped[str] = mapped_column(String(30),
nullable=False, index=True)
    assigned_to: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("users.id", ondelete="SET

```

```
NULL"), nullable=True)
    requested_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), nullable=False,
server_default="now()", index=True)
    decision: Mapped[str] = mapped_column(String(20), nullable=False,
default="pending", index=True)
    decided_by: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("users.id", ondelete="SET
NULL"), nullable=True)
    decided_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)
    reason: Mapped[str | None] = mapped_column(Text, nullable=True)
    ai_summary: Mapped[str | None] = mapped_column(Text, nullable=True)
    __table_args__ = (Index("ix_approval_pending", "decision",
"requested_at"),)
```

9.5 — Integration & Generated Documents

```

class IntegrationJob(Base):
    __tablename__ = "integration_jobs"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    transaction_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("trade_transactions.id", ondelete="CASCADE"), nullable=False,
index=True)
    target_system: Mapped[str] = mapped_column(String(10),
nullable=False, index=True) # tracker|sap|dms
    status: Mapped[str] = mapped_column(String(20), nullable=False,
default="queued", index=True)
    external_reference: Mapped[str | None] = mapped_column(String(255),
nullable=True)
    failure_reason: Mapped[str | None] = mapped_column(Text,
nullable=True)
    attempt_count: Mapped[int] = mapped_column(Integer, nullable=False,
default=0)
    last_attempted_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()")
    __table_args__ = (UniqueConstraint("transaction_id", "target_system",
name="uq_txn_target"),)

class DocumentPack(Base):
    __tablename__ = "document_packs"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    transaction_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("trade_transactions.id", ondelete="CASCADE"), nullable=False,
index=True)
    pack_type: Mapped[str] = mapped_column(String(30), nullable=False) #
purchase_file|sales_bank_docs|draft_contract|draft_invoice
    filename: Mapped[str] = mapped_column(String(255), nullable=False) #
AGFZE naming convention, e.g. S0-Batch-Qty-Final.pdf
    storage_ref: Mapped[str] = mapped_column(String(512), nullable=False)
    generated_by: Mapped[str] = mapped_column(String(10), nullable=False,
default="system") # system|user
    generated_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), nullable=False,
server_default="now()")
    dms_document_id: Mapped[str | None] = mapped_column(String(255),
nullable=True)
    dms_uploaded_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)

class AuditEvent(Base):
    __tablename__ = "audit_events"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    occurred_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), nullable=False,
server_default="now()", index=True)
    actor_id: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("users.id", ondelete="SET
NULL"), nullable=True)
    actor_type: Mapped[str] = mapped_column(String(10), nullable=False,
default="user") # user|system|agent
    event_type: Mapped[str] = mapped_column(String(50), nullable=False,
index=True) # BR-12: every action, approval, exception, downstream

```



```
update
    entity_type: Mapped[str] = mapped_column(String(30), nullable=False,
index=True)
    entity_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
nullable=False, index=True)
    event_metadata: Mapped[dict] = mapped_column(JSONB, nullable=False,
default=dict) # never full document/AI-prompt text – Section 16.8
    request_id: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("requests.id", ondelete="SET
NULL"), nullable=True)
    __table_args__ = (Index("ix_audit_entity", "entity_type",
"entity_id"),)
    # append-only by convention: no UPDATE/DELETE route is ever exposed
on this table (Section 16.8)

class BackgroundJob(Base):
    __tablename__ = "background_jobs"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    job_type: Mapped[str] = mapped_column(String(50), nullable=False,
index=True)
    status: Mapped[str] = mapped_column(String(20), nullable=False,
default="queued", index=True)
    progress: Mapped[int] = mapped_column(Integer, nullable=False,
default=0)
    result_url: Mapped[str | None] = mapped_column(String(512),
nullable=True)
    error_message: Mapped[str | None] = mapped_column(Text,
nullable=True)
    transaction_id: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("trade_transactions.id",
ondelete="CASCADE"), nullable=True, index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()")
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()", onupdate=datetime.utcnow)
```

9.6 — Configuration & Reference Data

```

class CommodityCode(Base):
    __tablename__ = "commodity_codes"
    code: Mapped[str] = mapped_column(String(16), primary_key=True) #
AL, CU, CUZNS, MIX, HMS, ...
    name: Mapped[str] = mapped_column(String(100), nullable=False)
    active: Mapped[bool] = mapped_column(Boolean, nullable=False,
default=True)

class RuleConfiguration(Base):
    __tablename__ = "rule_configurations"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    rule_id: Mapped[str] = mapped_column(String(10), nullable=False,
index=True)
    scope: Mapped[dict] = mapped_column(JSONB, nullable=False,
default=dict) # {commodity_code, territory, transaction_type}
    threshold_value: Mapped[Decimal | None] = mapped_column(Numeric(10,
4), nullable=True)
    active: Mapped[bool] = mapped_column(Boolean, nullable=False,
default=True)
    updated_by: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("users.id", ondelete="SET
NULL"), nullable=True)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()")
    change_reason: Mapped[str] = mapped_column(Text, nullable=False)

class DocumentTypeSchema(Base):
    __tablename__ = "document_type_schemas"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    document_type: Mapped[str] = mapped_column(String(30),
nullable=False, index=True)
    territory: Mapped[str | None] = mapped_column(String(20),
nullable=True)
    field_schema: Mapped[dict] = mapped_column(JSONB, nullable=False) #
[{name, type, required, tolerance}]
    mandatory_documents: Mapped[dict] = mapped_column(JSONB,
nullable=False, default=dict)
    updated_by: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("users.id", ondelete="SET
NULL"), nullable=True)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()")
    change_reason: Mapped[str] = mapped_column(Text, nullable=False)
    __table_args__ = (UniqueConstraint("document_type", "territory",
name="uq_doctype_territory"),)

class Report(Base):
    __tablename__ = "reports"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    report_type: Mapped[str] = mapped_column(String(10), nullable=False)
# daily|monthly|adhoc
    format: Mapped[str] = mapped_column(String(10), nullable=False) #
pdf|xlsx
    date_from: Mapped[date] = mapped_column(Date, nullable=False)
    date_to: Mapped[date] = mapped_column(Date, nullable=False)
    business_stream: Mapped[str] = mapped_column(String(10),
nullable=False, default="both")

```

```

storage_ref: Mapped[str] = mapped_column(String(512), nullable=False)
generation_ref: Mapped[str] = mapped_column(String(64), unique=True,
nullable=False) # FR-RP-04 traceability
generated_by: Mapped[uuid.UUID | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("users.id", ondelete="SET
NULL"), nullable=True)
generated_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), nullable=False,
server_default="now()")

class Notification(Base):
    __tablename__ = "notifications"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    user_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("users.id", ondelete="CASCADE"), nullable=False, index=True)
    type: Mapped[str] = mapped_column(String(30), nullable=False)
    message: Mapped[str] = mapped_column(Text, nullable=False)
    deep_link: Mapped[str | None] = mapped_column(String(255),
nullable=True)
    read: Mapped[bool] = mapped_column(Boolean, nullable=False,
default=False, index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()")

```

9.7 — Graph Data Layer SOURCE — REQUIRED, NOT IN REFERENCE STACK

TDD §10.1/10.3 requires a graph store as the source of truth for cross-entity relationships and traceability queries (“show me everything connected to this container,” “walk from this SAP posting back to the original email”) that are expensive to express as recursive SQL joins across 24 tables. This is implemented as a managed graph database (Neo4j AuraDB, or self-hosted Neo4j Community on the same GCP project) synchronised from the relational store via a change-data-capture worker — the relational schema above remains the transactional system of record; the graph is a derived, rebuildable read-optimised projection, never an independent source of truth for a value.

Node label	Backed by
EmailMessage	email_messages
Document	documents
TradeTransaction	trade_transactions (+legs)
Container	containers
Shipment	shipments
Supplier / Customer	derived from purchase_legs.supplier_name / sales_legs.customer_name
ApprovalTask / ExceptionCase	approval_tasks / exception_cases
IntegrationJob / DocumentPack	integration_jobs / document_packs

Relationship	Meaning
(EmailMessage) - [:HAS_ATTACHMENT] -> (Document)	Intake linkage
(Document) - [:EXTRACTED_AS] -> (ExtractionResult)	Extraction linkage
(TradeTransaction) - [:HAS_PURCHASE_LEG] -> (PurchaseLeg)	Purchase side
(TradeTransaction) - [:HAS_SALES_LEG] -> (SalesLeg)	Sales side
(PurchaseLeg) - [:ISSUED_BY] -> (Supplier)	Counterparty
(SalesLeg) - [:SOLD_TO] -> (Customer)	Counterparty
(TradeTransaction) - [:HAS_CONTAINER] -> (Container) - [:SHIPPED_UNDER] -> (BillOfLading)	Cargo linkage
(TradeTransaction) - [:REQUIRES] -> (ApprovalTask) / [:HAS_EXCEPTION] -> (ExceptionCase)	Governance linkage
(IntegrationJob) - [:COMMITTED_TO] -> (TargetSystem)	Downstream linkage

Exposed to the frontend only through `GET /api/v1/transactions/{id}/graph` (Section 12.4), which returns a bounded, pre-rendered subgraph (never an open Cypher endpoint) for the “trace this transaction” view inside the Transaction Workspace.

9.8 — Push Subscriptions SPECIFIED – added scope, Feature 12

```
class PushSubscription(Base):
    __tablename__ = "push_subscriptions"
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=uuid.uuid4)
    user_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
ForeignKey("users.id", ondelete="CASCADE"), nullable=False, index=True)
    endpoint: Mapped[str] = mapped_column(String(1024), nullable=False)
    p256dh_key: Mapped[str] = mapped_column(String(255), nullable=False)
    auth_key: Mapped[str] = mapped_column(String(255), nullable=False)
    user_agent: Mapped[str | None] = mapped_column(String(255),
nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=False, server_default="now()")
    last_used_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)
    __table_args__ = (UniqueConstraint("user_id", "endpoint",
name="uq_user_endpoint"),)
```

The existing `notifications` table (Section 9.6) is extended with two columns to record delivery outcome per channel: `email_sent_at: datetime | None` and `push_sent_at: datetime | None` — added via an Alembic migration rather than a new table, since a notification’s delivery channel is an attribute of the notification, not a separate entity.

SECTION 10

10 Backend Structure (FastAPI)

10.1 — Project Folder Structure

```

backend/ |— app/ | |— init.py | |— main.py # App
factory, middleware registration, router inclusion, Sentry init | |—
config.py # Pydantic Settings: DevelopmentConfig / TestingConfig /
ProductionConfig | |— dependencies.py # get_db, get_current_user,
get_current_admin_user, get_storage_service, get_ai_service | | | |—
models/ # SQLAlchemy 2.0 models – one file per Section 9 group | | | |—
base.py # DeclarativeBase | | | |— identity.py # User | | | |— intake.py #
EmailMessage, Request, Document, ExtractedField | | | |— transaction.py #
TradeTransaction, PurchaseLeg, SalesLeg, FaLeg | | | |— logistics.py #
Container, Shipment, BillOfLading, ShipmentIssue | | | |— governance.py #
RuleEvaluation, ExceptionCase, ApprovalTask | | | |— integration.py #
IntegrationJob, DocumentPack | | | |— audit.py # AuditEvent,
BackgroundJob | | | |— config_ref.py # CommodityCode, RuleConfiguration,
DocumentTypeSchema, Report, Notification | | | |— schemas/ # Pydantic
request/response models, one file per router | | | |— requests.py |—
documents.py |— transactions.py |— exceptions.py | | | |— approvals.py
|— integrations.py |— shipments.py |— dashboards.py | | | |—
reports.py |— admin.py |— audit.py |— common.py # envelope,
pagination | | | |— routers/ | | | |— requests.py |— documents.py |—
transactions.py |— exceptions.py | | | |— approvals.py |—
integrations.py |— shipments.py |— dashboards.py | | | |— reports.py
|— admin.py |— audit.py |— users.py | | | |— notifications.py |—
jobs.py |— health.py | | | |— services/ | | | |— gemini_service.py #
ALL Gemini/Vertex AI calls – Section 13.1/13.2 | | | |—
classification_service.py | | | |— extraction_service.py | | | |—
matching_service.py # rapidfuzz-based matching – Section 13.7 | | | |—
validation_service.py # Business Rule Engine – Feature 6 | | | |—
document_service.py # Orchestrates classify → extract → match → validate
| | | |— docx_service.py # python-docx contract/invoice generation | | | |—
|— pdf_service.py # PyMuPDF extraction + PDF report rendering | | | |—
excel_service.py # pandas/openpyxl tracker parsing & export | | | |—
storage_service.py # Local filesystem (dev) / Azure Blob (prod)
abstraction | | | |— graph_service.py # Microsoft Graph: mail,
SharePoint/Excel – Section 13.6 [SOURCE] | | | |— sap_service.py # SAP
OData/BAPI client, or Power-Automate-assisted fallback [SOURCE] | | | |—
dms_service.py # DMS REST API adapter [SOURCE] | | | |—
shipping_service.py # Carrier API/website adapters [SOURCE] | | | |—
neo4j_service.py # Graph data-layer sync & bounded subgraph queries
[SOURCE] | | | |— keycloak_service.py # Keycloak Admin API – role
assignment | | | |— job_service.py # Background job bookkeeping
(background_jobs table) | | | |— notification_service.py # Fan-out:
decides email vs push vs in-app per event + user preference | | | |—
email_service.py # SMTP send via smtplib + Jinja2-rendered HTML
templates [SPECIFIED – Feature 12] | | | |— push_service.py # Web Push
via pywebpush + VAPID keys [SPECIFIED – Feature 12] | | | |—
email_templates/ # HTML + plaintext-fallback Jinja2 templates
(server-side only – | | | |# unrelated to the frontend-templating
prohibition, Document Control) | | | |— base_layout.html
approval_requested.html exception_assigned.html | | | |—
report_ready.html missing_document.html and one file per notification
type | | | |— tasks/ | | | |— email_intake_tasks.py # Mailbox polling +
webhook handler (Feature 1) | | | |— extraction_tasks.py # Feature 2
background path | | | |— integration_tasks.py # Feature 8
per-target-system jobs | | | |— shipment_tasks.py # Feature 9 scheduled
status pulls | | | |— report_tasks.py # Feature 11 scheduled + ad-hoc

```



```

generation | | └─ maintenance_tasks.py # Ageing/escalation recompute,
Graph webhook renewal | | | └─ middleware/ | | | └─ cors.py └─ audit.py
└─ rate_limit.py └─ auth.py | | | └─ utils/ | | | └─ file_utils.py └─
security_utils.py └─ formatting_utils.py └─ logging_utils.py | | | └─
db/ | └─ session.py # Async engine + session factory | └─ init_db.py |
└─ alembic/{env.py, versions/, alembic.ini} └─ tests/{conftest.py,
test_*.py} └─ uploads/ generated/ # local dev storage roots └─
Dockerfile docker-compose.yml Makefile deploy.sh └─ requirements.in
requirements.txt requirements.lock └─ .env .env.example .gitignore
pyproject.toml pytest.ini

```

10.2 — FastAPI Application Factory

`main.py` builds the app via a factory function so tests can construct a fresh instance per test module: `create_app(settings: Settings) -> FastAPI`. Startup: opens the async SQLAlchemy engine/connection pool, initialises Sentry (`sentry_sdk.init(dsn=..., traces_sample_rate=0.1, environment=settings.ENV)`), registers the Graph webhook renewal scheduler, and validates that every required secret (Section 16.9) is present, failing fast rather than serving traffic with a missing credential. Shutdown: disposes the engine's connection pool cleanly. Middleware is registered in a fixed order — CORS first, then audit logging, then slowapi rate limiting, then JWT auth dependency injection at the router level (not global middleware, so `/health` stays unauthenticated). Routers are included with the `/api/v1` prefix and their own tag for Swagger UI grouping.

10.3 — Configuration Management

`config.py` defines `Settings(BaseSettings)` with `DevelopmentConfig`, `TestingConfig`, `ProductionConfig` subclasses selected by an `ENV` environment variable. Key variables: `DATABASE_URL`, `RATE_LIMIT_ENABLED` (False in dev/test, True in prod — Section 16.3), `AI_PROVIDER` (Section 13.1), `STORAGE_BACKEND` (`local` in dev, `azure_blob` in prod), `KEYCLOAK_ISSUER`, `KEYCLOAK_JWKS_URL`, `AZURE_AD_TENANT_ID` / `CLIENT_ID` / `CLIENT_SECRET` (Graph API app-only auth), `GRAPH_MAILBOX_ADDRESS`, `SAP_API_BASE_URL` (nullable — falls back to the PowerAutomate-assisted path when unset), `DMS_API_BASE_URL`, `NEO4J_URI` / `USER` / `PASSWORD`, `SENTRY_DSN`, `CONFIDENCE_THRESHOLD_DEFAULT` (0.75).

10.4 — Router Architecture

Router	Prefix	Contains
<code>requests.py</code>	<code>/requests</code>	Feature 1 endpoints
<code>documents.py</code>	<code>/documents</code>	Feature 2 endpoints
<code>transactions.py</code>	<code>/transactions</code>	Features 3/4/5 endpoints
<code>exceptions.py</code> / <code>approvals.py</code>	<code>/exceptions</code> , <code>/approvals</code>	Feature 7 endpoints
<code>integrations.py</code>	<code>/integrations/jobs</code>	Feature 8 endpoints

shipments.py	/shipments	Feature 9 endpoints
dashboards.py / reports.py	/dashboards, /reports	Features 10/11 endpoints
admin.py / audit.py	/admin, /audit	Configuration & compliance endpoints
users.py / notifications.py / jobs.py / health.py	/users, /notifications, /jobs, /health	Cross-cutting endpoints

10.5 — Dependency Injection

- `get_db()` — yields an `AsyncSession` from the pooled engine, always closed in a `finally` block.
- `get_current_user()` — reads the `Authorization: Bearer` header, verifies the JWT's RSA signature against Keycloak's JWKS endpoint via `python-jose`, checks `exp / iss / aud`, and loads/creates the local `User` row from the token's `sub` / roles claims (just-in-time provisioning on first login).
- `get_current_admin_user()` — wraps `get_current_user`, raises `403` if `"admin"` is not in the user's roles.
- `get_storage_service()` — returns the local-filesystem or Azure Blob implementation per `STORAGE_BACKEND` (Section 13.9).
- `get_ai_service()` — returns the Gemini-Flash/Vertex-AI-backed implementation per `AI_PROVIDER` (Section 13.1).

10.6 — Background Job Integration

Every feature that needs deferred work creates a `BackgroundJob` row first (status `queued`), then either (a) schedules an `async def` FastAPI `BackgroundTasks` callable for sub-second/low-latency work (single-document extraction, draft generation), or (b) triggers a Cloud Run Job for multi-document packs, scheduled shipment pulls, and report generation. Both paths update the same row's `status / progress / result_url / error_message` fields, so the frontend's single `GET /jobs/{job_id}/status` polling convention (Section 12.7) never needs to know which execution path handled a given job.

10.7 — Async Architecture

Every route handler is `async def`; all database access uses SQLAlchemy 2.0 async sessions over `asyncpg`; all external HTTP calls (Gemini, Keycloak JWKS fetch/ cache, Microsoft Graph, SAP, DMS, carrier APIs) use `httpx.AsyncClient` with connection pooling and explicit timeouts. CPU-bound work — PyMuPDF page rasterisation, pandas tracker parsing, rapidfuzz batch matching — is offloaded via `anyio.to_thread.run_sync()` so it never blocks the event loop, even when it isn't large enough to warrant a full Cloud Run Job.

SECTION 11

11 Frontend Structure (Next.js)

11.1 — Project Folder Structure

```

frontend/ └─ app/ │ └─ layout.tsx page.tsx loading.tsx error.tsx
not-found.tsx │ └─ offline/page.tsx # Service-worker navigation
fallback (Feature 12) │ └─ disclaimer/page.tsx privacy/page.tsx
terms/page.tsx │ └─ (auth)/signin/page.tsx (auth)/signout/page.tsx │
│ └─ (protected)/ │ │ └─ layout.tsx # Sidebar + header shell, session
guard │ │ └─ dashboard/page.tsx │ │ └─ inbox/page.tsx
inbox/upload/page.tsx │ │ └─ transactions/ │ │ │ └─ page.tsx
new/page.tsx │ │ │ └─ purchase/[id]/page.tsx │ │ │ └─
sales/[id]/page.tsx │ │ │ └─ fa/[id]/page.tsx │ │ └─
documents/page.tsx documents/[id]/page.tsx │ │ └─ exceptions/page.tsx
exceptions/[id]/page.tsx │ │ └─ approvals/page.tsx
approvals/[id]/page.tsx │ │ └─ shipments/page.tsx
shipments/[id]/page.tsx │ │ └─ analytics/page.tsx │ │ └─
reports/page.tsx reports/builder/page.tsx reports/[id]/page.tsx │ │ └─
admin/ │ │ │ └─ page.tsx users/page.tsx rules/page.tsx │ │ │ └─
document-types/page.tsx integrations/page.tsx audit/page.tsx │ │ └─
settings/page.tsx notifications/page.tsx │ │ └─ components/ │ └─ ui/ #
shadcn/ui primitives (Button, Card, Input, Select, Dialog, Sheet, Tabs,
Accordion, Table, Badge, Progress, Skeleton, Command ...) │ └─ layout/ #
Navbar.tsx, Sidebar.tsx, Footer.tsx, MobileNav.tsx │ └─ features/ #
DocumentSplitView.tsx, ExtractionFieldRow.tsx, MatchingPanel.tsx, │ │ #
ValidationChecklist.tsx, ExceptionActionPanel.tsx,
ApprovalDecisionForm.tsx, │ │ # IntegrationStatusPanel.tsx,
ShipmentCard.tsx, DraftPreview.tsx, │ │ # KpiTile.tsx,
AIDisclaimerBanner.tsx, JobProgressBar.tsx │ └─ shared/ #
LoadingSkeletons.tsx, EmptyState.tsx, ErrorDisplay.tsx,
ConfidenceBadge.tsx │ # InstallPWAButton.tsx, OfflineBanner.tsx,
NotificationPermissionPrompt.tsx [Feature 12] │ └─ lib/ │ └─ api.ts #
Fetch API wrapper, JWT injection, typed per-endpoint functions │ └─
auth.ts # NextAuth.js config – Keycloak OIDC provider │ └─ push.ts #
PushManager subscribe/unsubscribe helpers [Feature 12] │ └─ utils.ts
constants.ts validators.ts # Zod schemas mirroring Section 9/12 Pydantic
models │ └─ hooks/ │ └─ useJobStatus.ts useTransaction.ts
useExceptionQueue.ts useApprovalQueue.ts │ └─ useOnlineStatus.ts
usePwaInstall.ts # Feature 12 │ └─ types/api.ts types/auth.ts │
styles/globals.css │ └─ public/ │ └─ icons/ images/ │ └─
manifest.webmanifest # PWA
name/icons/theme-colour/start_url/display:standalone │ └─ sw.js #
Workbox-built service worker (precache + runtime caching) │ └─
icons/maskable-512.png icons/apple-touch-icon.png │ └─ middleware.ts
next.config.ts tailwind.config.ts tsconfig.json postcss.config.js │
package.json package-lock.json Dockerfile .eslintrc.json .prettierrc
.env.local

```

11.2 — NextAuth.js Configuration

`lib/auth.ts` configures a single Keycloak OIDC provider (`KEYCLOAK_CLIENT_ID` , `KEYCLOAK_CLIENT_SECRET` , `KEYCLOAK_ISSUER`); Keycloak itself is configured (server-side, outside this codebase) with Microsoft Entra ID as an upstream identity broker, so this is the only provider entry the frontend needs (Section 16.1). The `jwt` callback persists the access token, refresh token and expiry on the NextAuth JWT, and silently refreshes the access token against Keycloak's token endpoint when it is within 60 seconds of expiry. The `session` callback exposes `roles` (from the Keycloak-issued token's realm/client roles claim) to `useSession()` so client components can conditionally render role-gated UI, while every actual authorization decision is re-checked server-side (Section 16.2) rather than trusted from this claim alone. `middleware.ts` protects every route under `(protected)` , redirecting unauthenticated requests to `/signin` with a `callbackUrl` .

11.3 — API Client Architecture

`lib/api.ts` reads `NEXT_PUBLIC_API_BASE_URL` , attaches `Authorization: Bearer {session.accessToken}` to every request via a thin wrapper around `fetch` , and exposes one typed function per backend endpoint in Section 12 (e.g. `getTransaction(id)` , `submitTransaction(id)` , `decideApproval(id, decision)`) returning the shared response envelope (Section 12.3). A response interceptor normalises the envelope's `errors` array into field-level Zod-compatible error objects for form display, and a `401` triggers a NextAuth sign-in redirect rather than surfacing a raw error. File uploads (`/documents/upload`) build a `FormData` object directly rather than JSON-encoding file content.

11.4 — Component Architecture

`components/ui/` holds unmodified shadcn/ui primitives; `components/layout/` holds the app shell; `components/features/` holds the composite, data-aware components unique to this platform (the extraction split-view, the validation checklist, the approval decision form) that appear across Section 7/8's page catalog; `components/shared/` holds the loading/empty/error primitives used platform-wide for consistency (Section 14.7/14.8).

11.5 — State Management

Server Components fetch initial page data directly (transaction list, dashboard summary) to minimise client-side waterfalls; interactive workspaces (Purchase/Sales/FA Transaction Workspace, Document Review) are Client Components using `useState` / `useReducer` for in-progress edits, with a custom `useJobStatus(jobId)` hook polling `/jobs/{id}/status` at a 2-second interval with exponential backoff up to 10 seconds, stopping on `completed` / `failed` . `useSession()` from NextAuth.js supplies auth state platform-wide. No external state library (Redux/Zustand) is introduced — nothing in this application's data shape needs cross-route client state beyond what the URL, server components and the job-polling hooks already provide.

11.6 — Styling Architecture

Tailwind CSS utility classes are the primary styling mechanism; Radix UI primitives (via shadcn/ui) supply accessible unstyled behaviour for every interactive component (Dialog, Sheet, Tabs, Accordion, Select, DropdownMenu, Command); `globals.css` defines the CSS-variable-based theme (Section 14.9) consumed by shadcn's Tailwind preset; Framer Motion drives the first-login walkthrough callouts (Section 5, Stage 2), accordion expand/collapse easing, and toast enter/exit — deliberately not used for anything that would slow down a dense data workspace, per Section 14.2's anxiety-reduction principle (smooth, not showy).

11.7 — PWA & Service Worker SPECIFIED – added scope, Feature 12

`public/manifest.webmanifest` declares the app name, icon set (including a maskable icon for Android's adaptive-icon system and an Apple-touch-icon for iOS home-screen install), `theme_color` / `background_color` matching the ink-navy/copper palette (Section 14.9), and `display: "standalone"` so the installed app opens without browser chrome. The service worker (`public/sw.js`) is generated at build time with Workbox's `injectManifest` mode (giving full control over runtime-caching rules rather than relying on a framework plugin's defaults) and registered from the root layout on mount, behind a check for `'serviceWorker' in navigator` so unsupported browsers degrade silently to the plain web app.

`InstallPWAButton.tsx` listens for `beforeinstallprompt`, stores the event, and calls `prompt()` on click (Chromium/Android/Windows/macOS-desktop browsers); on iOS Safari, which never fires that event, the same button opens an instructional `Dialog` instead. Full caching-strategy detail is in Section 13.11.

SECTION 12

12 API Contract

12.1 — API Versioning

Every endpoint is prefixed `/api/v1/`. A breaking change to any response schema ships as `/api/v2/...` behind a new router set, with `v1` kept live until the Next.js frontend has migrated — the frontend and backend deploy independently (Cloud Run services), so version skew between them must never break in production.

12.2 — Authentication Header

Every authenticated request carries `Authorization: Bearer <jwt>`. The backend's `get_current_user` dependency (Section 10.5) verifies the signature against Keycloak's JWKS endpoint (cached in-process, refreshed on a `kid` cache-miss to survive Keycloak key rotation) via `python-jose`, and rejects tokens with the wrong `iss` / `aud` or an expired `exp` with `401`.

12.3 — Request/Response Format

JSON everywhere except file upload (`multipart/form-data`) and file download (raw bytes with a `Content-Disposition` header). Standard envelope:

```
{
  "success": true,
  "data": { ... },
  "message": "Operation completed successfully",
  "errors": null
}
```

```
{
  "success": false,
  "data": null,
  "message": "Validation error",
  "errors": [
    { "field": "quantity_mt", "message": "Must be within 5% of the
contracted 125.000 MT" }
  ]
}
```

12.4 — Complete Endpoint Catalog

See Section 7, Part B for the full catalog — every endpoint's HTTP method/path, request/response Pydantic schema, error responses, authentication and rate-limit status, and background-job/AI involvement is specified there rather than repeated here, to keep the two sections consistent by construction rather than by cross-checking two separate lists.

12.5 — Pagination Convention

List endpoints accept `page:int=1`, `page_size:int=25` (max 100), `sort_by:str`, `sort_order:Literal["asc","desc"]="desc"`, `search:str|None`, plus endpoint-specific filters (Section 7B). Response: `{items, total, page, page_size, total_pages}` — `total` is computed with a single `COUNT(*) OVER()` window function alongside the page query, not a separate round trip.

12.6 — File Upload Convention

Frontend builds a `FormData` object (Section 11.3); backend validates content-type by magic bytes, enforces the 25MB size limit before reading the full body into memory (streamed check), stores via `storage_service` under a UUID-derived key, and returns `{id, filename, storage_ref}` — the raw `storage_ref` is never returned to the frontend as a directly fetchable URL; downloads always go through an authenticated endpoint that resolves a short-lived signed URL server-side (Section 13.9/13.10).

12.7 — Job Status Polling Convention

Every long-running operation across every feature returns `{job_id}` immediately (`202 Accepted` semantics via a `200` with `JobAcceptedOut`, to keep client handling uniform). The frontend's `useJobStatus` hook (Section 11.5) polls `GET /api/v1/jobs/{job_id}/status` starting at 2s intervals, backing off to 10s after 30s of no change, and stops polling on `completed` or `failed`. `result_url` is populated only on `completed` and, like any file reference, is an authenticated download endpoint rather than a raw storage URL.

SECTION 13

13 Feature Logic

13.1 — AI Provider Switching Architecture

Every AI call in the platform goes through `gemini_service.py` — no router or task module ever imports a model SDK directly. Each capability is a typed function whose signature is stable across providers; only the private `call*` implementation differs.

```
import os
from google import genai

AI_PROVIDER = os.getenv("AI_PROVIDER", "gemini_flash") # gemini_flash |
vertex_ai | enterprise_agent

async def classify_email(subject: str, body: str) ->
ClassificationResult: ...
async def extract_document(doc_bytes: bytes, mime_type: str, schema:
dict) -> ExtractionResult: ...
async def generate_draft_content(transaction: dict, template_meta: dict)
-> DraftContent: ...
async def summarize_for_approval(transaction: dict) -> str: ...
async def parse_shipment_text(raw_status_text: str) ->
ShipmentMilestone: ...

async def _route(capability: str, *args, **kwargs):
    if AI_PROVIDER == "gemini_flash":
        return await _call_gemini_flash(capability, *args, **kwargs)
    elif AI_PROVIDER == "vertex_ai":
        return await _call_vertex_ai(capability, *args, **kwargs)
    elif AI_PROVIDER == "enterprise_agent":
        return await _call_enterprise_agent(capability, *args, **kwargs)
    raise ValueError(f"Unknown AI_PROVIDER: {AI_PROVIDER}")

async def _call_gemini_flash(capability: str, *args, **kwargs) -> dict:
    client = genai.Client()
    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=_build_prompt(capability, *args, **kwargs),
        config=genai.types.GenerateContentConfig(response_mime_type="application/
json"),
    )
    return _parse_and_validate(capability, response.text)
```

Every provider path returns the same Pydantic-validated result type; a provider that cannot fulfil a call (quota, timeout, malformed JSON) raises a typed `AIServiceError` that callers translate into a *Needs Review* state rather than a 500 — an AI failure degrades to “a human looks at this,” never to a silent skip or a hard crash (consistent with Section 2.3’s governance philosophy).

13.2 — Gemini Prompting Strategy

- **System prompt design.** Each capability has a fixed system prompt naming the domain precisely (“You are extracting structured fields from a scrap-metal trade document for AGFZE. Extract only what is explicitly stated; never infer a number that is not present.”) plus the JSON schema it must return, generated programmatically from the same Pydantic/JSONB schema stored in `document_type_schemas` (Section 9.6) — so a schema change in the admin UI changes the prompt automatically, with no separate prompt-template edit.
- **Structured output.** Every call sets `response_mime_type="application/json"` and includes an explicit example of the target shape; the response is parsed and validated against the matching Pydantic model before it touches the database — a response that fails validation is treated as a low-confidence extraction, not silently coerced.
- **Confidence/uncertainty.** The schema requires a `confidence: float (0-1)` and, where relevant, a one-sentence `rationale` per field; the prompt explicitly instructs the model to return a low confidence score rather than guess when a field is illegible or absent, since BR-11 depends on that score being meaningful, not decorative.
- **Multilingual input.** China-territory documents may include Mandarin text (e.g. a certificate of origin); Gemini is prompted to extract the requested fields in their original script *and* a normalised English/ASCII value where the field is a structured reference (a contract number, a quantity), so downstream matching (Section 13.7) always has a comparable value regardless of source-document language.
- **Safe injection of user text.** Email body/attachment text is passed as a clearly delimited data block (not concatenated into the instruction text), and the system prompt explicitly states that content inside the data block is data to extract from, never an instruction to follow — a defence against a malicious or malformed email attempting prompt injection against the extraction agent.
- **Chunking for long documents.** Multi-page scanned packs are processed per page (or per 4–6 page window for text-layer PDFs) with a consolidation pass that reconciles field values seen in multiple chunks by preferring the highest-confidence occurrence and flagging genuine conflicts for review rather than silently picking one.
- **Disclaimer instruction.** Every extraction/generation prompt’s response schema includes a fixed `disclaimer` field the frontend renders verbatim (Section 2.5) — the disclaimer text itself is not generated by the model, only echoed, so its wording can never drift.

13.3 — Document Processing Pipeline

1. File-type detection via magic bytes (`python-magic`) at upload/attachment ingest — never trusts the filename extension.
2. Text-layer PDF → PyMuPDF (`fitz`) extracts text with layout/position metadata, which also supplies the `source_page` / `source_ref` values stored per field (Section 9.1).

3. Scanned/image PDF or photo → each page is rasterised (PyMuPDF renders the page to an image) and sent to Gemini 2.5 Flash's multimodal endpoint for combined OCR + field extraction in one call, rather than a separate OCR pass followed by a second text-understanding pass.
4. DOCX → python-docx extracts paragraph/table text (used for the sample contract templates reviewed during discovery).
5. XLSX/XLS/CSV → pandas/openpyxl parse against the known tracker column layout (Loading Sheet, Sales Contract, Cost Sheet structures reviewed during discovery, Section 9) with header-row detection tolerant of the merged-cell/blank-row patterns real AGFZE trackers use.
6. Extracted text/structured data is handed to `gemini_service.extract_document(...)` with the schema selected by `(document_type, territory)` from `document_type_schemas`.
7. The returned `ExtractionResult` (fields + confidence + source_ref, Section 13.2) is validated against the Pydantic model for that document type and persisted as `ExtractedField` rows (Section 9.1).
8. Fields below `CONFIDENCE_THRESHOLD_DEFAULT` (0.75) are marked `needs_review` and surfaced first in the Document Review UI (Section 8, Archetype 1).
9. On user confirmation (`POST /documents/{id}/confirm`), the document transitions to `Extracted` and the Matching Agent (Feature 3/4/5, Section 13.7) is invoked.
10. Results are returned to the Next.js frontend via the standard envelope (Section 12.3) and rendered in the split-view (Section 8).

13.4 — Document Generation Logic

1. User completes/confirms structured transaction data in the Sales Transaction Workspace (Zod-validated client-side, Section 8).
2. Form data reaches `POST /transactions/{id}/generate-draft` (Pydantic-validated server-side, Section 7B).
3. A `BackgroundJob` row is created (Section 9.5); the task calls `gemini_service.generate_draft_content(...)`, which returns structured JSON: which template clauses to populate verbatim, which clause to mark for revision (e.g. the LME% clause becoming a fixed-price clause once the customer has fixed, Section 4 Feature 4), and which clause to remove.
4. The task hands that structured content to `docx_service.render_contract(template_id, content)` (or `pdf_service` for a PDF-only output), which populates the approved DOCX template using python-docx — the binary file is always produced deterministically from a template, never by asking Gemini to emit a DOCX/PDF byte stream.
5. The generated file is saved via `storage_service` under a UUID-derived key with a human-readable download filename following AGFZE's existing convention (Section 9.5's `DocumentPack.filename`).

6. A `DocumentPack` row is created, linked to the transaction and to the source evidence that justified each populated field.
7. The job's `status` is set to `completed` with `result_url` pointing at the authenticated download endpoint.
8. Frontend polling (Section 12.7) detects completion and renders the draft inline for review — the draft is never emailed or transmitted anywhere automatically; sending remains a manual, human action outside this platform's scope, exactly as today.

13.5 — Keycloak Integration Logic

A single AGFZE realm is configured in Keycloak 24 with clients for the Next.js frontend (public, PKCE-enabled) and the backend's resource server. Keycloak's realm is configured with **Microsoft Entra ID as an Identity Provider** (OIDC, brokered) so the realm's own login screen redirects straight through to AGFZE's Microsoft 365 sign-in rather than asking for a Keycloak-specific password — this satisfies the master specification's Keycloak-as-IdP requirement while giving users a single Microsoft SSO experience (Section 2, Stage 3). Realm roles (`purchase`, `sales`, `logistics`, `finance`, `fa`, `approver`, `admin`, `auditor`) are mapped from Entra ID group membership via a Keycloak identity-provider mapper on first login, so role assignment is governed by AGFZE's existing Entra ID group management, not a parallel Keycloak-only user store. NextAuth.js exchanges the resulting Keycloak-issued JWT exactly as in Section 11.2; the backend verifies it exactly as in Section 10.5/12.2. The Keycloak Admin API is used only for the `/admin/users` screen's role-assignment override path (Section 7B), for the rare case a role needs a manual exception outside the group-mapping rule. A FreeMarker custom theme applies AGFZE's Command Centre branding to the Keycloak-hosted portions of the login redirect sequence the user briefly sees.

13.6 — Microsoft Graph API Integration Logic

A dedicated Azure AD app registration (separate from the Keycloak/Entra ID login-brokering above — this one is a pure machine identity, never used for human sign-in) authenticates via the **OAuth2 Client Credentials Flow**: the backend requests a token from `https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token` with `grant_type=client_credentials` and a client secret held in Google Secret Manager, scoped to `https://graph.microsoft.com/.default`. Application (not delegated) permissions are granted and admin-consented once by AGFZE IT: `Mail.Read` (scoped, where Graph supports it, to the specific shared mailbox rather than tenant-wide) for Feature 1, and `Sites.ReadWrite.All` (or a narrower, site-scoped permission if AGFZE's SharePoint configuration allows it) for Feature 8's Excel/SharePoint tracker sync — write access is requested only for the specific tracker workbooks, never opened tenant-wide by default. `graph_service.py` caches the client-credentials token in memory for its lifetime (typically ~60 minutes) and refreshes proactively before expiry. Mailbox capture uses a Graph *change notification* (*webhook*) *subscription* on the shared mailbox's messages, renewed automatically before its ~3-day expiry (TDD-anticipated pattern), with a fallback delta-query poll

every 2 minutes in case a webhook delivery is missed — this dual path is why Feature 1's ingestion function is idempotent on `graph_message_id`. Tracker sync uses the Excel API's `workbook/worksheets/{id}/tables/{id}/rows` endpoints to append/update rows (never “open and save” the file), which avoids clobbering a row a human happens to be editing at the same moment.

13.7 — Fuzzy Matching Logic (rapidfuzz)

Use case	Algorithm	Threshold	Why
Supplier/customer name matching across documents (BR-02)	<code>token_sort_ratio</code>	≥ 90	Tolerates word-order and minor punctuation differences (“Meridian Metals Trading FZE” vs “Meridian Metals Trading, FZE”)
Container number matching (BR-03)	Exact match first; <code>ratio</code> fallback	= 100 exact, ≥ 95 OCR-fallback	Container numbers are structured codes — a fuzzy fallback exists only to catch a single OCR-misread character, never to “guess” a different container
Batch/contract reference matching	<code>partial_ratio</code>	≥ 92	One reference is often a substring of the other (a contract number embedded in a longer invoice reference string)
Duplicate document detection (BR-13)	Content hash (SHA-256) first; <code>token_set_ratio</code> on extracted text as a secondary signal	Hash exact match, or ≥ 97 text similarity	Catches both byte-identical re-sends and a re-scanned/re-saved copy of the same document

Matches at or above threshold are auto-linked; matches in a narrow band below threshold (configurable, default 80–90) are surfaced as a suggested match requiring one-click human confirmation rather than either auto-linking or silently creating a duplicate transaction; anything lower is treated as no match. Thresholds live in `rule_configurations` (Section 9.6), not in code, for the same reason as the tolerance values in Section 4 Feature 6.

13.8 — Background Job Tracking & Transaction State Machine

Every `TradeTransaction` / `Request` moves through a single shared state machine, enforced server-side (no API route can write a state that isn't a legal transition from the current one):

```

Received → Classified → Extraction Pending → Extracted → Matched →
Validation Pending ─(fail)→ Needs Review / Exception Open ─(resolved)→
back to Validation Pending ─(pass)→ Approval Pending Approval Pending
─(approved)→ Approved ─(reject/changes)→ back to the relevant prior
state Approved → Integration Pending → Committed → Closed (any state) →
Cancelled / Duplicate [only via BR-13's duplicate-detection path]

```

Every transition publishes a domain event (persisted to `audit_events`, Section 9.5):

```

RequestClassified, DocumentStored, DocumentClassified,
ExtractionCompleted, TransactionMatched, ValidationPassed /
ValidationBlocked, ExceptionCreated / ExceptionResolved,
ApprovalRequested / ApprovalGranted / ApprovalRejected,
IntegrationJobQueued / Succeeded / Failed, DocumentPackGenerated,
DmsUploadCompleted, ShipmentStatusUpdated, RequestClosed.

```

`BackgroundJob` rows (Section 9.5) track the execution of the work *behind* a transition (an extraction job, an integration job) separately from the transition itself, so a slow or retried job never leaves the transaction's own status field in an ambiguous state — the transaction only moves once its underlying job reaches a terminal outcome.

13.9 — File Storage Abstraction Logic

```

class StorageService(Protocol):
    async def upload(self, key: str, content: bytes, content_type: str) -
    > str: ...
    async def download(self, key: str) -> bytes: ...
    async def delete(self, key: str) -> None: ...
    async def get_signed_url(self, key: str, expires_in: int = 300) ->
    str: ...

class LocalStorageService(StorageService):
    """STORAGE_BACKEND=local – writes under /home/app/uploads (dev
    only)."""

class AzureBlobStorageService(StorageService):
    """STORAGE_BACKEND=azure_blob – production. Container-per-
    environment,
    key layout: {stream}/{yyyy}/{mm}/{uuid}-{sanitised_filename}."""

```

`get_storage_service()` (Section 10.5) returns the implementation selected by `STORAGE_BACKEND`. The database *never* stores a raw filesystem path or a permanent public blob URL — every `storage_ref` column (Documents, DocumentPacks, Reports) stores an opaque key, and every download flows through an authenticated API endpoint that (a) checks the requesting user's role/ownership against the linked transaction, then (b) calls `get_signed_url()` for a short-lived (5-minute) URL rather than exposing the blob container directly.

13.10 — File Security Logic

- **Type whitelist:** PDF, DOCX, XLSX/XLS, CSV, JSON, JPEG/PNG — enforced by magic-byte sniffing (`python-magic`) server-side, never by trusting the client-supplied `Content-Type` header or file extension alone.
- **Size limits:** 25MB per file, enforced client-side (immediate UX feedback) and server-side (authoritative — a streamed size check aborts the upload before the full body is buffered).
- **Filename sanitisation:** the original filename is retained only as display metadata; the actual storage key is a UUID, eliminating path-traversal and collision risk entirely rather than trying to sanitise arbitrary user-supplied filenames.
- **Path construction:** storage keys are built server-side from a fixed template (`{stream}/{yyyy}/{mm}/{uuid}-{safe_name}`) with no user-controlled path segment, closing the path-traversal class of bug at the design level rather than the validation level.
- **Content-type verification:** re-checked at download time too — a file cannot be served with a different `Content-Type` than what was verified at upload, preventing a stored file from later being served in a way that triggers browser content-sniffing exploits.

13.11 — PWA & Service Worker Caching Strategy

SPECIFIED — added scope

Request class	Strategy	Rationale
App shell / static build assets (JS/CSS chunks, fonts, icons)	Precache at SW install, cache-first at runtime	Never changes between deploys of the same build; instant repeat loads
Whitelisted GET reads (dashboard summary, transaction/shipment/exception/approval <i>lists</i> , KPI data)	Stale-while-revalidate	Instant paint from cache, silently refreshed — acceptable staleness for a list view, never for a decision screen
Single-transaction/approval/document detail GET	Network-first, falling back to cache only if the network request fails entirely	A user about to approve something must see the current state whenever a connection exists at all — cache is a fallback for connectivity loss, not a first choice
Every mutating call (POST/PATCH/PUT/DELETE) and every AI/extraction/approval-decision/integration endpoint	Network-only, no background-sync queue	Section 4 Feature 12's explicit governance boundary — nothing here is ever queued to “replay later”
Navigation requests with no network and no cache entry	Serve the precached <code>offline</code> route	Replaces the browser's generic offline error with an on-brand, informative page

Cached API responses are capped at a short TTL (default 15 minutes) enforced by the service worker's cache-expiration plugin, and the entire cache is version-keyed to the deployed build hash so a new release invalidates stale cached responses

automatically rather than requiring a manual cache-clear. Signing out clears all caches and unregisters push subscriptions for that browser (Section 16), since cached screens can contain counterparty names and commercial figures that must not persist on a shared or lost device after logout.

13.12 — Email Notification Logic (SMTP) SPECIFIED – added scope

1. A domain event that maps to a notification rule (Section 13.8's event list — e.g. `ApprovalRequested`, `ExceptionCreated`, report-generation completion) is published on the same in-process event bus every other feature uses.
2. `notification_service.py` resolves the recipient(s) and their `notification_channel` preference (Section 9.1), then calls `email_service.send(template_name, context, to)` for anyone subscribed to email.
3. `email_service.py` renders the named template from `app/email_templates/` using Jinja2 — a server-side templating use that is unrelated to the Document Control table's frontend-templating prohibition, which concerns replacing Next.js with a server-rendered page framework, not rendering an HTML email string — producing both an HTML part and a plaintext fallback part (for accessibility and spam-filter friendliness) inside a `MIMEMultipart("alternative")` message.
4. The message is sent via Python's `smtplib` over an authenticated, TLS-secured (STARTTLS) connection to a configured SMTP relay (`SMTP_HOST` / `SMTP_PORT` / `SMTP_USERNAME` / `SMTP_PASSWORD`, all in Google Secret Manager) — deliberately protocol-level SMTP rather than a third-party client-side email SDK, so delivery never depends on a browser-side integration or an external vendor JS snippet.
5. Every email template shares a common `base_layout.html`: the AGFZE Command Centre wordmark, the ink-navy/copper palette (Section 14.9), a clear one-line summary of the event, a prominent single call-to-action button deep-linking into the relevant screen, and a footer carrying the same AI disclaimer language used in-app (Section 2.5) wherever the email references AI-extracted content.
6. Send outcome (success/failure, timestamp) is written to the triggering `notifications` row's `email_sent_at` field (Section 9.8); a failure is retried up to three times with backoff and, after final failure, surfaces only in the Integration/ notification admin view — it never blocks or reverses the underlying business event.

13.13 — Web Push Notification Logic (VAPID) SPECIFIED – added scope

1. A VAPID key pair is generated once at setup time and stored as `VAPID_PUBLIC_KEY` (safe to expose to the frontend) and `VAPID_PRIVATE_KEY` (Google Secret Manager only, backend-side exclusively).
2. Frontend: after the user grants notification permission, `lib/push.ts` calls `serviceWorkerRegistration.pushManager.subscribe({userVisibleOnly: true, applicationServerKey: vapidPublicKey})` and posts the resulting

- `{endpoint, keys}` to `POST /api/v1/notifications/push-subscribe` (Section 7B), upserted into `push_subscriptions` (Section 9.8).
3. Backend: on a notification-mapped domain event, `push_service.py` uses `pywebpush` to sign and deliver a payload (title, body, deep-link URL, icon) to every active subscription for the resolved recipient(s), authenticating to each browser vendor's push service using the VAPID private key.
 4. The service worker's `push` event handler calls `self.registration.showNotification(...)` so the notification is delivered and displayed by the OS/browser even if the Command Centre tab is closed or the browser itself isn't running in the foreground; a `notificationclick` handler focuses an existing tab or opens a new one at the payload's deep link.
 5. A `410 Gone` (or `404`) response from a push service on delivery attempt is treated as "this subscription is dead" and removes the row from `push_subscriptions` rather than retrying indefinitely.
 6. Exactly as with email (13.12), push delivery is a notification of a decision already recorded elsewhere in the system — it is never itself the mechanism that records or triggers an approval, and its failure never affects the underlying transaction state.

SECTION 14

14 UX Principles

14.1 — Simplicity Over Comprehensiveness

A transaction workspace could show forty extracted fields, twelve rule evaluations and a full document history simultaneously — it never does. Extraction and Validation are the two **Accordion** sections open by default (Section 8, Archetype 1) because they answer “what did the AI find, and is it correct” — the two questions a user actually has first; Matching and History stay collapsed until asked for. The same discipline applies to the Dashboard: four KPI tiles and two charts above the fold, everything else one click away, never all at once.

14.2 — Anxiety Reduction

Given Section 1.6’s finding that this user base already operates in a state of chronic vigilance around financial-consequence errors, every design choice here works specifically against that: a validation checklist that names the exact rule and threshold rather than a bare “invalid,” a progress bar with honest time estimates (“usually 10–30 seconds”) rather than an indefinite spinner, and Framer Motion transitions tuned to feel unhurried and settled (200–300ms eases, no bounce) rather than snappy-for-its-own-sake — snappy reads as “the system moved faster than I could verify it,” which is the opposite of the trust this platform needs to build.

14.3 — Trust Building Through Transparency

Every AI-derived value is visibly badged (never presented indistinguishably from a human-entered value), every confidence score is shown as a number and a colour, every extracted field links back to the exact source page it came from, and the AI disclaimer (Section 2.5) appears on every screen where AI output is shown, not only in a footer. Processing status is always visible, never silent — a user should never wonder “is it doing something?”

14.4 — Decision Guidance Over Information Dumping

The Exception detail screen (Section 7B) doesn’t just show “quantity mismatch” — it shows the specific matrix-defined action (“hold invoice processing and route to Finance/transaction owner for review”) as the visible next step. The Approval screen doesn’t just show data — it leads with the AI-generated plain-language summary of what changed and why this approval is needed, with full data available below for anyone who wants to verify it themselves.

14.5 — Accessibility

WCAG AA colour contrast minimum across every badge/status colour pairing (validated against both the light theme and dark-mode variant); every interactive element is keyboard-navigable by virtue of using Radix UI primitives, which handle focus trapping in **Dialog** / **Sheet** and roving tabindex in **Tabs** / **Accordion** out of

the box; every icon-only button carries an `aria-label`; heading hierarchy follows document order (one `h1` per page, sequential `h2` / `h3`) rather than being chosen for visual size.

14.6 — Responsive Design

Sidebar → Radix `Sheet` drawer below `md`; queue tables → stacked cards below `md` (Section 8, Archetype 2); workspace two-column layout → stacked single column below `lg`, document viewer first; every touch target $\geq 44 \times 44$ px on mobile, specifically because the Approval Queue (Section 5, Stage 9) is a page the HOD is expected to use from a phone.

14.7 — Loading State Design

Shape-matched `Skeleton` loaders, not generic spinners, for anything that renders a known layout (queues, workspaces); a determinate `Progress` bar bound to the polled job `progress` value for anything AI/background-job-backed, with honest status microcopy; the user can always navigate away mid-job and is notified by toast on return.

14.8 — Error State Design

Inline field errors state the fix, not just the failure (“must be within 5% of 125.000 MT — you entered 132.000 MT,” never “invalid value”). A failed background job shows a specific `Alert` with a **Retry** that resumes rather than restarts data entry. Next.js `error.tsx` boundaries are scoped per route segment so a failure in, say, the Shipment widget on the Dashboard does not take down the whole Dashboard.

14.9 — Colour, Typography and Visual Language

Theme is defined as CSS variables consumed by the shadcn/ui Tailwind preset, in both a light and dark variant: an ink-navy primary (`#182338`) for headers/nav, a warm copper accent (`#A75D35`) reserved for primary actions and the active-nav indicator, a steel-blue secondary (`#3E5C76`) for informational/AI-related badges, and a fixed traffic-light triad (green `#2F6B4F` / amber `#95701A` / red `#A23B32`) used *exclusively* for confidence and validation states platform-wide, never repurposed for anything else, so colour always means the same thing everywhere in the product. Typography: Inter or the system UI sans stack for all interface text (dense data screens favour a neutral, highly legible sans over a display font), `lucide-react` for a single consistent icon set, Tailwind’s default spacing scale, and a 6px border radius standard across cards/inputs/buttons for a calm, non-toy-like enterprise feel appropriate to a financial trading tool.

SECTION 16

16 Security and Privacy

16.1 — Authentication Security

- Keycloak 24 as the sole portal identity provider, brokering Microsoft Entra ID (Section 13.5) — AGFZE staff authenticate with their existing corporate credentials and whatever MFA policy Entra ID already enforces; no parallel password store is created.
- OIDC Authorization Code Flow with PKCE between the Next.js frontend and Keycloak; JWT access tokens verified on every backend request via RSA signature check against Keycloak's JWKS endpoint (python-jose + cryptography).
- Refresh tokens are held server-side by NextAuth.js, never exposed to client-side JavaScript; silent refresh occurs within 60 seconds of expiry (Section 11.2).
- Session/idle timeout, account lockout policy and MFA enforcement are configured at the Entra ID/Keycloak layer, inherited from AGFZE's existing corporate security policy rather than re-implemented in this application.
- The separate Azure AD app registration used for Microsoft Graph API access (Section 13.6) is a distinct, least-privilege machine identity with no interactive login capability at all — a compromise of the portal's user-facing auth has no path to this credential, and vice versa.

16.2 — Authorization and Access Control

- Role-based access control with roles sourced from Entra ID group membership via Keycloak's identity-provider mapper (Section 13.5): `purchase`, `sales`, `logistics`, `finance`, `fa`, `approver`, `admin`, `auditor`.
- Every router enforces authorization via `get_current_user` / `get_current_admin_user` (Section 10.5) at the dependency-injection layer, not only via UI hiding — a Sales-only user's API calls physically cannot return Purchase-stream detail, regardless of what the frontend renders.
- Resource-level checks filter every transaction/document/exception/approval query by the caller's permitted stream(s) before any row is returned, never after.
- The `Approved` state transition (Section 13.8) additionally checks that the deciding user's role matches the transaction's required `approver_role` — a Sales user cannot approve a Purchase transaction even if they somehow reached the endpoint.

16.3 — API Security

- CORS restricted to the deployed Next.js frontend's exact origin(s) — never `*` — in every environment above local development.
- slowapi rate limiting disabled in development, enabled in production (Section 10.3); the push-subscription and notification endpoints (Feature 12) are rate-limited like any other authenticated endpoint to prevent subscription-spamming.

- Request size limits enforced ahead of Pydantic validation for file-upload endpoints (Section 13.10).
- Every request/response body validated by Pydantic; every list endpoint's filter/pagination parameters validated by type before reaching the query layer.
- HTTPS enforced by Cloud Run at the platform level; Cloud Armor WAF rules filter malicious request patterns and provide DDoS protection ahead of the application.

16.4 — File Upload Security

Covered in full in Section 13.10 (type whitelist by magic bytes, 25MB limit enforced client- and server-side, UUID-based storage keys with no user-controlled path segment, files served only through authenticated, ownership-checked, short-lived signed URLs — never a direct storage URL, never a web-accessible static path).

16.5 — AI API Security

- Gemini/Vertex AI credentials held exclusively in Google Secret Manager (production) or local `.env` (development); never logged, never returned in any API response, never present in frontend code or the Next.js client bundle.
- All AI calls are server-side only, inside `gemini_service.py` (Section 13.1) — no router or frontend code calls a model API directly.
- User-supplied text (email bodies, document content) is passed to the model as clearly delimited data, with an explicit system-prompt instruction that content inside the data block is never to be treated as an instruction (Section 13.2's prompt-injection defence).
- Model output is Pydantic-validated before storage or display — a malformed or out-of-schema response is treated as a failed extraction (routed to human review), never rendered or persisted as-is.

16.6 — Database Security

- SQLAlchemy ORM parameterised queries throughout — no raw string-interpolated SQL anywhere in the codebase.
- Cloud SQL (PostgreSQL 15) reachable only over private IP / the Cloud SQL Proxy, never a public endpoint; database credentials in Google Secret Manager.
- Automated daily backups with point-in-time recovery; connection pooling via SQLAlchemy's `QueuePool` with a bounded pool size to prevent connection exhaustion under load.
- The graph data layer (Section 9.7) is treated with the same access discipline as the relational store — no open Cypher endpoint is ever exposed; the only access path is the bounded, pre-defined subgraph query behind `GET /transactions/{id}/graph`.

16.7 — Frontend Security

- HTTP-only, Secure, SameSite session cookies managed exclusively by NextAuth.js — never a token in `localStorage` / `sessionStorage`.

- Content Security Policy headers restricting script/style/connect sources to known origins (the API, the Keycloak issuer, the Gemini/Graph endpoints called only server-side so they need not appear in a browser-facing CSP at all).
- `dangerouslySetInnerHTML` avoided platform-wide; the one place raw HTML rendering could arise (a generated document preview) renders via a sandboxed `iframe` pointed at the authenticated PDF endpoint, never by injecting extracted or AI-generated markup into the page DOM directly.
- **Service worker cache scope (Feature 12):** the cache is scoped to the app's own origin only, never stores the `Authorization` header or any token, and is version-keyed to the build hash so it cannot silently serve a previous, possibly-revoked session's data across a deploy; sign-out explicitly clears all Cache Storage entries and unregisters any push subscription for that browser (Section 13.11), since cached screens can contain counterparty names and commercial figures.

16.8 — Audit Logging

- Every workflow transition, approval decision, exception event, override, admin configuration change, and downstream posting is written to the append-only `audit_events` table (Section 9.5) — no update/delete route is ever exposed on it (BR-12).
- Audit records store *metadata* (who, when, what changed, what rule fired) — never the full text of a document or an AI prompt/response, consistent with the confidentiality of the underlying trade data; the extraction/document content itself remains reachable only through its own access-controlled record, one hop away, for anyone with a legitimate need and the role to view it.
- The Audit Explorer (Section 6/7) is the read surface for this table, restricted to Admin/Auditor roles, with CSV export for external compliance review.

16.9 — Environment and Secret Management

Every credential — Keycloak client secret, the Graph API app registration's client secret, SAP/DMS connector credentials, Neo4j credentials, Sentry DSN, SMTP username/password, and the VAPID private key (Feature 12) — lives in Google Secret Manager in production and in a git-ignored `.env` in development, with a fully-documented `.env.example` containing placeholders only. No secret is ever baked into a Docker build layer (injected at container runtime instead) or referenced in a GitHub Actions log. The VAPID *public* key is the one credential in this list that is intentionally exposed to the frontend bundle, since it is designed to be public by the Web Push standard itself.

16.10 — Data Privacy

- A published privacy statement (the `/privacy` page) describes what is collected (trade documents, extracted commercial data, user activity) and why, scoped to AGFZE's own operational need rather than any third-party data-sharing.

- Document/transaction retention follows AGFZE's own record-keeping policy (to be confirmed during discovery, consistent with several BRD data points marked "subject to AGFZE confirmation") — the schema supports a configurable retention/auto-archival job without a structural change, since every table already carries the timestamps such a job needs.
- AI interaction logging (Section 9.1's `ExtractedField.original_ai_value`) is retained specifically because BR-11 requires it for audit — this is a deliberate, documented exception to "minimise everything," not an oversight, since audit defensibility is itself a stated business requirement here (Section 1.2).

SECTION 17

17 Scalability and Infrastructure

17.1 — Cloud Run Scaling

Backend and frontend deploy as two independent Cloud Run services. Backend: min instances 1 (avoids a cold start on the first request of the AGFZE workday), max instances configured for the deal volume the trading desk actually produces (this is a bounded internal user base, not an open consumer product — headroom is set generously but not speculatively), concurrency 40–60 requests per instance given the mix of lightweight CRUD calls and heavier extraction-triggering calls. Frontend: min instances 1, higher concurrency (Next.js request handling is comparatively light). Liveness/readiness probes hit `/health` and `/health/ready` respectively (Section 7B).

17.2 — Database Scaling

Cloud SQL for PostgreSQL 15, sized to the transaction/document volume from BRD §9's rollout rather than over-provisioned upfront; a read replica is added only if/when the Dashboard/Analytics/Reports read load (Features 10–11) measurably contends with the transactional write path, not pre-emptively. SQLAlchemy `QueuePool` sized to Cloud Run's per-instance concurrency so the database never sees more concurrent connections than the application layer can actually generate. Alembic migrations run as a pre-deploy CI step (Section 17.7), never against a live-traffic instance without a maintenance window for anything non-additive.

17.3 — AI API Quota Management

Gemini 2.5 Flash quota and rate limits are tracked per-environment; `gemini_service.py` implements retry with exponential backoff (Section 13.1) and a circuit breaker that, on sustained failure, routes affected requests straight to *Needs Review* rather than queuing indefinitely. `AI_PROVIDER` switching to `vertex_ai` is the designed fallback path if Gemini API-direct quota is exhausted. Usage (call volume, token count, cost) is tracked and alerted on (Section 17.8) against the agreed volume band from Section 15.4.

17.4 — File Storage Scaling

Azure Blob Storage in production, with lifecycle rules moving older, rarely accessed document packs to a cooler access tier after a configurable age, and a retention/deletion policy aligned to Section 16.10's data-retention confirmation. Local filesystem storage in development only, explicitly not used for anything beyond a single developer's machine.

17.5 — Background Job Scaling

Cloud Run Jobs handle multi-document extraction packs, scheduled shipment-status pulls (Feature 9), and scheduled/ad-hoc report generation (Feature 11); Cloud

Scheduler triggers the periodic ones (shipment polling, daily/monthly reports, exception-ageing recompute). Each target-system integration job (Feature 8) runs independently per transaction so one slow/failed SAP posting never blocks a DMS upload or a tracker sync for the same or any other transaction. A dead-letter path captures any job that exhausts its retry budget, surfaced in the Integration Monitor (Section 6/7) rather than silently disappearing.

17.6 — Caching

Dashboard/KPI aggregate endpoints (Feature 10) cache for 30–60 seconds server-side to absorb concurrent dashboard views without re-aggregating on every request. AI analysis results are never re-requested for an already-extracted document — the persisted `ExtractedField` rows themselves are the cache. Next.js built-in caching applies to the handful of static public pages (Section 6). The service worker's client-side caching (Feature 12, Section 13.11) is a distinct, deliberately narrower layer — it exists for offline resilience on a single device, not as a substitute for the server-side caches above.

17.7 — CI/CD Pipeline

GitHub Actions, two pipelines:

- **Backend:** flake8 → black/isort format check → pytest/pytest-asyncio → Alembic migration dry-run against a disposable test database → build multi-stage Docker image → push to Google Artifact Registry → deploy to Cloud Run (staging, then production on a tagged release).
- **Frontend:** ESLint 9 → Prettier check → Vitest/Jest + React Testing Library → Workbox service-worker build step (validates the precache manifest against the current build output, Feature 12) → build Docker image → push to Artifact Registry → deploy to Cloud Run.

Rollback is a Cloud Run traffic-split revert to the prior revision for either service independently; a database migration is only ever forward-compatible with the immediately prior application revision, so a rollback never requires an emergency down-migration.

17.8 — Monitoring and Observability

Sentry on both services for error tracking (including client-side errors surfaced through the `error.tsx` boundary's "Report issue" link, Section 8); structured JSON logs via pythonjsonlogger; Cloud Run's native metrics/logging integration for request latency, instance count and cold starts. Alerts: AI quota approaching the agreed band (Section 17.3), integration job failure-rate spikes (Feature 8), exception-queue ageing breaching a configured threshold (Feature 7), and — for Feature 12 — email/push delivery failure-rate spikes, so a broken SMTP relay or an expired VAPID configuration is caught operationally rather than discovered when the HOD says "I never got notified."

17.9 — Docker and Containerisation

Multi-stage Docker builds for both services (build stage with full toolchain, slim runtime stage), non-root runtime user in both images, Docker Compose for local development bringing up PostgreSQL, Keycloak, the FastAPI backend and the Next.js frontend together; Makefile targets (`make setup` , `make dev` , `make test` , `make lint` , `make migrate` , `make build`) wrap the common developer workflows described in Section 18.1.

SECTION 18

18 Developer Handoff Notes

18.1 — Development Environment Setup

Backend: Python 3.11 + a virtual environment; `pip-compile requirements.in > requirements.txt` then `pip install -r requirements.txt`; copy `.env.example` to `.env` and fill in `DATABASE_URL`, `AI_PROVIDER=gemini_flash`, `GEMINI_API_KEY`, `KEYCLOAK_ISSUER / JWKS_URL`, `AZURE_AD_TENANT_ID / CLIENT_ID / CLIENT_SECRET` (Graph API app registration — a developer-sandbox tenant, never AGFZE's production tenant, for local work), `STORAGE_BACKEND=local`, `VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY` (generate once with `vapid -gen` or the `pywebpush` helper script), `SMTP_HOST / PORT / USERNAME / PASSWORD` (point at a local dev SMTP catcher such as a disposable Mailpit/MailHog container in development so no real email ever sends during local work); start PostgreSQL and Keycloak via Docker Compose; run `alembic upgrade head`; run `uvicorn app.main:app --reload`.

Frontend: Node.js LTS; `npm install`; copy to `.env.local` and set `NEXT_PUBLIC_API_BASE_URL`, `KEYCLOAK_CLIENT_ID / SECRET / ISSUER`, `NEXT_PUBLIC_VAPID_PUBLIC_KEY`; run `npm run dev`. Service-worker registration is skipped automatically in the Next.js dev server (hot-reload and a persistent SW actively fight each other) and only builds/registers in a production build (`npm run build && npm run start`) — this is a standard, deliberate Workbox/Next.js development pattern, not an oversight, and should be called out to any developer confused why offline mode “doesn’t work” in `npm run dev`.

Full-stack local development: a single `docker-compose.yml` brings up PostgreSQL, Keycloak (pre-seeded with a dev realm/roles), the FastAPI backend and the Next.js frontend together. Makefile targets: `make setup` (installs everything, seeds dev data), `make dev` (starts the full compose stack), `make test` (backend + frontend test suites), `make lint` (flake8/black/isort + ESLint/Prettier), `make migrate` (Alembic), `make build` (production Docker builds for both services).

18.2 — Feature Priority and Development Phases

Aligned to BRD §9’s phased implementation plan rather than an arbitrary internal split:

Phase	Focus
Phase 1 — Foundation	FastAPI app factory + routers; PostgreSQL + SQLAlchemy 2.0 async models (Section 9.1–9.6) + Alembic; Keycloak deployment brokering Entra ID; NextAuth.js integration; JWT verification; Next.js App Router + Tailwind/shadcn scaffold; Docker Compose; health checks; Sentry on both services; Azure AD app registration for Graph API (least-privilege, read-only to start).
Phase 2 — Core Workflow	Email intake & classification (Feature 1); document classification/extraction engine (Feature 2); Purchase transaction processing (Feature 3) as the first fully governed end-to-end path, since Purchase is the higher-volume, better-

	documented workflow; Validation/Business Rule Engine (Feature 6); Exception & Approval workflow (Feature 7); local-filesystem storage service.
Phase 3 — Full Workflow Coverage	Sales transaction processing + draft generation (Feature 4); Shipment & Cargo Tracking (Feature 9); Tracker/SAP/DMS Integration Hub (Feature 8) — this is the phase where the exact SAP object/API and DMS endpoint contract (flagged as TBD in TDD §11) must be finalised with AGFZE IT before Feature 8 can be completed, not before; Dashboard & Workbench (Feature 10); an explicit FA-discovery workshop with AGFZE (per BRD §6.4) followed by FA Transaction Processing (Feature 5) configuration.
Phase 4 — Reporting, PWA & Production Readiness	Automated Reporting Engine (Feature 11); Progressive Web App, offline resilience, SMTP email and Web Push (Feature 12); Azure Blob Storage cutover; Cloud SQL production provisioning; Cloud Run deployment for both services; Cloud Armor; Google Secret Manager for every credential in Section 16.9; slowapi rate limiting turned on; GitHub Actions CI/CD (Section 17.7); Playwright/Cypress E2E coverage; UAT against BRD §9's sign-off criteria; production go-live and hypercare.

18.3 — Critical Implementation Decisions

- **Keycloak brokering Entra ID, not a parallel credential store** — satisfies the specified stack's Keycloak mandate while giving AGFZE staff single sign-on with the Microsoft 365 credentials they already have; do not “simplify” this later by pointing NextAuth.js directly at Entra ID and dropping Keycloak, since that would silently drop the specified stack's OIDC-broker requirement and Keycloak's role-mapping/theming layer.
- **A dedicated Azure AD app registration for Graph API, separate from Keycloak's Entra ID brokering** — one is a human-login broker, the other is a machine identity for mailbox/SharePoint access; do not collapse these into one registration, since their permission scopes and blast radius on compromise are intentionally different.
- **A graph database layer alongside PostgreSQL** — added because TDD §10.1/10.3 names cross-entity traceability as a first-class requirement the reference stack's relational-only design doesn't cover; PostgreSQL remains the transactional system of record, the graph store is a derived, rebuildable projection — do not let any write path treat the graph store as authoritative.
- **FastAPI over Flask/Django** — native async, automatic OpenAPI docs, Pydantic validation, dependency injection, and the async performance profile this workload (many concurrent AI/API calls, not CPU-bound compute) actually benefits from.
- **Next.js App Router over Pages Router** — Server Components for the read-heavy list/dashboard pages, nested layouts for the protected-area shell, streaming for the document-viewer-heavy workspace pages.
- **PostgreSQL over SQLite** — concurrent multi-user writes from day one, JSONB for the FA stream's undetermined field set (Section 9.2) and configuration tables (Section 9.6), Cloud SQL's managed backup/HA story.
- **python-docx/PyMuPDF for binary generation, Gemini for content only** — deterministic, template-based, reliable binary output; the model proposes

structured clause content, it never emits the DOCX/PDF bytes directly (Section 13.4).

- **SMTP over a client-side email SDK, and Web Push over any third-party push vendor** (Feature 12) — both were explicit, later-added requirements; implementing them as protocol-level SMTP and standards-based VAPID/Web Push keeps delivery entirely under the platform's own infrastructure and Secret Manager-held credentials, with no external vendor dependency or client-side script to audit.
- **Offline mode is read-only** (Feature 12) — do not extend the service worker's background-sync capability to queue approval, submission or extraction requests; this is a deliberate governance boundary (Section 2.3, Section 4 Feature 12), not a missing feature to “finish later.”
- **Rate limiting is environment-controlled** — off in development for a frictionless inner loop, on in production, exactly as specified.

18.4 — UX Expectations for Developers

- Every form shows inline validation errors matching the exact fix needed (Section 14.8), never a bare “invalid.”
- Every AI-backed or background-job-backed action shows a shape-matched skeleton or a determinate progress bar with honest status microcopy (Section 14.7) — never an indefinite spinner with no context.
- Every screen displaying AI-extracted or AI-generated content carries the standard disclaimer (Section 2.5) — this is a shared `AIDisclaimerBanner` component, not copy-pasted text, so its wording can only ever change in one place.
- `react-hot-toast` for success/error notifications, auto-dismiss after 5 seconds, never used for something the user must act on immediately (those are inline `Alert`s or blocking dialogs instead).
- Every file download is served through an authenticated API endpoint (Section 13.9/13.10) — never a direct storage URL, even temporarily during development.
- Every list/queue has a designed empty state (Section 8); every route segment has its own error boundary.
- Keyboard navigation and focus management come from Radix UI primitives — do not replace a Radix `Dialog` / `Select` with a hand-rolled equivalent that loses this for free.
- Mobile navigation uses the Radix `Sheet` pattern (Section 8); the Approval Queue and Approval Decision screens in particular must be fully usable one-handed on a phone (Section 5, Stage 9; Feature 12's install/push flow exists specifically to make this realistic).

18.5 — Testing Requirements

- **Backend unit/integration (pytest, pytest-asyncio):** service-layer tests with mocked `gemini_service` / Graph/SAP/DMS calls; database tests against a disposable test database; Pydantic schema validation tests; state-machine transition tests (Section 13.8) explicitly asserting that illegal transitions —

especially anything attempting to reach `Committed` without passing through `Approved` — are rejected.

- **Backend API tests:** `AsyncClient`-based endpoint tests covering every router in Section 7B, authentication/authorization edge cases (wrong role, expired token, missing token), and file upload validation (Section 13.10).
- **Frontend unit tests (Vitest/Jest + React Testing Library):** component rendering, the `useJobStatus` / `useOnlineStatus` hooks, Zod form validation.
- **E2E (Playwright and/or Cypress):** the full Section 5 user journey per persona (sign-in through Stage 8); a document-upload-to-approval-to-integration happy path; an exception-creation-and-resolution path; an offline-navigation test asserting the `/offline` fallback renders and that a mutating action is correctly blocked rather than silently queued (Feature 12); a push-notification-permission and subscription-registration test.
- **Code quality gates:** black + isort + flake8 (backend), ESLint 9 + Prettier (frontend), all enforced in CI (Section 17.7) — a PR that fails any of these does not merge.

18.6 — Deployment Considerations

- HTTPS via Cloud Run — no separate certificate management.
- Cloud SQL Proxy for secure connectivity; Google Artifact Registry for both service images; Google Secret Manager for every credential in Section 16.9, including the VAPID key pair and SMTP credentials added for Feature 12.
- Cloud Armor WAF rules; Alembic migrations as a pre-deploy CI step; Cloud Run's native rolling-update/traffic-split mechanism for zero-downtime, independently-reversible deploys of the two services.
- Domain/DNS and the production Keycloak deployment (Cloud Run or dedicated compute) provisioned ahead of UAT, not at the end.
- Azure Blob Storage container provisioning and access-key rotation policy; the production Azure AD app registration for Graph API scoped and admin-consented by AGFZE IT specifically (BRD §3's IT/SAP/DMS/M365 stakeholder group) before Feature 1/8 can go live against real AGFZE mail/SharePoint data.
- A production SMTP relay (or AGFZE's own outbound mail relay) configured and its sending domain's SPF/DKIM records verified before Feature 12's email notifications go live, so AGFZE-branded emails don't land in recipients' spam folders.
- Sentry project setup, Cloud Scheduler jobs, and the monitoring/alert thresholds from Section 17.8 all configured before go-live, not discovered missing during hypercare.

18.7 — Known Limitations and Accepted Tradeoffs

- Gemini API latency is real and is why every AI-backed action is a background job with polling, not a synchronous request — this is a deliberate mitigation, not a workaround for a problem that should instead be “fixed” by making the call synchronous.

- Cloud Run cold starts are mitigated by a minimum-instance-count of 1 on both services, at a small standing cost, rather than accepted as user-facing latency.
- Keycloak adds real infrastructure and operational surface beyond “just use Entra ID directly” — accepted because it is the specified stack’s identity layer and because it gives a single abstraction point if AGFZE ever federates a second upstream IdP.
- SAP/DMS integration cannot be fully finalised until AGFZE confirms the exact SAP object/API surface and the DMS’s endpoint contract (TDD §11) — Section 9.5’s schema is deliberately generic (`target_system` , `external_reference` , `failure_reason`) so this remains a connector-implementation task, not a schema migration, once confirmed.
- FA-specific fields, tolerances and mandatory documents are genuinely undetermined at PRD time (BRD §6.4) — modelled as JSONB (Section 9.2’s `FaLeg.extra_fields`) rather than guessed at, so the base schema ships without blocking on a discovery workshop that hasn’t happened yet.
- Offline mode (Feature 12) is intentionally read-only and time-boxed (15-minute cache TTL) — this is a usability improvement for connectivity gaps, not an offline-first architecture, and should not be marketed to AGFZE users as “works fully offline.”
- iOS Safari’s more limited PWA/push support (manual “Add to Home Screen,” Web Push support only in reasonably recent iOS/Safari versions) means the HOD’s iPhone experience may lag slightly behind an Android/desktop install until Apple’s own platform support catches up further — the instructional-dialog fallback (Section 4 Feature 12) exists specifically to handle this gracefully rather than silently failing.
- Cloud SQL’s always-on cost versus a serverless alternative is accepted for the consistent low-latency connection pooling a synchronous validation/approval workflow needs.

18.8 — What Must Never Be Done

- Never expose an API key, client secret, VAPID private key, or SMTP credential in frontend code, a Git repository, a Docker build layer, or a log line.
- Never let any code path reach the `Committed` transaction state without having passed through a recorded `Approved` decision (BR-10) — this includes any future “convenience” bulk-action feature.
- Never queue a mutating action (approval, submission, draft generation, field correction) in the service worker’s background sync for later replay — offline is read-only, full stop (Section 4 Feature 12, Section 13.11).
- Never use `dangerouslySetInnerHTML` without explicit security review; never store an uploaded file under a web-accessible static path.
- Never allow unauthenticated access to a transaction, document, report, or downloaded file — every download resolves a short-lived signed URL server-side after an ownership/role check.

- Never use OpenAI GPT, spaCy, BERT, Tesseract, Google Vision, or Google Translate API as the primary AI/NLP/OCR solution in place of the specified Gemini-based pipeline.
- Never make a blocking, synchronous Gemini/Vertex AI call inside a route handler for anything beyond the smallest single-document case — use a background job.
- Never hard-code a tolerance, mandatory-document list, or approval threshold in application code — these are configuration rows (Section 9.6) precisely so a business-rule change is never a deployment.
- Never disable CORS or set allowed origins to `*` in any environment above local development.
- Never skip Pydantic validation on a request or response schema, “just for an internal admin endpoint.”
- Never query for a user-owned or stream-scoped resource without filtering by the caller’s role/stream at the query layer.
- Never deploy without a passing test suite and lint/format checks (Section 17.7’s CI gates are non-optional).
- Never store a session token in `localStorage` / `sessionStorage`, and never let the service worker cache an `Authorization` header.
- Never bypass Keycloak/Entra ID for authentication with a custom password store.
- Never log full AI prompt/response text, full document content, or full email body into the audit log — metadata only (Section 16.8).
- Never use SQLite in production; never commit a `.env` file.
- Never send an email or push notification that itself constitutes or triggers the approval action — notifications inform about a decision recorded elsewhere; they are never the recording mechanism.